

РУСЕНСКИ УНИВЕРСИТЕТ “АНГЕЛ КЪНЧЕВ”

Факултет: Електротехника, електроника и автоматика

Катедра: Компютърни системи и технологии

ДИПЛОМЕН ПРОЕКТ

Дипломант:

Георги Живков Чапъров

Фак. No 223025

РУСЕ

2026 г.

РУСЕНСКИ УНИВЕРСИТЕТ “АНГЕЛ КЪНЧЕВ”

Факултет: Електротехника, електроника и автоматика

Катедра: Компютърни системи и технологии

ДОПУСКАМ ДО ЗАЩИТА

Ръководител на катедра КСТ:

/ проф. д-р Цветозар Георгиев /

ОБЯСНИТЕЛНА ЗАПИСКА НА ДИПЛОМЕН ПРОЕКТ

ТЕМА:

Класификация на фазите на ръчен монтаж чрез
мултимодални сигнали от проследяване на движенията на
очите и главата

Дипломант:

/ Георги Чапъров /

ОДОБРЯВАМ:

Ръководител:

/ Prof. Marco Porta /

Консултант:

/ Assoc. Prof. St. Smrikarova /

РУСЕ

2026 г.

РУСЕНСКИ УНИВЕРСИТЕТ “АНГЕЛ КЪНЧЕВ”

Факултет: **Електротехника, електроника и автоматика**

Катедра: **Компютърни системи и технологии**

РЕГИСТРИРАЛ:

Орг.фак.к-я:

/ М. Дечева /

УТВЪРЖДАВАМ:

Ръководител на катедра КСТ:

/ проф. д-р Цв. Георгиев /

ЗАДАНИЕ ЗА ДИПЛОМЕН ПРОЕКТ

На студента: **Георги Живков Чапъров**, Фак. No: 223025

Специалност: **Компютърни системи и технологии**

1. Тема:

„Класификация на фазите на ръчен монтаж чрез мултимодални сигнали от проследяване на движенията на очите и главата“

2. Изходни данни:

Модел за машинно обучение за класифициране на фазите за производство на поточна линия.

3. Съдържание на обяснителната записка:

- Анотация
- Увод
- Обзор на съществуващите решения. Изводи. Цел и задачи.
- Проектиране на софтуер за класифициране на данни.
- Проектиране на модел за машинно обучение.
- Основни резултати. Изводи и препоръки.
- Използвана литература
- Приложения

4. Съдържание на графичната част:

- UML диаграми.
- Екранни снимки.

Дата на задаване: 22.02.2026

Срок за предаване: 31.08.2026

Дипломант:

/ Георги Чапъров /

Ръководител:

/ Prof. Marco Porta /

Консултант:

/ Assoc. Prof. St. Smrikarova /

Table of contents

1. List of figures.....	6
2. List of tables.....	7
3. Abbreviations used.....	8
4. Abstract.....	9
5. Introduction.....	10
5.1. Basic concepts in artificial intelligence, machine learning, and deep learning.....	10
5.2. The characteristics of the gaze.....	10
5.3. Gaze tracking.....	11
5.4. Classification of assembly line phases.....	11
5.5. Project objective.....	12
6. Overview of existing solutions.....	13
6.1. Video analysis-based approaches.....	13
6.2. Approaches based on eye tracking.....	13
6.3. Approaches using a combination of multiple sensors.....	13
6.4. Classification models used.....	14
6.5. Analysis of the solutions considered.....	14
6.6. Conclusions.....	14
7. Description of experiment and data collection hardware.....	16
7.1. Structure and method of conducting the experiment.....	16
7.2. NEON from Pupil Labs.....	16
8. Data classification software.....	18
8.1. Requirements and architecture.....	18
8.2. Choosing a programming language and environment.....	20
8.3. Implementation of the software system.....	21
8.3.1. Description of the program modules.....	21
8.3.2. Structure and organization of the programming interface.....	24
8.4. Input data format.....	28
8.5. Output data format.....	28
9. Development environment, remote access and home server.....	29
9.1. Creating a Docker image.....	29
9.2. Creating a compose.yml file.....	30
10. Creating and training the model.....	34
10.1. Input data.....	34
10.2. Used libraries.....	35

10.3. <i>Data pre-processing</i>	36
10.4. <i>Data windowing</i>	37
10.4.1. <i>Windowing procedure</i>	39
10.5. <i>Model architectures</i>	39
10.5.1. <i>CNN</i>	40
10.5.2. <i>LSTM</i>	40
10.5.3. <i>CNN-LSTM</i>	41
10.5.4. <i>LSTM-CNN</i>	41
10.5.5. <i>CNN-LSTM-Fusion</i>	41
10.5.6. <i>CNN-Transformer</i>	41
10.5.7. <i>MLP</i>	42
10.6. <i>Training methodology</i>	42
10.6.1. <i>Experimental scenarios</i>	42
10.6.2. <i>Dataset splitting</i>	43
10.6.3. <i>Hyperparameter optimization</i>	44
10.6.4. <i>Final model training and evaluation</i>	46
10.6.5. <i>Reproducibility</i>	47
11. <i>Main results</i>	50
11.1. <i>Final model configuration</i>	50
11.2. <i>Comparison between models</i>	51
11.3. <i>Subject-dependent results</i>	53
11.4. <i>Subject-independent results</i>	56
11.5. <i>Analysis of the results</i>	59
12. <i>Conclusions, Limitations and Recommendations</i>	65
12.1. <i>Conclusions</i>	65
12.2. <i>Limitations</i>	65
12.3. <i>Recommendations</i>	67
13. <i>References</i>	69
14. <i>Applications</i>	71

1. List of figures

Figure 1. Functional requirements.

Figure 2. Component diagram.

Figure 3. Overview of the application interface.

Figure 4. Timeline widget.

Figure 5. Categorized segment drop-down menu.

Figure 6. “Skip to” input section.

Figure 7. Overlay settings.

Figure 8. Event skipping buttons.

Figure 9. Categorizing buttons.

Figure 10. Overall subject-dependent model comparison.

Figure 11. Overall subject-independent model comparison.

Figure 12. Subject-level accuracy comparison for the subject-dependent experiment.

Figure 13. Per-class F1-score comparison for the subject-dependent experiment.

Figure 14. Subject-level accuracy comparison for the subject-independent experiment.

Figure 15. Per-class F1-score comparison for the subject-independent experiment.

Figure 16. Accuracy decrease between subject-dependent and subject-independent experiments.

Figure 17. Accuracy compared with model complexity – subject-dependent.

Figure 18. Reliability of the subject-dependent models.

Figure 19. Reliability of the subject-independent models.

2. List of tables

Table 1. Main used libraries for creating and training the models

Table 2. Standard-library modules used for creating and training the models

Table 3. Trials in the Optuna study

Table 4. Initial hyperparameter search space

Table 5. Saved file and there description

Table 6. Versions of the main software components

Table 7. Versions of the main software components

Table 8. Final parameters used for training the models in the subject-dependent experiment

Table 9. Final parameters used for training the models in the subject-independent experiment

3. Abbreviations used

VPN – Vitrual Private Network

CSV – Comma Separated Values

AI – Artificial Intelligence

ML - Machine Learning

DL - Deep Learning

CNN - Convolutional Neural Networks

LSTM - Long Short-Term Memory

IMU - Inertial Measurement Units

EMG - Electromyography

SVM - Support Vector Machines

k-NN - k-Nearest Neighbors

GRU - Gated Recurrent Unit

LAN – Local Area Netlork

YAML - YAML Ain't Markup Language (Yet Another Markup Language)

TCP - Transmission Control Protocol

DNS - Domain Name System

IP - Internet Protocol

DDNS - Dynamic Domain Name System

LTS – Long Term Support

HTTPS - Hypertext Transfer Protocol Secure

MLP - Multilayer Perceptron

LOSO - Leave-One-Subject-Out

GELU - Gaussian Error Linear Unit

4. Abstract

The subject of the thesis is the modeling and training of a model that can classify the individual actions of a person working on an assembly line, using only data from glasses that track eye and head movement.

A LEGO set was used to simulate an assembly line. Participants in the experiment wore glasses to track their eyes and head movements. During the experiment, participants followed the steps in the LEGO set's guide book while their actions were recorded. In addition to various types of movements, the glasses recorded the workspace on which the participant assembled the LEGO set. The total number of participants was 27, but due to technical problems, two of the recordings were unusable.

Software has been developed to classify the obtained data. When launching the software, the user can select a folder with the data of a given participant. The software can play the recording on the work area, overlaying the point at which the participant was looking at the given moment on the played recording.

The VS Code text editor with multiple modules and the Python programming language were used to develop the classification software. The PyQt6 library was used for the graphical user interface. The NumPy library was used for interpolation between two points of the gaze data of the participant in the experiment, as well as the pandas library for loading and saving the classified data in a file with the CSV extension.

For data processing, model creation and training, the Python programming language was used, as well as a self-hosted instance of JupyterLab, running in a Docker container based on the image "nvidia/cuda:12.1.1-runtime-ubuntu22.04". The instance is located on a home computer and is accessible via the local network. The local network was accessed remotely via a locally hosted virtual private network (VPN). As a backup option in case of a problem with the VPN, a connection was established via AnyDesk, as well as a self-hosted instance of Sunshine and Moonlight.

Several deep learning architectures were evaluated for classifying three activities: looking at the guide book, assembling, and searching for pieces. In the subject-dependent experiment, the best model achieved an accuracy of **76.87%**, while in the more difficult subject-independent experiment, the best result was **52.84%**. These results show that the collected eye and head movement data can be used for activity classification, but also that generalizing the models to previously unseen participants remains a significant challenge.

5. Introduction

Manual assembly on an assembly line is still widespread, despite the high level of automation in various industries. The prolonged performance of the same type of repetitive operations can lead to fatigue, reduced concentration and an increased likelihood of making errors due to inattention.

The classification of the phases that workers go through on an assembly line allows analyzing the duration of individual operations, the time spent searching for parts and tools, as well as the way the work process is performed.

By analyzing this information, the company can optimize the layout of the workspace, improve the sequence of operations, or, if necessary, change individual stages of the production process. Such optimizations can lead to a reduction in the risk of human error, improved ergonomics of the workplace, and limiting defects in the final product.

One approach to analyzing worker actions is the use of eye-tracking technologies that provide information about the visual attention of the worker during the execution of assembly operations.

Before presenting the proposed approach for classifying the phases that workers go through on an assembly line, it is necessary to consider several basic concepts on which this study is based.

5.1. *Basic concepts in artificial intelligence, machine learning, and deep learning*

Artificial Intelligence (AI) is a field of computer science that aims to create systems capable of performing tasks that normally require human intelligence. Such tasks include object recognition, information analysis, decision-making, and discovering patterns in large amounts of data.

Machine Learning (ML) is a subset of artificial intelligence that allows computer systems to learn from data without being explicitly programmed for each individual task. Instead of being taught rules, machine learning models detect patterns and relationships in given examples and then use what they have learned to make predictions or classifications on new data.

Deep Learning (DL) is a specialized approach to machine learning that uses artificial neural networks with multiple successive layers. These models are capable of automatically extracting complex features from large amounts of data, making them suitable for tasks such as image processing, object recognition, behavior analysis, and time series processing.

5.2. *The characteristics of the gaze*

In this thesis, gaze is one of the main sources of information for training the model. It can be considered as a source of information about the intentions of a person.

Gaze movements are usually divided into three main types, which are characterized by the speed of eye movement and the duration of fixation.

Fixations - Fixations are periods in which the gaze is relatively stable and directed at a specific point for a certain interval of time. During fixation, the main part of visual information processing takes place. [1]

Saccades - Saccades are rapid, ballistic eye movements between two fixations. During a saccade, visual information is limited, and their main function is to move the gaze between different objects or areas. [1]

Smooth pursuits - Smooth pursuit is an eye movement in which the gaze follows a continuously moving object. This type of movement is used when observing dynamic scenes. [1]

5.3. Gaze tracking

It is important to define what eye tracking technology is and how it works. This is because it is key to the development of the thesis.

An eye tracking device is any device that tracks various signals from the eyes such as movements, pupil size, blinks, and others. [31]

They are divided into two main types, namely invasive, which use sensors attached directly to the eye, and non-invasive, which use sensors placed around the eye or so that it is in direct view. Non-invasive is further divided into electro-oculography, which uses electrodes placed in close proximity to the eye, and video-oculography, which uses cameras that capture the eye. [31]

Eye tracking using video capture can be further divided into two types. [31]

Remote eye capture - cameras are most often placed behind or at the level of the work area. They are mainly used to capture the gaze in front of a computer screen and generally in most cases where the participant is sitting largely still. [31]

Eye tracking with attached camera - Cameras are worn, most often through glasses, allowing for a more dynamic work environment. Very often, they also include a camera that captures the participant's work area. [31]

Eye tracking devices typically measure several key metrics, including:

- Gaze direction – this is a fundamental parameter used to identify and classify eye movements, such as fixations, saccades, and smooth pursuit. Gaze direction analysis can provide additional information about cognitive, emotional, physiological, and neurological processes.
- Pupil size and reactivity – through the method of pupillometry (measuring pupil parameters), these indicators can be used to study various cognitive functions, including emotional responses, language processing, perception, and memory. [24][27]
- Blink rate – the number of blinks over a period of time can serve as a reliable indicator of cognitive activity related to learning processes and goal-directed behavior. [28]
- Eye state (open or closed) – tracking whether the eyes are open or closed allows for the assessment of physiological states such as alertness, fatigue, or sleepiness. [29]

5.4. Classification of assembly line phases

The classification of the phases that a worker goes through when performing assembly operations is a process of dividing the work cycle into separate stages with a clearly defined purpose. Each stage can be associated with a specific action, such as

picking up a component, using a tool, performing an assembly operation, or checking the work performed.

Traditionally, the analysis of work processes in an industrial environment is carried out by observing the worker and using video recordings of the workplace. Through subsequent analysis of the video materials, the moments of start and completion of individual operations can be determined, as well as the time required for their execution. This approach provides detailed information about the worker's behavior, but requires a significant amount of video data and human intervention in the analysis.

5.5. *Project objective*

This thesis examines an approach to classifying the phases of an assembly line, in which data related to the movement of the eyes and head of the worker are used instead of full video recordings. The main goal of this approach is to reduce the amount of information required for processing, which can lead to faster analysis and lower requirements for computational resources.

The use of data from the gaze and head movements provides an opportunity to extract information about the visual attention and intentions of the worker, while preserving the main characteristics of the work process. This allows the development of more effective automatic analysis systems that can be used to optimize production processes, improve ergonomics and reduce the likelihood of human errors.

6. Overview of existing solutions

6.1. Video analysis-based approaches

Much of the existing research on assembly process analysis uses workplace video footage as the primary source of information. In this approach, one or more cameras monitor the worker and their workspace, and computer vision methods are used to determine which operation is being performed at any given time.

Modern systems use deep neural networks to automatically extract spatial and temporal features from the video. Combinations of convolutional neural networks (CNN) and Long Short-Term Memory (LSTM) are often used, as well as Transformer-based architectures that allow modeling the sequence of actions over time. Such models achieve high accuracy in recognizing individual stages of assembly and allow tracking the sequence of performed operations. [2]

The main advantage of video analysis is the availability of rich visual information. In addition to the worker's movements, the tools used, individual components and their location can be recognized. This allows the construction of systems that not only classify the phases of assembly, but also detect missed or incorrectly performed operations. [26]

A disadvantage of this approach is the need to process a large volume of video data. This increases the computational requirements and creates difficulties for real-time use. In addition, video recordings raise issues related to the protection of personal data and the privacy of workers.

6.2. Approaches based on eye tracking

In recent years, there has been a growing interest in the use of eye tracking technologies in the analysis of assembly operations. The basic idea is that the direction of gaze provides information about the visual attention and intentions of the worker even before the corresponding action is performed.

In such systems, data from the eye tracking device is used to determine the object to which the operator's attention is directed, as well as to analyze the sequence of observed elements. This allows for predicting the next actions, assessing cognitive load, and detecting deviations from the standard workflow. [30]

Some modern solutions combine eye tracking with worker assistance systems that display instructions or mark the necessary components in real time. This reduces the likelihood of errors and speeds up the assembly process. [2]

Despite its advantages, the use of visual data alone is not always sufficient to unambiguously determine the operation being performed. Different assembly actions may be accompanied by similar patterns of visual attention, which is why additional sensors are often used.

6.3. Approaches using a combination of multiple sensors

In order to increase reliability, multimodal systems are increasingly being used that combine information from several different types of sensors. In addition to visual data, inertial measurement units (IMU), RGB or RGB-D cameras, force sensors, electromyography (EMG) and other sources of information can be used.

Combining data allows compensating for the limitations of individual sensors. For example, IMU provides accurate information about head or limb movements, while eye tracking provides information about the visual attention of the worker. [30]

A number of publications use precisely the combination of gaze and video, which allows for simultaneous recognition of assembly operations and tracking of the operator's attention. The results obtained show higher accuracy compared to systems using only one type of data. [2]

A disadvantage of multimodal solutions is the greater complexity of the system. Data synchronization between different devices is required, as well as significantly more complex processing during model training.

6.4. *Classification models used*

The choice of machine learning model has a significant impact on the accuracy of assembly phase recognition.

In earlier studies, classical machine learning algorithms such as Support Vector Machines (SVM), Random Forest and k-Nearest Neighbors (k-NN) were widely used, in which manually defined features are extracted from the signals in advance.

With the development of deep learning, neural networks are increasingly used, which automatically extract the necessary features. Convolutional neural networks (CNN) are suitable for extracting local patterns in signals, while Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) model dependencies in time sequences. In recent years, Transformer architectures have been increasingly widely used, which allow for efficient modeling of long-term dependencies and often achieve better results when analyzing sequential data. [2]

6.5. *Analysis of the solutions considered*

The approaches considered show that currently the most common systems for the analysis of assembly activities use video data as the main source of information. These solutions achieve high accuracy due to the rich visual information, but require significant computational resources and processing of large volumes of data.

Systems using eye tracking offer the possibility of analyzing the visual attention and intentions of the worker with a significantly smaller volume of information. However, eye data alone are not always sufficient to reliably distinguish between individual assembly operations.

The highest results are usually achieved by combining several types of sensors. However, most existing solutions use video recordings as a mandatory component of the system, which increases the complexity and required computational resources.

6.6. *Conclusions*

The review shows that existing solutions increasingly use deep learning methods to automatically recognize individual stages of the assembly process. The most common systems rely mainly on video data, and in many cases they are combined with information from eye tracking devices and other sensors.

Despite the high accuracy of these solutions, the need to process video recordings leads to an increase in computational complexity, the volume of stored data and the emergence of issues related to the protection of personal data.

Therefore, there is interest in developing systems that use less information, while maintaining high accuracy in classifying assembly phases. The present thesis follows this direction, exploring the possibility of classifying the phases of the assembly process using only eye tracking and IMU data, without the need to analyze full video recordings.

7. Description of experiment and data collection hardware

7.1. Structure and method of conducting the experiment

Due to the impossibility of collecting data in a real assembly line environment, the decision was made to use a LEGO set to be assembled by the participants in the experiment. The set used is from the LEGO 3-in-1 series, and its name is “Space Shuttle”. [11] The set provides the opportunity for three different buildings, the one used here being a spaceship with a total number of parts – 72.

Before the start of the experiment, each participant together with the organizer of the experiment must sign a contract identical to Appendix A. The contract is prepared and signed in two identical copies – one for each party.

Each participant begins the experiment with all the necessary parts, as well as the guide book gathered in front of them. The experiment ends when the participant has finished building, or when the participant no longer wishes to continue the experiment. If the participant refused to complete the experiment and the recording was less than ten minutes long, the recording was discarded.

The total number of participants was 27. Several participants expressed a desire to end the experiment prematurely, but in each of these cases, the recordings were longer than ten minutes. Due to technical problems with the glasses, two of the recordings were lost, and the only information from these participants was the raw data about their gaze. The loss of the recording prevented the classification of this data.

The total number of recordings used to train the model was 25.

To record the participants' actions, NEON eye-tracking glasses from Pupil Labs were used. [17] The creators of the glasses say that they do not require pre-calibration. Therefore, no such calibration is provided before the start of each experiment.

The choice of eye-tracking glasses, rather than a stationary remote tracking system, is due to their ability to register both gaze movement and head position. This makes them more suitable for application in assembly line conditions, where the orientation of the worker's head and body changes dynamically, and the use of multiple external sensors around the workspace would significantly increase the required resources and system complexity.

7.2. NEON from Pupil Labs

Pupil Labs was founded in Berlin, Germany in 2014 and is engaged in the development of software and hardware for eye tracking. Pupil Labs' latest product is the NEON eye tracking glasses. [16]

NEON is a frame of glasses with a built-in module between the two lenses.

The glasses have a camera that looks forward and records what the person wearing them sees. The resolution of this camera is 1600px by 1200px and operates at a frequency of 30Hz. The horizontal viewing angle is 103°, the vertical 77°, and the horizontal 128°. [19]

The glasses have two infrared sensors, each of which tracks one eye. The resolution of one of them is 192px by 192px, and the operating frequency is 200Hz. [19]

Another important feature is the inertial measurement module. It has an accelerometer, magnetometer and gyroscope. Thanks to it, head movements are recorded at a frequency of 110Hz. [19]

Another thing that the glasses have is two stereo microphones, but they were not used during data collection. [19]

NeoNet is the name of the machine learning model that is fed data from the infrared cameras. It is he who returns information about gaze, blinks, pupil diameter, orientation and position of the eyes and eyelid opening. [20]

The glasses have a USB-C 2.0 port for charging and transmitting data in real time to the mobile application – Neon Companion [19]

Neon Companion is a mobile application that allows you to watch a recording while it is being recorded. In addition, you can also see the position of the eyes, as well as the recordings from the cameras that capture them. After the recording is finished, the application allows it to be uploaded to Pupil Cloud. [18]

Pupil Cloud allows for the storage of recordings in one place, as well as their further processing and visualization. [18]

8. Data classification software

After collecting the data, it needs to be categorized. Part of the data is expected to be categorized by hand, and after collecting a sufficiently large number of categorized data, a machine learning model is to be created that, with sufficient accuracy, manages to categorize the data from a single record, and the user only has to correct the errors. For this purpose, an application is needed that allows not only the easy addition of events, but also their editing.

When searching for existing solutions, no one was found that would allow easy modification of the data in the required format. Most of the existing solutions offer only paid versions, many of which are cloud-based services. In order to reduce the risk associated with uploading sensitive information (such as the voice and eye data of the experiment participants) to external cloud storages, as well as in order to reduce costs, a decision was made to develop a specialized application for data classification.

The developed application is entirely for personal use and is not expected to be used outside the scope of the current thesis. It will not be maintained or updated. For this reason, the application will not be reviewed in the depth in which a similar application for public use would be.

8.1. Requirements and architecture

Since the purpose of the application being developed is clear and precise, and it is not the main goal of this project, the functional requirements are few and short.



Figure 1.1. Functional requirements.



Figure 1.2. Functional requirements.

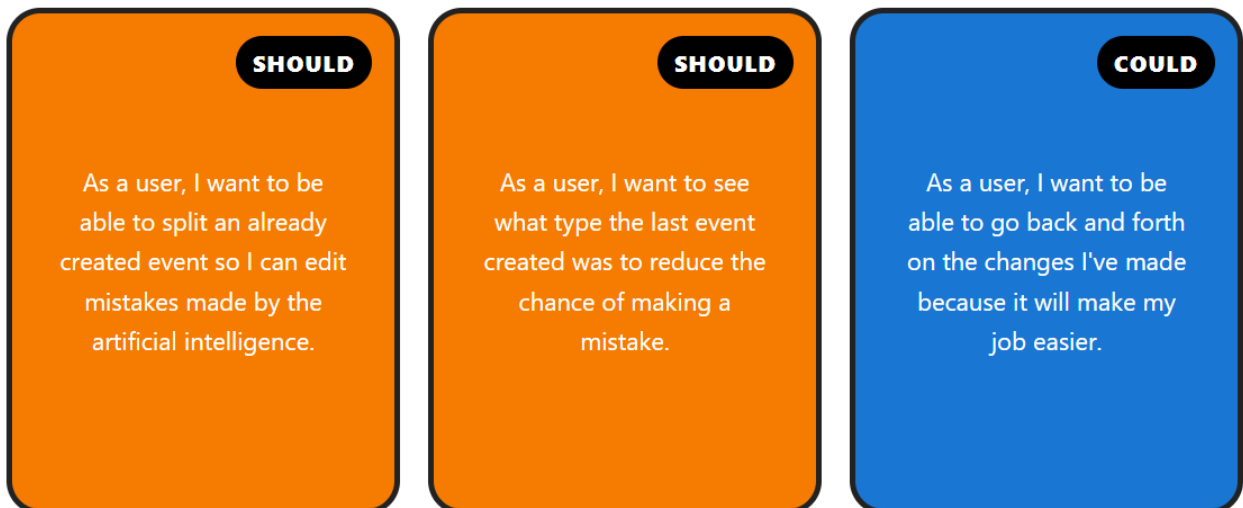


Figure 1.3. Functional requirements.



Figure 1.4. Functional requirements.

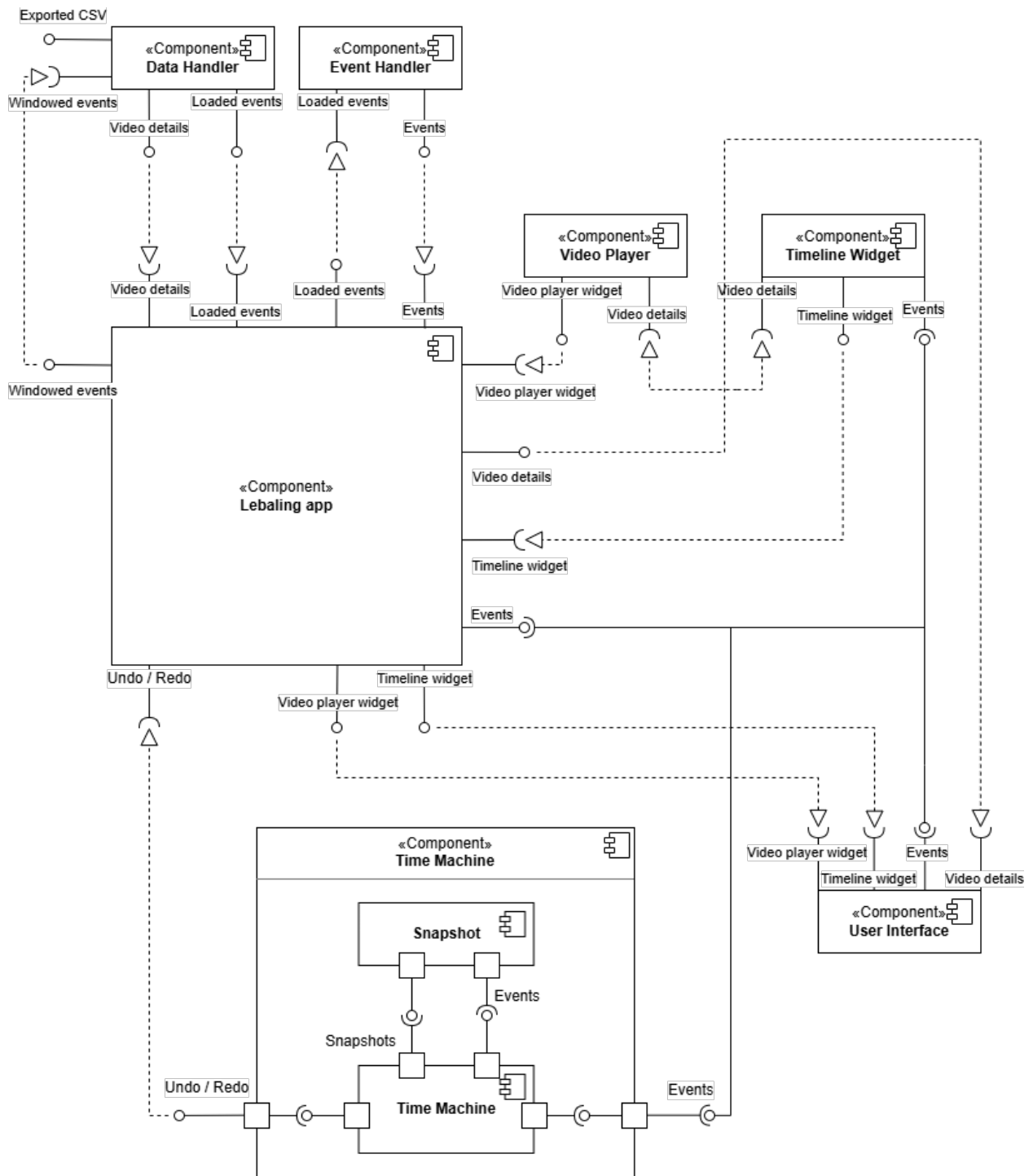


Figure 2. Component diagram. Shows the way data moves trough the system.

8.2. *Choosing a programming language and environment*

The Python programming language was chosen because of the availability of numerous graphical user interface libraries that allow rapid prototyping and development.

The Visual Studio Code text editor was chosen as the programming environment. This is because the support of multiple modules allows for enormous flexibility in the size and functionality of Visual Studio Code, and this in turn guarantees the availability of a given functionality of the text editor at the moment it is needed.

8.3. *Implementation of the software system*

8.3.1. *Description of the program modules*

Module: Main coordinating module (LabelingApp)

Purpose: It manages the application lifecycle. It initializes all the main components, establishes connections between them using Qt's signal-slot mechanism, and coordinates data exchange between the user interface, video player, timeline, and event handler.

Dependencies:

- User interface management module;
- Video playback module;
- Event management module;
- Timeline module;
- Action history module;
- Data reading and writing module;
- A file containing constants;
- A file containing information about the current video;
- The PyQt6 library;
- The numpy library;

Interface:

Class LabelingApp in file /classes/LabelingApp.py

__init__() - Initializes the application, loads the data, creates all the main components, and establishes the connections between them.

init_shortcuts() - Registers all keyboard shortcuts.

update_overlay() - Updates the position of the visualized gaze point.

update_timeline_text() - Updates the current time and timeline.

fit_video() - Scales the video to fit the window size.

handle_add_event() - Handles adding a new event.

undo_event() / redo_event() - Manages the recovery of previous actions.

split_event() - Splits the current event.

change_event_start_time() / change_event_end_time() - They change the boundaries of the current event.

on_current_event_changed() - Synchronizes the selected event with the user interface.

on_events_change_requested() - Updates the timeline after changes to events. Saves the action in the action history module.

Module: User interface management

Purpose: Builds the application's graphical user interface. Creates and arranges all visual components, including the video window, timeline, controls, labeling buttons, and settings panel. Associates graphical elements with appropriate event handlers.

Dependencies:

- Main coordinating module;
- The timeline module;
- The PyQt6 library;

Interface:

Class MainWindowUI in file /classes/MainWindowUI.py

setup_ui(parent) - It builds the entire graphical interface, creates all the visual components, and establishes the connections between them and the event handlers.

make_slider() - Creates and configures a slider for setting parameters.

Module: Timeline visualization

Purpose: Implements a graphical visualization of the video timeline. Displays the current playback position and all defined labeled events through colored intervals. Allows the user to change the current position of the video by selecting a desired position on the timeline.

Dependencies:

- The Event model
- A file containing constants;
- The PyQt6 library;

Interface:

Class TimelineWidget in file /classes/TimelineWidget.py

set_duration(duration) - Sets the total duration of the video and updates the preview.

set_position(pos) - Sets the current playback position and updates the indicator on the timeline.

set_events(events) - Sets the list of events to visualize and updates the timeline.

seek_requested(int) - A signal generated when a position is selected on the timeline. Used to change the current position of the video.

Module: Manage action history

Purpose: Provides the ability to restore previous and reapply undone changes to the event list. Implements Undo/Redo functionality.

Dependencies:

- The Event and CurrentEvent models;
- The standard library copy;

Interface:

Class Snapshot and Class TimeMachine in file /classes/TimeMachine.py

Snapshot(events, last_event) - Creates a snapshot of the current state, containing the event list and the currently selected event.

TimeMachine(events, last_event, max_snapshots=10) - Initializes the change history.

take_snapshot(events, current_event) - Writes a new state to history and removes all future states after the current one when a new change occurs.

undo() - Restores the previous state of the event list and the currently selected event.

redo() - Restores the next state from history.

Module: Video playback control

Purpose: Provides control over video playback. It implements the basic operations of starting, stopping, moving to a specific point, and moving relative in time. It also creates and manages the graphic element used to visualize the gaze point on the video.

Dependencies:

- The PyQt6 library;
- The standard library copy;

Interface:

Class VideoPlayer in file /classes/VideoPlayer.py

toggle_play() - switches between play and pause.

jump_to_time(time) - moves playback to a specified point.

seek(time) - moves the current position by a relative offset.

Module: Data loader and exporter

Purpose: Loads the data from the disk and exports the windowed frames in a csv file.

Dependencies:

- The PyQt6 library;
- The pandas library;
- The standard library json;
- The standard library os;
- The Event module;

Interface:

Functions in file /classes/DataHandler.py

load_settings() - Loads the settings from the disk if they are present.

load_data() - Loads the data (gaze, video) from the disk

save_df_to_csv() - Saves the windowed data to the disk.

save_settings() - Saves the settings to the disk.

8.3.2. *Structure and organization of the programming interface*

When developing the user interface, the main goal is the quick and easy classification of data. For this reason, a large part of the system's functions are accessible only through keyboard shortcuts.

Only those functions that allow the user to configure the system, correct errors, navigate through the record, or those that remind him of the last action performed are depicted as graphic elements.

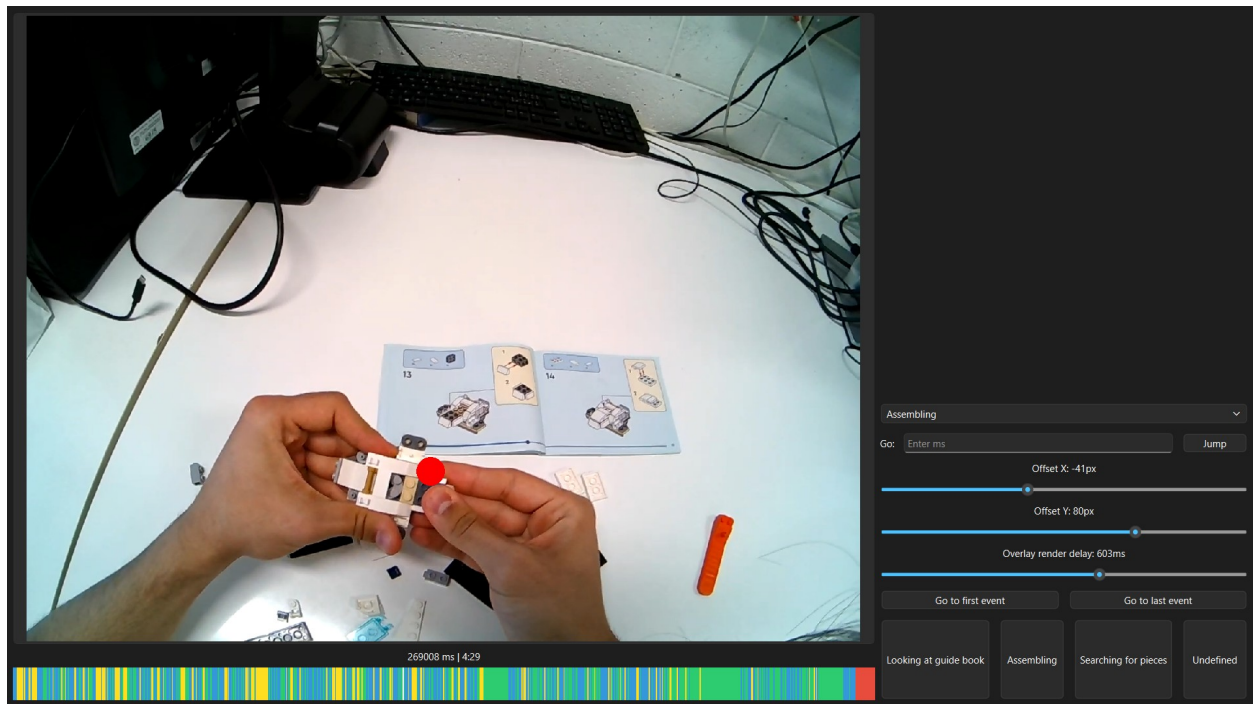


Figure 3. Overview of the application interface. Shows all of the main components of the interface of the data classification software.

The main graphic element is the video recording being played. It has an aspect ratio that takes up the maximum part of the application in order to make it as easy as possible to notice the details. A red dot is superimposed on the recording, which shows where the participant in the experiment is looking in the given frame.

Below it is the elapsed time, from the beginning of the recording, in milliseconds and minutes. The decision to display it in milliseconds was made for the purpose of easier comparison of the current frame with the already existing data. This is because the majority of time data is measured in milliseconds.

Immediately after the elapsed time is the timeline. Its purpose is to depict where in time the current frame is located, as well as what the time segment in which it is located is categorized as. This is achieved by depicting the current frame with a horizontal white line, and the time segment with a rectangle of a different color, relative to the given category. The left side of the rectangle is located at the beginning of the given event, and the right side is located at the end.

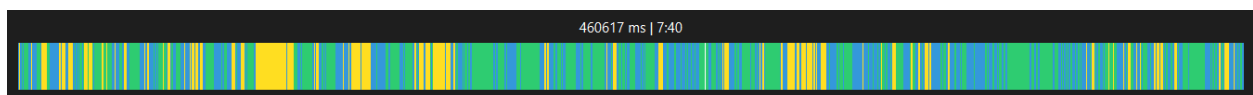


Figure 4. Timeline widget. Shows the timeline widget and how the windowed events appear in the interface.

All other elements are located to the right of the playing track.

The first of these is a drop-down menu. It allows you to see what the current time segment is categorized as and change it if necessary.

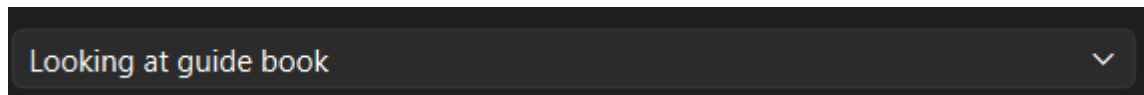


Figure 5.1. Categorized segment drop-down menu. Shows the drop-down menu when its folded.

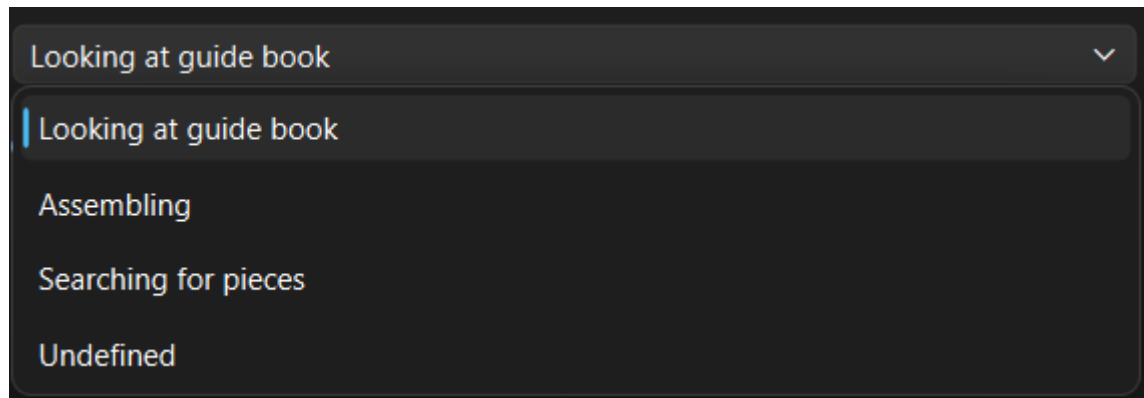


Figure 5.2. Categorized segment drop-down menu. Shows the drop-down menu when its unfolded.

Below the drop-down menu, there is a section that allows the user to jump directly to a frame located at a given millisecond from the beginning of the recording. This is done through a text field in which the given millisecond is written, and a button that, when pressed, jumps the recording to the given millisecond. This section has been added to make it easier to check a specific time fragment taken from the csv file in which the time fragments are written.



Figure 6. “Skip to” input section. Shows the input section that enables the jumping to specific millisecond in the video.

The next section allows the user to adjust the red dot that indicates where the participant is looking in a given frame. The user can add horizontal and vertical offset, as well as offset the dot in time. This is done using three sliders.

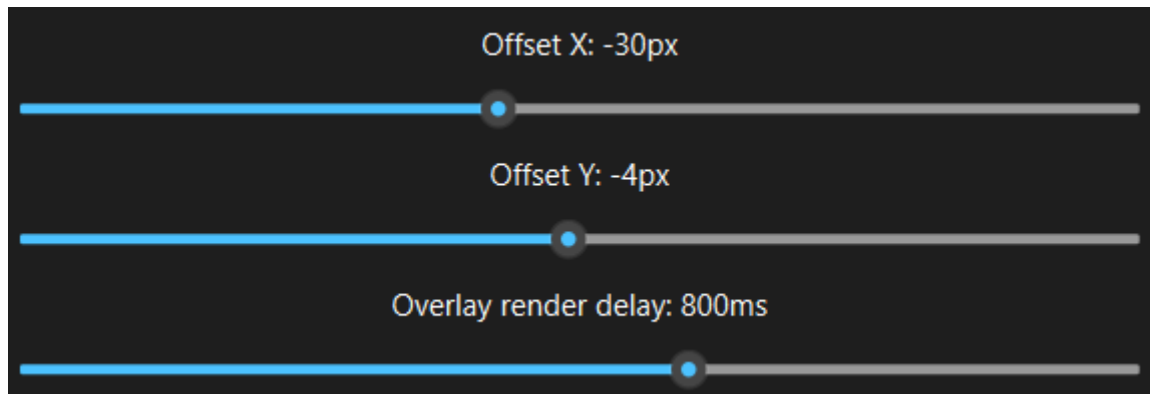


Figure 7. Overlay settings. Shows the 3 slides that control when and where exactly the red dot for the position of the gaze appear.

The last two sections allow event-related control, with the first of them allowing the selection of the first or last created time segment. Thanks to this, it is easier to temporarily stop and resume the classification of the recording.



Figure 8. Event skipping buttons. Shows the 2 buttons that enable the jumping to the first and the last categorized section

The last section consists of four buttons, each labeled with one of the four categories. When one of them is pressed, the current time segment is categorized with the corresponding category, and the button changes color. This way, the user always knows what the last selected category was.

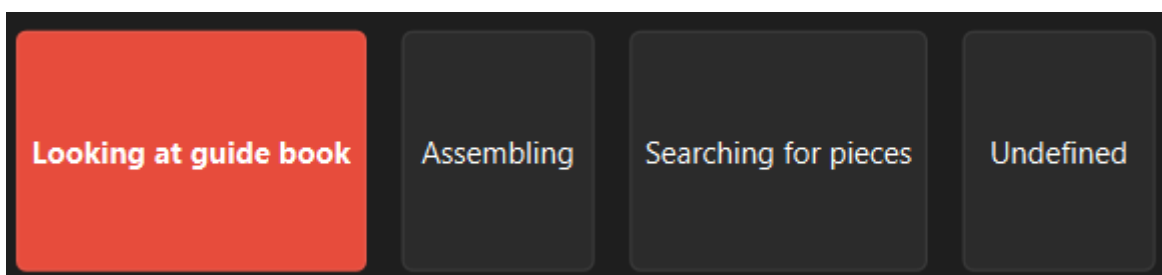


Figure 9. Categorizing buttons. Shows 4 buttons that when pressed, categorize the current section with the given category.

8.4. *Input data format*

Project Directory: Checks whether the selected folder contains the required files. If the files are missing, a message is displayed to the user and the user is given the option to reselect the directory.

Millisecond to jump to: Checks whether the entered number is an integer. If the entered value is not an integer, a message is displayed to the user.

8.5. *Output data format*

A CSV file containing the time windows created by the user. Each window contains the recording id, the start timestamp [ms], the end timestamp [ms], and the category it is assigned to.

Example output:

```
recording id,start timestamp [ms],end timestamp [ms],label
5bcd3401-e378-482f-b9c6-37edbad97d1a,0,6315,Undefined
5bcd3401-e378-482f-b9c6-37edbad97d1a,6315,9215,Looking at guide book
5bcd3401-e378-482f-b9c6-37edbad97d1a,9215,13941,Undefined
5bcd3401-e378-482f-b9c6-37edbad97d1a,13941,14391,Looking at guide book
5bcd3401-e378-482f-b9c6-37edbad97d1a,14391,14566,Searching for pieces
5bcd3401-e378-482f-b9c6-37edbad97d1a,14566,15691,Looking at guide book
5bcd3401-e378-482f-b9c6-37edbad97d1a,15691,17834,Searching for pieces
5bcd3401-e378-482f-b9c6-37edbad97d1a,17834,18267,Looking at guide book
5bcd3401-e378-482f-b9c6-37edbad97d1a,18267,18701,Searching for pieces
5bcd3401-e378-482f-b9c6-37edbad97d1a,18701,19151,Looking at guide book
5bcd3401-e378-482f-b9c6-37edbad97d1a,19151,19526,Assembling
```

9. Development environment, remote access and home server

JupyterLab was chosen as the development environment because it is one of the most common solutions for developing and experimenting with machine learning models. In addition to that, the environment's interface is web based and it makes it easier to host so that it can be accessed from the LAN. The platform supports notebooks, which allow the combination of code, visualizations, and documentation in a single document.

The choice to install a local instance of JupyterLab was made in order to avoid dependency on a foreign platform, as well as to allow the entire experiment to be as reproducible as possible.

To achieve this goal, Docker was used. Docker is a containerization platform that allows the development and execution of applications in isolated environments called containers. [6]

Containerization in this case is achieved through a Docker container. A Docker container is a standardized unit for distributing and executing software that contains the application and all the dependencies necessary for its operation. This allows the application to function in the same way in different execution environments. [6]

9.1. Creating a Docker image

A Docker image is created to create the container. A Docker image is an immutable template that contains the necessary instructions and files to create a Docker container. [6]

The created Docker image contains the following directives:

FROM nvidia/cuda:12.1.1-runtime-ubuntu22.04

WORKDIR /workspace

*RUN apt-get update && apt-get install -y python3-pip git && rm -rf /var/lib/apt/lists/**

RUN pip install --no-cache-dir fastai jupyter matplotlib pandas scikit-learn polars

EXPOSE 8888

CMD ["jupyter", "lab", "--ip=0.0.0.0", "--port=8888", "--no-browser", "--allow-root"]

The *FROM* directive indicates which other image this image is based on. [5] In this case, it is based on the image “*nvidia/cuda:12.1.1-runtime-ubuntu22.04*”

The *WORKDIR* directive specifies the working directory, and if it does not exist, it will be created. The working directory indicates where subsequent directives that have something to do with the file system will be executed. [5] In this case, the working directory is “*/workspace*”

The *RUN* directive specifies a command to be executed during the image creation [31]. Here, we specify the command *apt-get update && apt-get install -y python3-pip git && rm -rf /var/lib/apt/lists/**

This command is actually a combination of 3 commands, namely *apt-get update*, *apt-get install -y python3-pip git*, and *rm -rf /var/lib/apt/lists/**. These commands are separated by *&&*. These two characters represent the logical AND operator, used to execute commands sequentially in Unix-like operating systems. The next command is executed only if the previous one completed successfully with an exit code of 0. [22]

The first two commands start with *apt-get*. This is a console-based package manager.

The *apt-get update* command updates the local package index by retrieving up-to-date information from the configured software repositories (/etc/apt/sources.list). This way, the system receives information about the available versions of the packages before installing or updating them. [15]

The *apt-get install* command installs or updates all the libraries listed after it. [15] In this case, we are installing the *python3-pip* and *git* libraries. The *-y* flag is added to ensure that all requests are automatically confirmed during the installation. This is necessary when building Docker images, since the process is performed automatically and does not allow interactive user input.

The *rm* command removes all found files or directories that match the following parameter. [23] In this case, everything in the */var/lib/apt/lists/* directory is removed in order to reduce the size of the image. Two flags are used, the first of which is *-r*. This flag allows the command to recursively remove everything in a given directory [23]. The *-f* flag tells the command not to display any messages and not to throw errors if the given directory does not exist. [23]

The next command that we put before the *RUN* directive is *pip install --no-cache-dir fastai jupyter matplotlib pandas scikit-learn polars*

pip is the most common and frequently used package manager for Python. It allows you to find, download, and install packages from the Python Package Index (PyPI), as well as other packages. [12]

The *install* command tells *pip* that the following libraries should be installed. [13] The *--no-cache-dir* option tells *pip* that no cache should be stored when installing the given libraries. [14] Here again, the goal is to keep the image size as small as possible.

The *EXPOSE 8888* directive specifies that the application in the container listens on port 8888. It is informative and does not automatically publish the port to the host system. [5]

The *CMD* directive defines the default command that is executed when the container starts. [5] In this case, a JupyterLab server is started, connecting to all network interfaces in the container (0.0.0.0) and using port 8888. The *--no-browser* option prevents the automatic opening of a browser [30], and *--allow-root* allows execution with root privileges. [9]

9.2. Creating a *compose.yml* file

Compose files are files that allow the user to configure and orchestrate multiple containers at once. [3] In this case, *compose.yml* is used to ensure the reproducibility of the runtime environment. This is because otherwise all additional commands, except for the name of the image we want to use, would have to be written separately each time the container is created.

As you can see from the file extension, this file is a YAML file. YAML is a human-readable data serialization language used to represent structured data. [25]

In this case, *compose.yml* looks like this:

services:

fastai_and_jupyter:

```

image: fastai-jupyter-lab
container_name: fastai-jupyter-lab
ports:
  - "8888:8888"
volumes:
  - ../notebooks:/workspace/notebooks # Notebooks
  - ../data:/workspace/data # Used for datasets
deploy:
  resources:
    reservations:
      devices:
        - capabilities: [gpu]
stdin_open: true
tty: true

```

The *services* element is a set of container service configurations, with each service defining a way to create one or more containers. [7] This is a required element, without which we cannot describe the rest of the file.

The *fastai_and_jupyter* element is created by the user, it is the name of the service.

The *image* element specifies which image will be used to create the service. [7] In this case, *fastai-jupyter-lab* is used, as this is the name of the image that was created previously

The *container_name* element specifies the name of the container. [7] Here, the container is the same as the image – *fastai-jupyter-lab*.

The *ports* element is used to define the relationship between the machine ports and the container ports. [7] First, the host machine port is specified, and then which port on the container it should correspond to. In this case, *port 8888* on the machine will correspond to *port 8888* on the container.

The *volumes* element is used to mount directories from the host system to the container file system, allowing data to be shared and saved between them. [7] In this case, we can see that two directories are being mapped – the *../notebooks* directory on the host machine to the */workspace/notebooks* directory on the container, and the *../data* directory on the host machine to the */workspace/data* directory on the container.

The *deploy* element specifies additional information about how the containers will be executed. [7]

The *resources* element specifies the physical resource constraints. [7]

The *reservations* element tells the system that the following defined resources must be made available to the container; if this is not possible, the container will not be created. [7]

The *devices* element reserves a specific hardware device. [7]

The *capabilities* element specifies which device to reserve. [7]

The *stdin_open* and *tty* elements together serve to create a terminal for the purpose of an interactive session. [4]

9.3. Remote access

Since the Jupyter notebook is installed on a home computer, a way is needed to turn it on and access it remotely. For this purpose, an ASUS RT-AC68U router with the "Dynamic Domain Name System" (DDNS) option enabled was used, creating a permanent domain that could be used to access a local instance of OpenVPN and WireGuard.

DDNS is a service that updates the DNS records with the correct IP address, even if that IP address is constantly being updated. [21]

Another supported feature is "wake on lan". It allows a command to be sent to the network card of a system with a specific IP, which tells the network card to start the system. The system on which JupyterLab is installed is started using this feature of the router.

The aforementioned router supports the option to run a local instance of OpenVPN directly on it. Due to its more complex implementation and the fact that the router's processor is significantly weaker, it was decided to use a local installation of WireGuard located on a home server. This allows for faster internet speeds. More information about the home server in the next subsection.

OpenVPN is reserved as an option, which is used when connecting through highly restricted public networks, such as edurome, which block the necessary ports for WireGuard to work properly. Due to the fact that it can work over Transmission Control Protocol (TCP), OpenVPN is able to present its packets as internet traffic over port 443.

After the remote connection to the home network, locally located instances of Moonlight and Sunshine are used to access the desktop of the host machine. They use a client-server architecture with Sunshine installed on the host machine and Moonlight installed on the client. They are used due to the fact that they are pre-installed on the respective machines.

Thanks to the home server, access to the workspace is only needed if there is some unforeseen problem with the Docker container in which JupyterLab is running or some other unforeseen problem with the host machine.

9.4. Home server

The home server is an Acer Aspire A515-51G laptop [10] with an Intel(R) Core(TM) i5-7200U processor, an NVIDIA Corporation GM108M [GeForce MX130] graphics card, and an Ubuntu 24.04.3 LTS operating system.

Among many other services, the server has locally deployed Pihole, Caddy, and WireGuard services. Thanks to their combination, remote access to JupyterLab is possible.

To provide secure remote access to the JupyterLab development environment, the system combines a WireGuard VPN, Pi-hole DNS, and the Caddy reverse proxy. Each component has a separate role in establishing the connection.

WireGuard provides the secure network connection between the remote client and the private network. Once the VPN tunnel is established, the client can access services that are not directly exposed to the Internet.

Pi-hole is used as the DNS server for the private network. A local DNS record associates the domain name used to access JupyterLab with the IP address of the server running the service. This allows the service to be accessed using a domain name instead of requiring the user to remember an IP address and port.

Caddy operates as a reverse proxy in front of JupyterLab. It receives HTTPS requests addressed to the configured domain name and forwards them to the JupyterLab instance running on the internal server. Caddy also handles the HTTPS connection, allowing communication between the client and the reverse proxy to be encrypted.

The resulting communication path is therefore:

Client → WireGuard VPN → Pi-hole DNS resolution → Caddy reverse proxy → JupyterLab

When the client requests the JupyterLab domain, Pi-hole resolves the domain to the internal server address. The client then establishes an HTTPS connection to Caddy. Caddy identifies the requested domain and forwards the request to the corresponding JupyterLab instance. Because JupyterLab is accessible only through the private network, the application itself does not need to be directly exposed to the public Internet.

This architecture separates the responsibilities of network access, name resolution, and application access. WireGuard provides secure remote connectivity, Pi-hole provides convenient internal name resolution, while Caddy provides HTTPS termination and reverse proxy functionality.

For more information on the Docker Compose files used to host those services see APPLICATION B for Pihole's compose file, APPLICATION C for Caddy's compose file and APPLICATION D for WireGuard's compose file.

For detailed documentation of the home server visit:
<https://github.com/GeorgeChaparov/Homelab>

10. *Creating and training the model*

10.1. *Input data*

After classifying the data using the application developed for this purpose, the following useful files are available for each participant in the experiment:

"3d_eye_states.csv", "blinks.csv", "fixations.csv", "gaze.csv", "imu.csv", "labels.csv" and "saccades.csv".

These are files in the .csv format. This format is used to store plain text data that represents a worksheet. This is done by treating each new line in the file as a separate row in the table, and the values in that row are separated by a comma. Each value defines a separate column in the table. The values in the first row of the file are treated as the headers of the corresponding column. [8]

Each of the files contains the column "recording id", which serves to identify the given recording, and its value is unique for all records. The column is used repeatedly in organizing and pre-processing the data, as described in the next subsection.

The files "3d_eye_states.csv", "gaze.csv" and "imu.csv" share the same column, namely "timestamp [ns]". This column shows in which nanosecond the given measurement was made.

The files "fixations.csv", "saccades.csv" and "blinks.csv" share three identical columns, namely "start timestamp [ns]", "end timestamp [ns]" and "duration [ms]". The first two show in which nanosecond the beginning and end of the given event occurred, and the third – how many milliseconds the given event lasted. Each of these files also has a column that shows the identification number of the given event. The name of this column is a suffix of the given file name, and the word "id" separated by a space.

The file "labels.csv" also has two columns indicating the start and end of the event, but the unit of measurement is milliseconds. Their names are "start timestamp [ms]" and "end timestamp [ms]" respectively.

All other columns, in separate files, are:

3d_eye_states.csv – Contains the physical parameters of the eye, which indicate its current state such as: pupil diameter left/right [mm], eyeball center left/right x/y/z [mm], optical axis left/right x/y/z, eyelid angle top/bottom left/right [rad], and eyelid aperture left/right [mm]. This information is largely used by the Pupil Labs model to extract additional data, which is stored in the files below.

blinks.csv – Contains no additional columns other than those listed above.

fixations.csv – Contains information about each fixation such as the position of the work area in pixels (fixation x/y [px]), as well as the horizontal (azimuth [deg]) and vertical (elevation [deg]) angle relative to the center of gaze measured in degrees.

saccade.csv – Contains the distance between the start and end points of the saccade in pixels (amplitude [px]) and in degrees (amplitude [deg]). It also stores the maximum and average saccade velocity as pixels per second (peak velocity [px/s], mean velocity [px/s])

gaze.csv – Stores information about the gaze as a whole. The position of the gaze on the work area in pixels (gaze x/y [px]), as well as the horizontal (azimuth [deg]) and vertical (elevation [deg]) angle relative to the center of gaze measured in degrees. The blinks id and fixation id columns indicate the sequential number of the fixation or blink, if

any. Additionally, it contains information about whether the glasses were worn at a given time (worn).

imu.csv - Contains data from the inertial measurement module (IMU) of the glasses. The columns "gyro x/y/z [deg/s]" represent the angular velocity around the three axes, measured in degrees per second, and "acceleration x/y/z [g]" - the linear acceleration along the three axes, measured in units of the acceleration of gravity (g). The columns "roll [deg]", "pitch [deg]" and "yaw [deg]" describe the orientation of the glasses using the three Euler angles, measured in degrees. The columns "quaternion w/x/y/z" represent the same orientation in the form of a quaternion.

label.csv - Additionally, it contains the column "label", which indicates how the given time range is identified.

10.2. Used libraries

Several Python libraries were used throughout the data-processing, model development, training, evaluation, and visualization stages.

Table 1. Main used libraries for creating and training the models

Main libraries	Purpose
Polars	Tabular data manipulation and preprocessing.
NumPy	Numerical operations and array manipulation where applicable.
PyTorch	Machine learning framework for implementing, training, and evaluating the neural network models. Responsible for model definition, tensor operations, optimization, loss calculation, and the overall training process.
scikit-learn	Model evaluation and dataset splitting.
Optuna	Automated hyperparameter optimization.
Matplotlib	Data visualization and for generating graphs used to analyze the training process and experimental results.

In addition to the main libraries, several Python standard-library modules were used to support the implementation.

Table 2. Standard-library modules used for creating and training the models

Standard-library module	Purpose
enum	To define enumerations for fixed sets of related values, such as model and experiment types.
pathlib	Platform-independent file and directory handling.

json	Storing and loading configuration and experiment information.
os	operating-system-related functionality
copy	Independent copies of objects during the training and evaluation process.
hashlib	Generating different hashes for each configuration for windowing of the data.
random	Randomization for controlling random behavior during experiments.

10.3. Data pre-processing

Before the data can be used for windowing and model training, the raw data produced by the eye-tracking system must be preprocessed and combined into a unified dataset. The purpose of this stage is to remove irrelevant or unusable measurements, align the different data sources temporally, associate the recorded events and experimental labels with the corresponding gaze samples, and reduce the memory and storage requirements of the resulting dataset.

The preprocessing is performed independently for each recording. The processed recordings are initially stored as separate Parquet files and are subsequently combined into a single Parquet file containing the complete dataset.

The preprocessing consists of the following steps:

- **Loading the recording data:** All relevant data files belonging to a recording are loaded. These include gaze data, saccades, fixations, blinks, IMU measurements, and 3D eye-state measurements. For recordings used for training, the manually assigned activity labels are loaded as well.
- **Removing invalid gaze measurements:** Rows in the gaze data for which the worn column has a value of 0 are removed. These measurements indicate that the eye-tracking device was not being worn and therefore do not represent valid observations. After this filtering, the worn column is removed because all remaining measurements have the same value. The columns gaze mono left x [px], gaze mono left y [px], gaze mono right x [px], and gaze mono right y [px] are also removed because they do not contain useful information in the collected dataset.
- **Processing activity labels:** The manually assigned activity labels are first sorted chronologically. Consecutive label intervals are merged when they have the same label and are directly adjacent in time. This produces continuous intervals for each activity instead of multiple consecutive intervals representing the same activity. The label intervals are then temporally associated with the corresponding gaze measurements. Gaze samples that do not belong to a defined activity are removed from the training dataset.

- **Associating saccades with gaze samples:** Saccade intervals are temporally aligned with the gaze measurements using an as-of join. Each gaze sample is associated with the most recently started saccade interval, provided that the gaze timestamp falls within the corresponding saccade interval. This allows information such as saccade duration, amplitude, and velocity to be associated with the individual gaze measurements.
- **Associating fixations and blinks:** Fixation and blink information are joined with the gaze data using their respective event identifiers. This associates measurements with the corresponding gaze samples.
- **Temporal alignment of multimodal sensor data:** The gaze data are combined with the IMU and 3D eye-state measurements using their timestamps. These data sources operate at different sampling frequencies, so the nearest available measurement is selected for each gaze sample. A maximum temporal difference of 50 ms is allowed for this alignment. This produces a unified time-series in which gaze and head-motion information can be processed together.
- **Removal of redundant columns:** Columns duplicated during the merging of the different data sources, such as section id imu, recording id imu, section id eye3d, and recording id eye3d, are removed. These columns do not provide additional information after the datasets have been combined and their removal reduces the size of the resulting dataset.
- **Data type conversion:** Selected numerical columns are converted to the Float32 data type. By doing this the memory and storage requirements are reduced compared with higher-precision floating-point representations while providing sufficient numerical precision for the measurements used in the subsequent processing and machine learning stages.
- **Handling missing values:** Missing values in selected columns are replaced with predefined values. Event identifiers such as fixation and blink identifiers are assigned the value unknown when no corresponding event is present. Numerical values for measurements such as fixation and blink information are assigned -1 when the corresponding event does not exist. This provides an explicit representation of the absence of an event and prevents missing values from propagating into later processing stages.

After these operations have been performed, each recording is saved as an individual Parquet file. Once all recordings have been processed, the individual files are combined into a single merged_output.parquet file. This file represents the unified dataset used by the subsequent feature calculation and windowing stages.

10.4. Data windowing

After the preprocessing stage, the continuous multimodal time-series data must be divided into fixed-length temporal segments.

Windowing is required to transform the continuous multimodal time-series data into fixed-length input samples suitable for machine learning. Since the neural network models used in this diploma operate on inputs with a predefined temporal dimension, each input sample must contain the same number of consecutive measurements. For that reason, the continuous sensor signals are divided into fixed-duration temporal windows, with each window representing an individual input instance.

The windowing pipeline is controlled by several settings that determine the temporal size of each window, the minimum amount of data required for a valid window, which part of the recording is used, and which feature representation is generated. The main settings are:

- **Window duration:** Determines the temporal duration of each window. A duration of 0.22 seconds was selected for the final experiments.
Target length: Determines the number of samples provided to the model from each window. With the gaze sampling frequency of 200 Hz, a 0.22-second window contains approximately 44 samples. The target length was set to 45 samples, providing a consistent temporal dimension for all inputs.
- **Window overlap:** Determines how much consecutive windows share the same data. An overlap of 0% means that consecutive windows are adjacent and do not contain duplicated measurements. Different overlap configurations were tested, but no meaningful improvement in classification performance was observed when overlap was introduced. An overlap of 0% was used in the final experiments. This also avoids generating redundant samples and reduces the computational and memory requirements of the training process.
- **Minimum samples per window:** Specifies the minimum number of measurements required for a window to be considered valid. This value was set to 5 samples. Windows containing fewer than five measurements are discarded because they do not contain sufficient temporal information to form a meaningful input sample.

Additional settings were introduced to investigate whether the temporal boundaries of the recordings influenced classification performance:

- **Use video minimum length:** When enabled, all recordings are truncated to the length of the shortest recording. This ensures that every subject contributes the same amount of temporal data.
- **Video percentage:** Specifies the fraction of each recording that should be retained.
- **Video offset:** Specifies the relative position from which the selected portion of the recording should begin.

These settings were introduced because the beginning and end of a recording may contain segments that are less relevant to the actual assembly activities. It was investigated whether excluding parts of the recordings could remove potentially irrelevant information and improve classification performance. However, experiments using these restrictions resulted in lower classification performance. Consequently, the final experiments used the complete available recordings without applying a video-length restriction, percentage-based cropping, or temporal offset.

The pipeline also supports two different feature representations:

- **Raw features:** The original time-series measurements contained within each window are retained. This representation preserves the temporal evolution of the sensor signals and is provided directly to the neural network.
- **Engineered features:** Statistical and domain-specific characteristics are extracted from each window. These include measures describing gaze velocity, fixation and saccade activity, gaze dispersion and entropy, head movement, pupil

behavior, and other characteristics of the eye and head signals. This representation reduces the temporal data within each window to a fixed set of descriptive features.

10.4.1. Windowing procedure

The windowing process begins by loading the preprocessed data as a Polars LazyFrame. The data are organized by recording, and each recording is processed independently. This prevents windows from crossing the boundary between two different recordings or subjects.

Before the windows are generated, additional features are calculated for each recording. These include gaze velocity, the mean pupil diameter, normalized pupil measurement and others. The data are then sorted according to their timestamps to ensure that the measurements within each window are in chronological order.

For each recording, the first timestamp is used as the starting point of the windowing process.

The window is then populated with all measurements whose timestamps fall within this interval. If the number of measurements is below the minimum required value, the window is discarded. Otherwise, the first label occurring within the window is assigned as the label of that window.

The temporal position of each window is also calculated relative to the complete recording. The center of the window is used for this purpose, producing a normalized value between 0 and 1 that represents the relative position of the window within the recording. This value is included as an additional feature, allowing the model to account for the temporal position of an observation within the recording.

Since the number of measurements inside a window can vary slightly due to the sampling process, every valid window is converted to the same target length. If a window contains more measurements than the target length, only the first measurements up to the target length are retained. If it contains fewer measurements, the last available measurement is repeated until the required target length is reached.

For the final configuration, the resulting samples therefore have a temporal dimension of 45 measurements per window, corresponding to approximately 0.22 seconds of sensor data. The generated windows are subsequently stored in a cache, separately for each recording, so that the computationally expensive windowing and feature-extraction operations do not need to be repeated when running subsequent experiments.

To avoid accidentally reusing windowed data generated with different settings, the configuration of the windowing process is serialized and hashed. The resulting hash is used as the name of the cache directory. Consequently, changing a relevant setting, such as the window duration, overlap, target length, or selected features, produces a different cache location and prevents incompatible cached data from being used.

10.5. Model architectures

To investigate how different approaches to temporal and feature representation affect the classification of assembly activities, seven neural network architectures were implemented and evaluated: CNN, LSTM, CNN-LSTM, LSTM-CNN, CNN-LSTM-Fusion, CNN-Transformer, and MLP.

The architectures were selected to cover several different approaches to processing the multimodal time-series data. CNNs were included because they can extract local temporal patterns from consecutive measurements, while LSTMs were selected for their ability to model sequential dependencies. Hybrid CNN-LSTM and LSTM-CNN architectures were used to investigate whether combining these two approaches provides an advantage and whether the order in which temporal feature extraction is performed affects the results. A CNN-Transformer architecture was included to investigate the use of self-attention for modeling temporal relationships. The CNN-LSTM-Fusion architecture provides two parallel representations of the input, allowing convolutional and recurrent features to be combined before classification. Finally, an MLP was included as a simpler baseline that does not explicitly model the temporal sequence.

All neural network architectures use the same final number of output classes, corresponding to the activity classes defined in the dataset. The architectural hyperparameters, such as the number of channels, hidden dimensions, number of recurrent or Transformer layers, and dropout rate, were optimized separately during the hyperparameter optimization stage.

10.5.1. CNN

The CNN model uses a sequence of one-dimensional convolutional layers to extract local temporal patterns from the input time-series data. The input is initially rearranged so that the feature dimension is treated as the channel dimension required by Conv1d. Three convolutional layers are then applied sequentially, with batch normalization and a configurable activation function after each layer.

The number of convolutional channels can increase between layers according to the selected channel multiplication factor. This allows later layers to represent increasingly complex features derived from the patterns detected by the preceding layers.

After the convolutional feature extraction stage, adaptive average pooling is applied over the temporal dimension. This converts the variable temporal representation into a fixed-size feature vector. The resulting representation is passed through fully connected layers and a dropout layer before the final classification layer.

The CNN therefore primarily focuses on local temporal patterns within the sensor signals rather than explicitly modeling long-term sequential dependencies.

10.5.2. LSTM

The LSTM model processes the input as a temporal sequence, with each timestep containing the available sensor features. One or more stacked LSTM layers are used to model dependencies between measurements at different points in time.

The output produced by the final LSTM layer contains a hidden representation for every timestep. These representations are averaged across the temporal dimension to obtain a single fixed-size representation of the complete window. This representation is then passed through a fully connected classification head consisting of linear, activation, and dropout layers.

The LSTM architecture therefore focuses on temporal dependencies between consecutive measurements rather than explicitly extracting local patterns using convolutional filters.

10.5.3. CNN-LSTM

The CNN-LSTM architecture combines convolutional feature extraction with recurrent temporal modeling. First, several one-dimensional convolutional layers extract local patterns from the input sequence. A max-pooling layer then reduces the temporal resolution and consequently the number of timesteps that must be processed by the recurrent component.

The resulting feature sequence is passed to an LSTM, which models temporal dependencies between the extracted convolutional features. Layer normalization is subsequently applied, followed by an attention-based pooling mechanism. The attention pooling assigns different weights to the individual timesteps, allowing the model to emphasize portions of the sequence that are more relevant for classification.

The resulting representation is passed through a multi-layer classification head with dropout before producing the final class prediction.

This architecture combines the local pattern extraction capability of CNNs with the sequential modeling capability of LSTMs.

10.5.4. LSTM-CNN

The LSTM-CNN architecture reverses the order of the recurrent and convolutional components used in CNN-LSTM.

The input sequence is first processed by one or more LSTM layers, producing a sequence of temporally contextualized feature representations. These representations are then treated as channels and passed through a series of one-dimensional convolutional layers. Max pooling is used to reduce the temporal dimension, followed by adaptive average pooling to obtain a fixed-size representation.

The resulting feature vector is passed through a fully connected classification head.

Comparing CNN-LSTM and LSTM-CNN makes it possible to investigate whether it is more effective to perform local convolutional feature extraction before recurrent processing, or to first obtain temporally contextualized representations using an LSTM and subsequently extract local patterns from them using CNN layers.

10.5.5. CNN-LSTM-Fusion

The CNN-LSTM-Fusion architecture uses two parallel processing branches. The first branch consists of convolutional layers that extract local temporal features from the input sequence. The second branch processes the original sequence using an LSTM to obtain a representation based on temporal dependencies.

The outputs of the two branches are independently pooled and then concatenated into a single feature representation. This combined representation is passed to a fully connected classification head.

Unlike the sequential CNN-LSTM and LSTM-CNN architectures, the fusion architecture does not force one representation to be generated from the output of the other. Instead, it allows the model to retain parallel convolutional and recurrent representations and combine them before classification.

10.5.6. CNN-Transformer

The CNN-Transformer architecture combines convolutional feature extraction with a Transformer encoder. Initially, several one-dimensional convolutional layers extract

local temporal patterns from the input. A max-pooling layer reduces the temporal dimension before the resulting sequence is passed to the Transformer encoder.

The Transformer encoder consists of multiple Transformer encoder layers and uses multi-head self-attention to model relationships between different positions in the temporal sequence. The resulting temporal representations are averaged to obtain a fixed-size representation.

In addition to the temporal representation generated by the CNN and Transformer, the model incorporates the engineered features extracted from each window. The Transformer representation and the engineered feature vector are concatenated and passed through a fully connected fusion layer followed by the final classification layer.

This architecture therefore combines three types of information: local temporal patterns extracted by the CNN, temporal relationships modeled by self-attention, and manually engineered statistical and domain-specific features.

10.5.7. MLP

The MLP architecture was included as a simpler baseline. Unlike the other architectures, it does not explicitly process the temporal sequence. Instead, it receives a fixed-size feature vector and processes it through a sequence of fully connected layers.

Each hidden layer consists of a linear transformation, layer normalization, an activation function, and dropout. The final linear layer produces the class probabilities.

The MLP provides a baseline for determining whether the additional temporal modeling capabilities of CNNs, LSTMs, and Transformers provide a measurable advantage over a simpler feed-forward architecture.

10.6. Training methodology

10.6.1. Experimental scenarios

Two experimental scenarios were defined to evaluate the ability of the developed models to classify the different assembly activities: subject-dependent and subject-independent experiments. The two scenarios evaluate different forms of generalization and were therefore treated as separate experimental settings throughout the hyperparameter optimization and final model evaluation processes.

The subject-dependent scenario evaluates the ability of the models to classify activity samples when the subjects present in the evaluation data are also represented in the training data. Consequently, the model can learn characteristics that are specific to individual subjects in addition to characteristics associated with the performed activities.

This scenario represents a less restrictive classification setting and is used to determine how accurately the activities can be recognized when the model has previously been exposed to data from the subjects being evaluated. It provides an indication of the achievable classification performance when the system is intended to operate on subjects that have already contributed data during model development.

The subject-independent scenario evaluates the ability of the models to generalize to subjects that were not present during training. Data belonging to a subject are

therefore kept entirely within a single dataset partition and are never shared between the training, validation, and test sets.

This scenario represents a more demanding and realistic generalization setting. The model cannot rely on subject-specific characteristics learned during training and must instead identify patterns associated with the performed activities that remain informative across different subjects.

The two scenarios were evaluated independently. Separate hyperparameter optimization procedures were performed for each scenario, and the resulting optimized configurations were subsequently used for the corresponding final model evaluations.

10.6.2. Dataset splitting

The dataset was divided into training, validation, and test subsets. The splitting procedure differs between the subject-dependent and subject-independent experimental scenarios because the two scenarios have different requirements regarding subject overlap between the subsets.

An additional grouping mechanism was used during splitting to prevent highly correlated temporal windows from being distributed between different subsets. This is particularly important because consecutive windows originating from the same recording can contain very similar sensor measurements. Allowing such windows to appear in both the training and evaluation data could result in overly optimistic performance estimates.

For the subject-dependent experiments, the data were divided into training, validation, and test subsets using Stratified Group K-Fold with five folds.

To reduce the possibility of temporal data leakage, each window was assigned to one of 20 temporal groups according to its relative position within the corresponding recording. The relative position of a window was calculated during the windowing process, where the center of each window was normalized to the interval $[0, 1]$ with respect to the duration of the recording. The resulting value was used to assign the window to one of 20 equally sized temporal blocks.

This grouping ensures that windows originating from the same temporal region of a recording are kept within the same dataset partition. This is particularly important because neighboring windows can contain highly similar measurements and therefore may not represent independent samples.

Five stratified groups were generated, with one fold selected as the test set and another as the validation set. The remaining three folds were used for training. This resulted in an approximately 60% training, 20% validation, and 20% test division of the data.

Stratification was used to maintain a similar distribution of the activity classes between the partitions, while grouping ensured that the temporal groups were not divided between them. The resulting partitions were additionally checked programmatically to verify that no temporal group was shared between the training, validation, and test sets.

For the subject-independent experiments, a Leave-One-Subject-Out (LOSO) strategy was used. During each iteration, all samples belonging to one subject were excluded from the training and validation data and used exclusively as the test set. The remaining subjects formed the development dataset.

The development dataset was subsequently divided into training and validation subsets using Group Shuffle Split, with 20% of the development data assigned to the validation subset.

The same temporal grouping strategy used in the subject-dependent experiments was applied during this split. Consequently, temporally related windows belonging to the same temporal block were kept within the same partition.

The process was repeated for each subject, such that every subject was used exactly once as the held-out test subject. The resulting test performance therefore represents the model's ability to generalize to a previously unseen subject.

The splitting strategy was designed to prevent two different forms of information leakage.

- First, temporal leakage is avoided by keeping related windows within the same group. Without grouping, adjacent windows from the same recording could be placed in different subsets, allowing the model to encounter highly similar samples during training and evaluation.
- Second, subject leakage is prevented in the subject-independent scenario by keeping all data belonging to the held-out subject exclusively in the test set. This makes the subject-independent results a measure of cross-subject generalization rather than the model's ability to recognize previously observed subjects.

10.6.3. Hyperparameter optimization

Hyperparameter optimization was performed using the Optuna framework. Optuna was used to automatically evaluate different combinations of training and model hyperparameters and identify configurations that maximized classification accuracy.

Separate optimization procedures were performed for the subject-dependent and subject-independent experimental scenarios. The optimization was performed independently for each model architecture, since the architectures have different hyperparameters and computational requirements.

The optimization process was conducted iteratively rather than using a single fixed search space. Initial trials were performed over relatively broad parameter ranges. After each batch of trials, the highest-performing configurations were analyzed and the search ranges were adjusted based on the parameter values observed among the best-performing trials. This allowed subsequent trials to concentrate on regions of the search space that showed promising performance.

To determine the parameter ranges for the subsequent optimization stage, completed trials were sorted according to their validation accuracy. The best-performing trial was identified, after which a set of trials with similar performance was selected. The tolerance around the best accuracy was gradually increased until at least ten trials were included. The minimum and maximum values of each hyperparameter among these selected trials were then used to determine the search range for the following batch of trials.

This procedure was repeated several times, progressively narrowing the search space around promising hyperparameter configurations.

Table 3. Trials in the Optuna study

Optimization stage	Subjects included	Approx. trials
Initial	5	50–60
Second	12	30–50
Final	All subjects	50–60

The hyperparameters considered during optimization included both training and architectural parameters. The training parameters included the learning rate, weight decay, label smoothing, batch size and gradient accumulation steps. Depending on the model architecture, additional architectural parameters were optimized, such as the dropout rate, number of LSTM layers, LSTM hidden size, number of convolutional output channels and convolutional channel multiplication factor.

The activation function was fixed to GELU, while the maximum number of training epochs and early-stopping patience were kept constant. The specific search space depended on the architecture being optimized, as not all hyperparameters were applicable to every model.

Table 4. Initial hyperparameter search space

Hyperparameter	Search range / values
Learning rate	$5 \cdot 10^{-5}$ - $2.5 \cdot 10^{-4}$, logarithmic
Weight decay	$5 \cdot 10^{-5}$ - $3.5 \cdot 10^{-4}$, logarithmic
Label smoothing	0 - 0.55
Batch size	32, 64, 128, 256, 512, 1024
Gradient accumulation	1, 2, 4
Dropout	0 - 0.75
LSTM hidden size	32, 64, 128, 256, 512
LSTM layers	1, 2, 3
CNN output channels	32, 64, 128, 256
CNN channel multiplier	1, 2

Classification accuracy was used as the optimization objective. For each Optuna trial, the selected hyperparameter configuration was used to train and evaluate the corresponding model, and the mean accuracy across the evaluated subjects was returned to Optuna. Since the objective was to maximize classification accuracy, the Optuna study was configured with a maximization objective.

For the subject-dependent scenario, each Optuna trial evaluated the selected hyperparameter configuration independently for every subject. For each subject, the corresponding dataset was divided into training, validation and test subsets using grouped stratified splitting. The validation subset was used during training for model selection and early stopping, while the test subset was used to obtain the performance value for the subject. The accuracies obtained for all subjects were averaged and this mean accuracy was returned as the objective value for the Optuna trial.

The resulting mean accuracy was used solely as the optimization objective for selecting the hyperparameter configuration. The final reported model performance was subsequently obtained using the separate final evaluation procedure described in Section 10.6.4.

For the subject-independent scenario, Leave-One-Subject-Out (LOSO) evaluation was used during hyperparameter optimization. In each iteration, one subject was completely excluded from the training data and used as the test subject, while all remaining subjects were used for training. The training subjects were further divided into training and validation subsets using a grouped split, ensuring that samples originating from the same subject remained within the same subset.

The model was trained using the training subjects, with the validation subset used for model selection and early stopping. The resulting model was then evaluated on the held-out subject. This process was repeated until every subject had served as the held-out subject. The mean accuracy across all held-out subjects was returned as the Optuna objective value.

As in the subject-dependent optimization, these scores were used to compare hyperparameter configurations during optimization. The final performance reported in the results section was obtained separately using the final model evaluation procedure.

10.6.4. Final model training and evaluation

After the hyperparameter optimization stage, the best-performing hyperparameter configuration for each model was selected and used for the final evaluation. The evaluation procedure was performed separately for the subject-dependent and subject-independent scenarios. In both cases, the models were trained using the Adam optimizer and evaluated using classification accuracy.

For the subject-dependent scenario, each participant was evaluated independently. For a given participant, their data were divided into training, validation, and test subsets using the same grouped and stratified splitting procedure described in Section 10.6.2. The validation subset was used during training to monitor the model's performance and select the best model state through early stopping. The test subset

was not used during training or model selection and was evaluated only after the training process was completed.

Before training, the input features were standardized separately for each participant. The mean and standard deviation were calculated only from the training subset and subsequently applied to the validation and test subsets. This prevents information from the validation or test data from influencing the normalization parameters.

Each model was trained for a maximum of 150 epochs, with early stopping controlled by a patience value of 15 epochs. After training, the model state corresponding to the highest validation accuracy was restored and used to generate predictions for the test subset.

The procedure was repeated independently for every participant, with each participant therefore producing one test accuracy value. The resulting accuracies were then used to compare the performance of the different model architectures.

The subject-independent evaluation followed a LOSO procedure. In each iteration, all data belonging to one participant were reserved as the test set, while data from all remaining participants were used for training and validation. Consequently, the model was always evaluated on a participant whose data had not been used during training.

The training data were further divided into training and validation subsets using `GroupShuffleSplit`, with 20% of the available training data assigned to the validation subset. The participant identifier was used as the grouping variable, ensuring that samples from the same participant could not appear in both subsets.

As in the subject-dependent scenario, normalization parameters were calculated exclusively from the training data and then applied to the validation and test data. This is particularly important in the subject-independent scenario because using information from the held-out participant during preprocessing could introduce data leakage.

For each LOSO iteration, the model was trained using the selected hyperparameters, and the model state with the highest validation accuracy was restored before evaluating the held-out participant. This process was repeated until every participant had served exactly once as the test subject.

The final performance was therefore obtained from the collection of predictions generated across all LOSO folds. In addition to the overall performance, the accuracy of each individual fold was retained to allow analysis of performance variation between participants.

10.6.5. Reproducibility

Several measures were taken to ensure the reproducibility of the experiments. A fixed random seed was used during the final model training and evaluation. For the final experiments, the seed was set to 42 before each complete evaluation procedure. This ensures consistent initialization of the neural network models and reproducibility of other operations used during training.

The configuration of each final experiment was saved together with its results. The saved configuration contains the training hyperparameters, such as the learning rate, weight decay, number of training epochs, early-stopping patience, label smoothing, batch size and gradient accumulation steps. It also contains the model-specific

parameters, including the selected architecture, activation function, dropout rate and architecture-dependent parameters such as the number of layers and hidden or convolutional channels.

For each experiment, the trained model parameters were saved separately using the PyTorch *state_dict* format. In addition to the model parameters, the true labels, predicted labels, class probabilities and prediction confidence values were stored for each evaluated subject. The aggregated evaluation results were stored separately from the per-subject results.

The configuration of the windowed dataset used for an experiment was also stored using a unique configuration hash. This allows the results to be associated with the exact version of the windowed data used during training and evaluation.

Table 5. Saved file and there description

Stored artifact	Purpose
config.json	Training and model hyperparameters and windowing configuration identifier
result.json	Aggregated evaluation results
subject_id.pt	Trained model parameters for the corresponding subject/fold
y_true.npy	Ground-truth labels
y_pred.npy	Model predictions
prob.npy	Predicted class probabilities
conf.npy	Prediction confidence values

The final experiment directory contains both the information required to reproduce the training configuration and the information required to analyze the obtained results without repeating the experiment.

The versions of the main software components used during the experiments are as follows:

Table 6. Versions of the main software components

Component	Version
Python	3.10.12
PyTorch	2.10.0+cu128
CUDA	12.8

NVIDIA driver	610.88
JupyterLab	4.5.6
Optuna	4.9.0
NumPy	2.2.6
Polars	1.40.1
scikit-learn	1.7.2
Docker	29.6.2

The hardware configuration of the machine used to optimize and train the model is as follows:

Table 7. Versions of the main software components

Component	Name
Video card	Palit GeForce RTX 5070 TI
CPU	AMD Ryzen 7 9700X
Motherboard	ASUS TUF GAMING B650-PLUS WIFI
RAM	2x Kingston FURY 16GB 6000 30-36-36-80

11. Main results

This section presents the main results from the experiments with the different deep learning models. The results are divided into two main experiments: **subject-dependent** and **subject-independent** classification.

In the subject-dependent experiment, the model is trained and tested using data from the same subjects. In the subject-independent experiment, the subject used for testing is completely left out of the training data. This makes the second experiment more difficult because the model has to classify a person that it has not seen during training.

The tested models are CNN, LSTM, CNN-LSTM, LSTM-CNN, CNN-Transformer, CNN-LSTM-Fusion, and MLP. The main metrics used for comparison are accuracy, macro F1-score, balanced accuracy, macro precision, and macro recall. The results were also analyzed for individual subjects and individual classes. In addition, prediction confidence, calibration, statistical differences between the models, and model size were considered.

11.1. Final model configuration

The parameters were selected during the hyperparameter optimization process and were then used for the corresponding final experiments. Because different architectures can require different parameter values, the current results represent the performance of the complete optimized configurations rather than a controlled comparison using identical hyperparameters.

Table 8. Final parameters used for training the models in the subject-dependent experiment

Model	CNN	LSTM	CNN LSTM	LSTM CNN	CNN LSTM Fusion	CNN Transformer	MLP
Learning rate	0.00207	0.000119	0.0000615	0.0000759	0.0000981	0.000176	0.00073
Weight decay	0.00295	0.0000668	0.000319	0.00025	0.000319	0.00347	0.000163
Dropout	0.3837	0.0467	0.166	0.0899	0.163	0.743	0.293
Batch size	64	32	32	32	256	64	512
Accumulation step	1	1	1	2	1	1	1
Label smoothing	0.337	0.0571	0.0395	0.0896	0.836	0.517	0.427
Output channels	32	32	32	32	256	512	256
Output channels multiplier	1	1	1	1	1	1	1
Hidden size	-	512	128	256	32	512	-
Num of layers	-	2	2	3	2	2	-

Table 9. Final parameters used for training the models in the subject-independent experiment

Model	CNN	LSTM	CNN LSTM	LSTM CNN	CNN LSTM Fusion	CNN Transformer	MLP
Learning rate	0.0000665	0.0000653	0.0000879	0.00013	0.0000124	0.0000519	0.000116
Weight decay	0.000201	0.00009	0.000116	0.000147	0.000102	0.000132	0.000123
Dropout	0.231	0.0948	0.157	0.23	0.333	0.229	0.37
Batch size	256	64	256	128	1024	256	23
Accumulation step	4	1	2	4	4	1	1
Label smoothing	0.349	0.332	0.455	0.334	0.34	0.0358	0.346
Output channels	64	64	128	128	256	32	256
Output channels multiplier	1	2	1	1	2	1	1
Hidden size	-	128	64	64	256	64	-
Num of layers	-	2	2	2	1	1	-

11.2. Comparison between models

The results show a clear difference between the CNN/LSTM-based models and the MLP model. In the subject-dependent experiment, the CNN and LSTM-based models all achieved an accuracy of around 76–77%. The best result was achieved by **LSTM-CNN**, with an accuracy of **76.87%**, closely followed by **CNN-Transformer** with **76.84%**. CNN and CNN-LSTM-Fusion both achieved approximately **76.55%**, while LSTM achieved **76.50%**. The lowest result among these models was obtained by CNN-LSTM, with **75.89%**.

The differences between these six models are quite small. The difference between the best and worst model is less than one percentage point. This means that, when the model is tested on subjects that are also represented in the training data, the choice between these six architectures does not have a very large effect on the final accuracy.

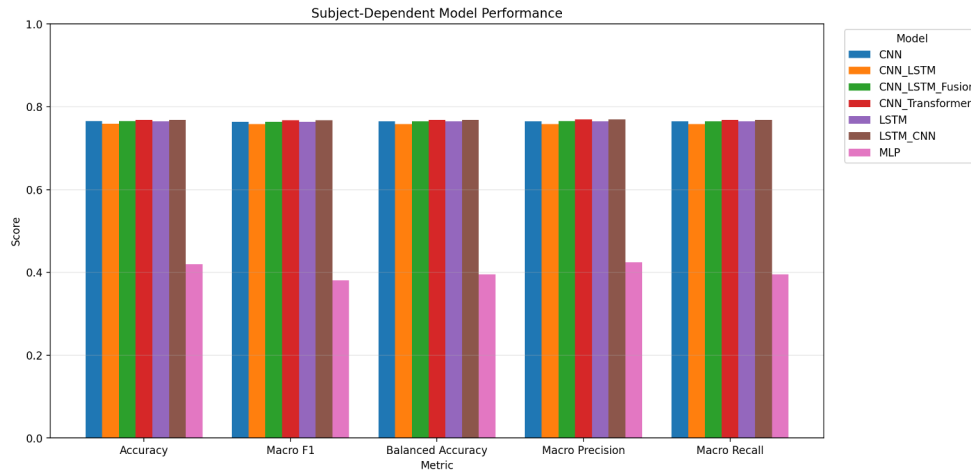


Figure 10. Overall subject-dependent model comparison. Shows that CNN/LSTM-based models all have between 76–77% Accuracy, Macro F1 and all other base metrics while MLP have only 41.96% Accuracy

The MLP performed much worse than the other models. Its subject-dependent accuracy was only **41.96%**, with a macro F1-score of **38.04%** and balanced accuracy of **39.58%**. This is a large difference compared with the approximately 76–77% accuracy achieved by the CNN/LSTM-based models.

This suggests that the temporal information in the input data is important for this classification problem. The MLP uses the extracted features without processing the data in the same temporal way as the CNN and recurrent models. Its much lower performance suggests that this is not enough to properly distinguish the three activities.

The statistical analysis also supports this result. Pairwise Wilcoxon signed-rank tests using the results from the 25 subjects showed statistically significant differences between the MLP and each of the six CNN/LSTM-based models for accuracy, with p-values below 0.001. However, no statistically significant difference was found between the six CNN/LSTM-based models themselves at the 0.05 significance level.

This means that although LSTM-CNN has the highest numerical accuracy, it cannot be said that it is clearly better than the other CNN/LSTM-based models based on the statistical tests. Their performance is too similar for such a conclusion.

The results are different in the subject-independent experiment. In this case, the best model was **CNN-LSTM-Fusion**, with an accuracy of **52.84%**. CNN achieved **51.96%**, while LSTM achieved **51.79%**. CNN-LSTM achieved **50.62%**, LSTM-CNN **50.43%**, and CNN-Transformer **49.91%**. The MLP achieved **44.40%**.

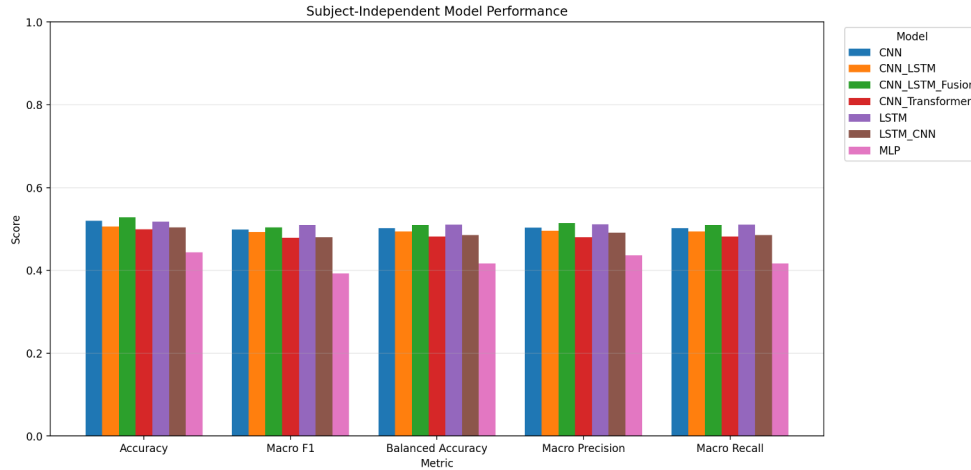


Figure 11. Overall subject-independent model comparison. Shows that CNN/LSTM-based models all have between 49–53% Accuracy, Macro F1 and all other base metrics while MLP have only 44.40% Accuracy

The subject-independent results also show some statistically significant differences between the models. For example, CNN-LSTM-Fusion performed significantly better than CNN-Transformer and LSTM-CNN. However, the differences between CNN-LSTM-Fusion and CNN-LSTM or LSTM were not statistically significant. The MLP also performed significantly worse than CNN-LSTM-Fusion.

It should also be noted that several pairwise comparisons were made, so the p-values should not be treated as absolute proof that one model is better than another. They are mainly useful for showing that some of the differences are larger than the differences between the other models.

The most important result from this comparison is not that one model is always the best. Instead, the results show that **the performance of the models depends strongly on whether the test subject was seen during training.**

11.3. Subject-dependent results

The subject-dependent experiment produced relatively high results for all CNN- and LSTM-based models. The highest accuracy was achieved by LSTM-CNN with **76.87%**, while CNN-Transformer achieved **76.84%**. Their macro F1-scores were also almost the same, at approximately **76.76%**.

Looking at the results for each subject shows that the models do not perform equally well for every participant. The average subject-level accuracy for the six main models was between approximately **74.79% and 75.51%**, with standard deviations of around 0.11–0.12. The accuracy for individual subjects ranged from approximately **42% to 94%**, depending on the model and subject.

This shows that some subjects are much easier for the models to classify than others. This difference is not only caused by the model architecture. It can also be related to differences between the participants, such as their gaze behaviour, head movements, execution speed, or the way they performed the assembly task.

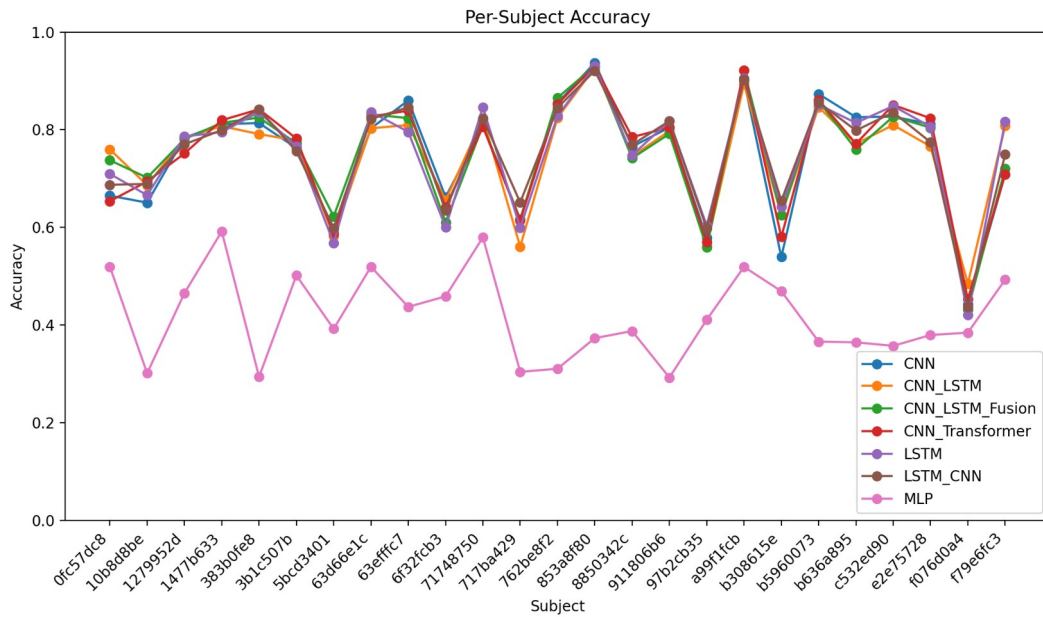


Figure 12.1. Subject-level accuracy comparison for the subject-dependent experiment. Small differences between the different CNN/LSTM-based models can be seen.

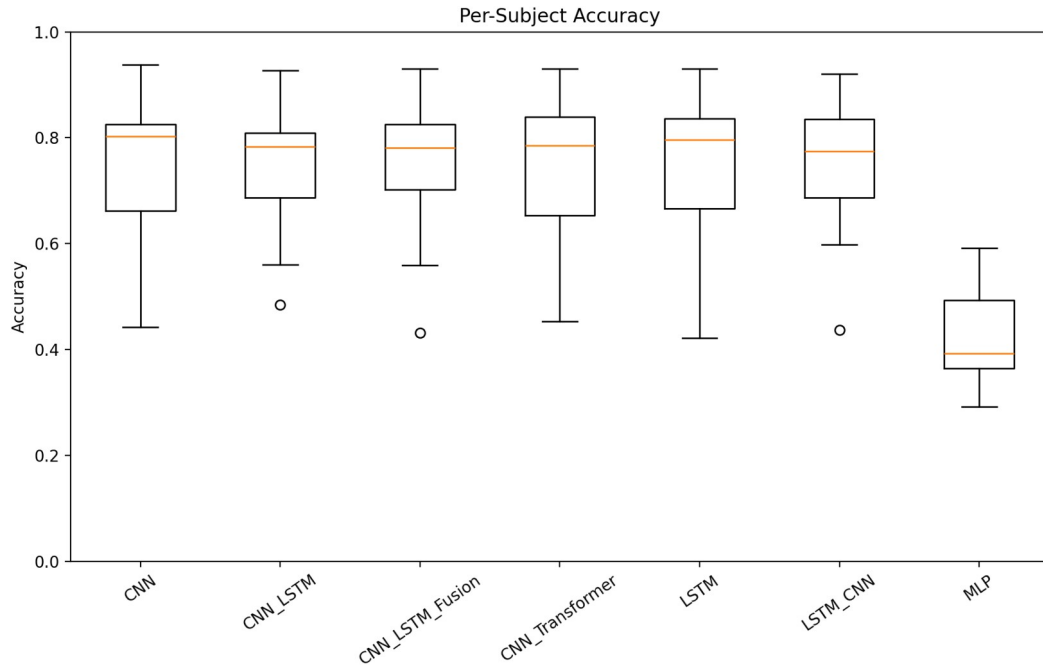


Figure 12.2. Subject-level accuracy comparison for the subject-dependent experiment. It can be seen that the model's accuracy have big difference depending on the subject.

The per-subject results also show that there is no single model that is clearly the best for every subject. CNN-Transformer and LSTM achieved the best accuracy for six of the 25 subjects each. CNN was the best model for five subjects, while CNN-LSTM-Fusion was the best for four subjects.

This supports the previous conclusion that the small differences between the main models are not large enough to say that one architecture is always better than the others.

The class-level results give some additional information. For the six main models, **Assembling** was generally the easiest class to recognize, with F1-scores around **0.79–0.80**. **Searching for pieces** was also classified relatively well, with F1-scores around **0.76–0.77**. **Looking at guide book** had slightly lower F1-scores of around **0.73–0.74**.

CNN-Transformer achieved the highest F1-score for Searching for pieces at approximately **0.77**, while LSTM-CNN also performed well for this class. However, again, the differences between the main models were relatively small.

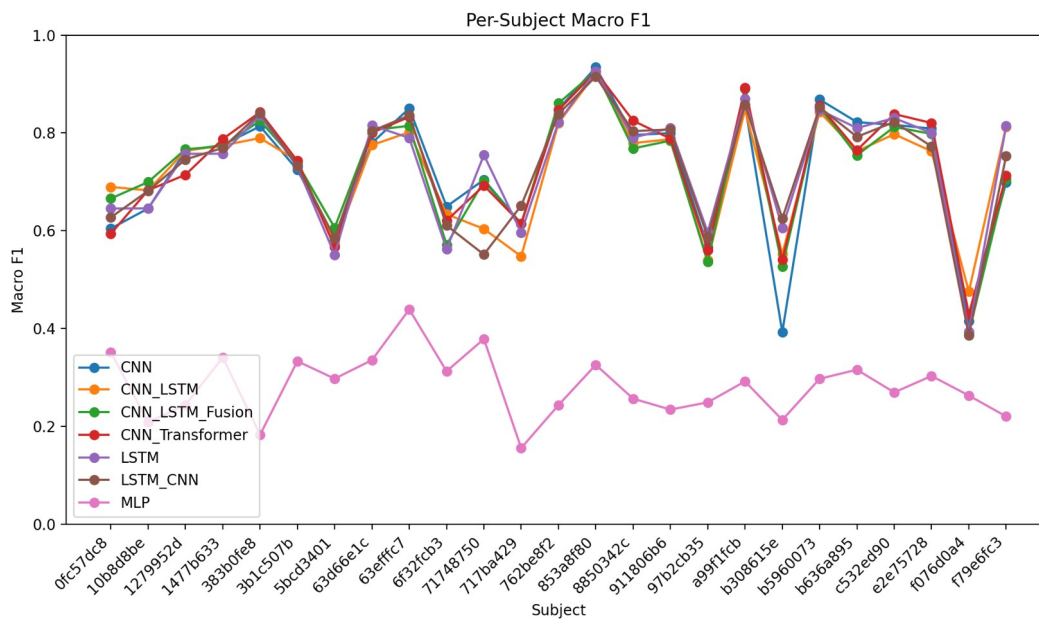


Figure 13.1. Per-class F1-score comparison for the subject-dependent experiment.

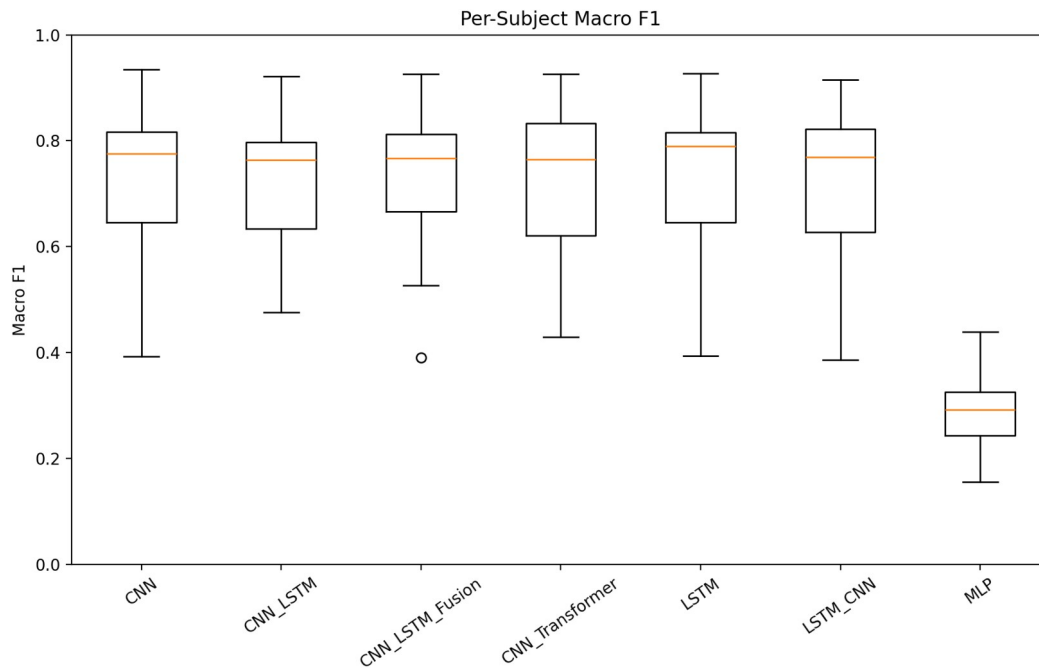


Figure 13.2. Per-class F1-score comparison for the subject-dependent experiment.

The MLP had much worse results for all three classes. Its F1-score was approximately **0.44** for Looking at guide book, **0.48** for Assembling, and only **0.22** for Searching for pieces. The particularly low result for Searching for pieces shows that the MLP has serious difficulties with this class.

Overall, the subject-dependent results show that the three activities can be classified reasonably well when the model has access to data from the same subjects during training. At the same time, the large differences between individual subjects show that there are important differences in how the participants perform the task.

11.4. Subject-independent results

The subject-independent experiment produced much lower results than the subject-dependent experiment. This is expected because the test subject was completely excluded from the training data. Therefore, the model had to classify behaviour from a person that it had not seen before.

The best result was achieved by **CNN-LSTM-Fusion**, with an accuracy of **52.84%**, balanced accuracy of **50.92%**, and macro F1-score of **50.45%**. CNN was the second-best model with **51.96%** accuracy, followed by LSTM with **51.79%**. The other CNN/LSTM-based models achieved around 50%, while the MLP achieved **44.40%**.

The difference compared with the subject-dependent results is quite large. For example, CNN-LSTM-Fusion decreased from **76.55% to 52.84%**, which is a decrease of approximately **23.7 percentage points**. LSTM-CNN decreased by approximately **26.4 percentage points**, while CNN-Transformer decreased by approximately **26.9 percentage points**.

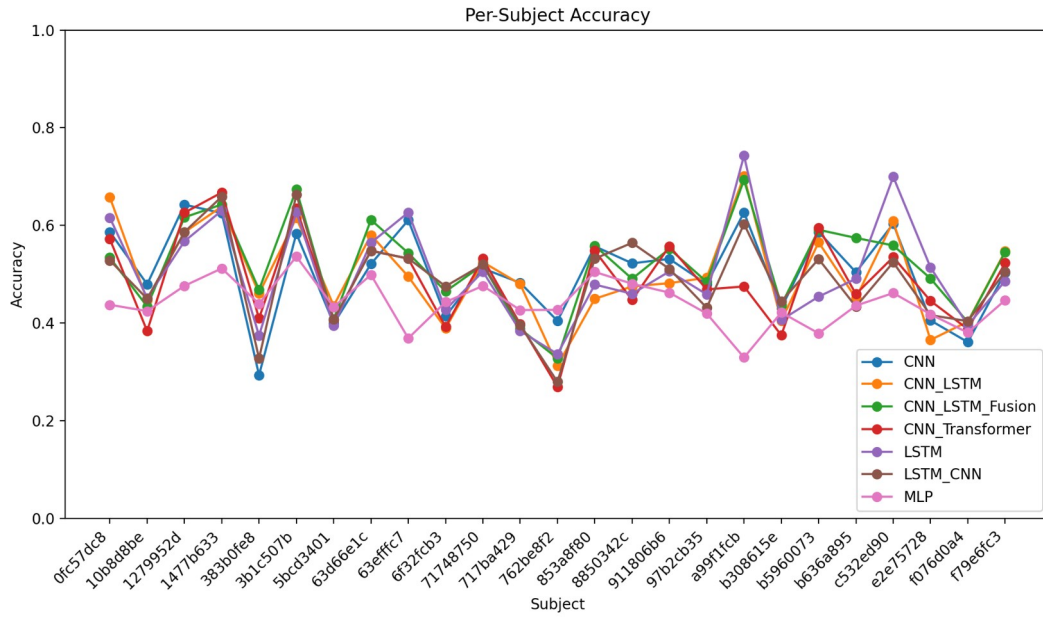


Figure 14.1. Subject-level accuracy comparison for the subject-independent experiment.

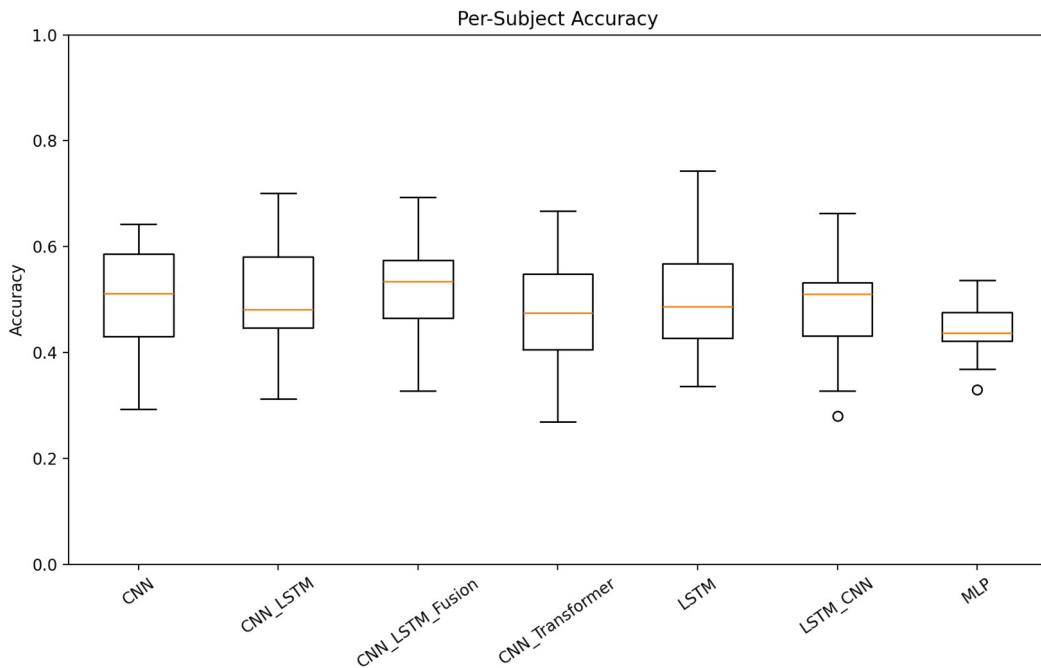


Figure 14.2. Subject-level accuracy comparison for the subject-independent experiment.

This shows that the models learn a significant amount of information that is specific to the individual subjects. When the model sees data from the same subjects during training, it can achieve around 76–77% accuracy. When it has to classify a completely new subject, the accuracy drops to around 50–53%.

The results for individual subjects show the same problem. The mean subject-independent accuracy of the main models was around 49–52%, with standard

deviations of approximately 0.09–0.11. Individual subject accuracies ranged from approximately **27% to 74%**, depending on the model and subject.

CNN-LSTM-Fusion was the most consistent model when looking at the different classes. It achieved the highest overall class-level results, with F1-scores of approximately **0.55** for Looking at guide book, **0.60** for Assembling, and **0.36** for Searching for pieces.

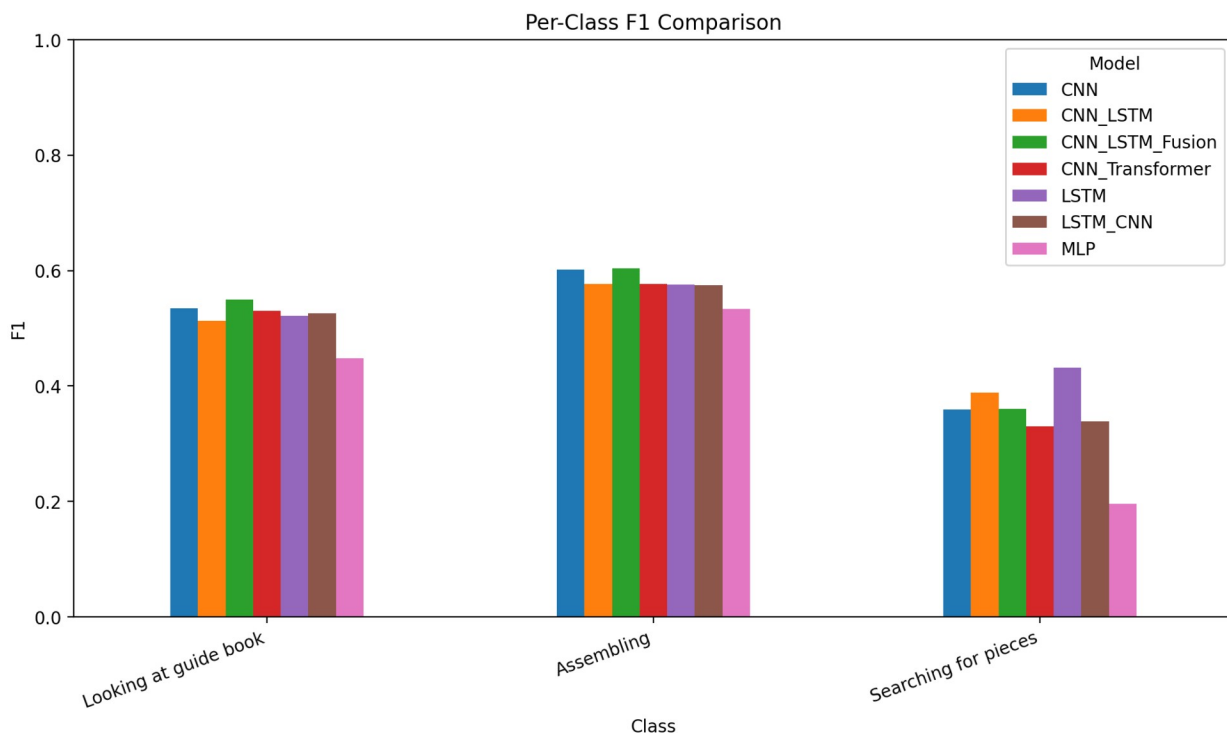


Figure 15. Per-class F1-score comparison for the subject-independent experiment.

The class results also show an important problem with **Searching for pieces**. For the main models, its F1-score was between approximately **0.33 and 0.43**, which is considerably lower than the results for the other two classes. The best result for this class was achieved by LSTM, with an F1-score of approximately **0.43**.

The recall results help explain this. In the subject-independent experiment, the recall for Searching for pieces was between approximately 0.28 and 0.44 depending on the model. This means that many examples of this activity were classified as one of the other activities.

On the other hand, **Assembling** was the easiest class to recognize, with F1-scores of approximately **0.58–0.60** for the main models. This could mean that the gaze and head movement patterns during active assembly are more similar between different subjects than the patterns produced while searching for pieces.

The MLP again performed poorly. Its subject-independent accuracy of **44.40%** was slightly higher than its subject-dependent accuracy of 41.96%, but its macro F1-score was still much lower than that of the other models. Therefore, the small increase in accuracy should not be interpreted as the MLP generalizing better. It is more likely related to differences in the class distribution and the way the model makes its predictions.

11.5. Analysis of the results

The biggest result of the experiments is the large difference between the subject-dependent and subject-independent results. The six CNN/LSTM-based models achieved approximately **76–77% accuracy** in the subject-dependent experiment, but only around **50–53%** in the subject-independent experiment.

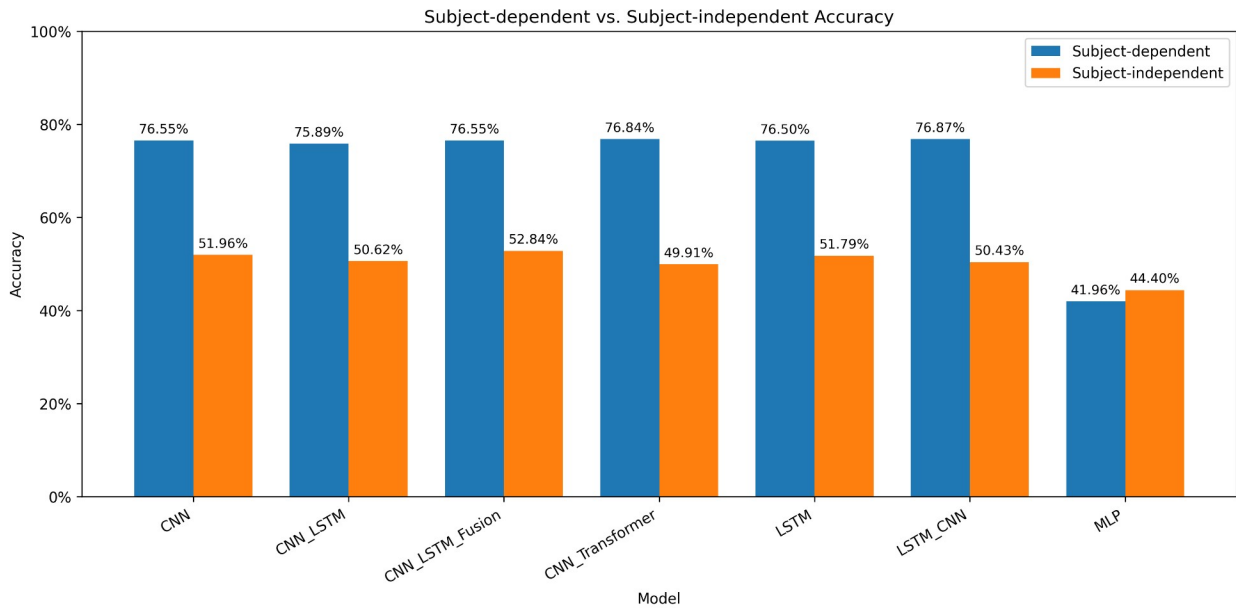


Figure 16. Accuracy decrease between subject-dependent and subject-independent experiments.

The largest decrease was observed for CNN-Transformer, which went from **76.84% to 49.91%**. LSTM-CNN also had a large decrease, from **76.87% to 50.43%**. CNN-LSTM-Fusion had the smallest decrease among the main models, going from **76.55% to 52.84%**.

This makes CNN-LSTM-Fusion the most promising model for the subject-independent experiment, even though it was not the best model in the subject-dependent experiment.

Another interesting result is that a larger model does not automatically produce better results. CNN-Transformer has approximately **9.2 million parameters** in the subject-dependent experiment, which is considerably more than some of the other models. However, its accuracy is almost the same as that of models with much fewer parameters.

CNN-LSTM-Fusion, on the other hand, achieved the best subject-independent result with approximately **1.1 million parameters**. This shows that simply increasing the number of parameters does not solve the problem of generalization to new subjects.

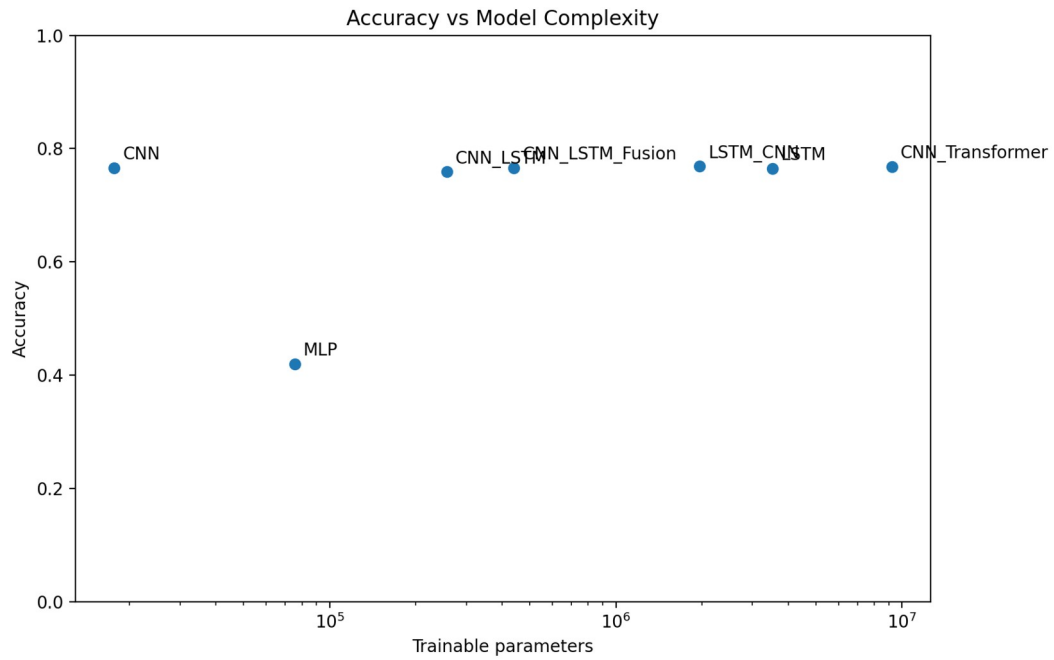


Figure 17.1. Accuracy compared with model complexity – subject-dependent.

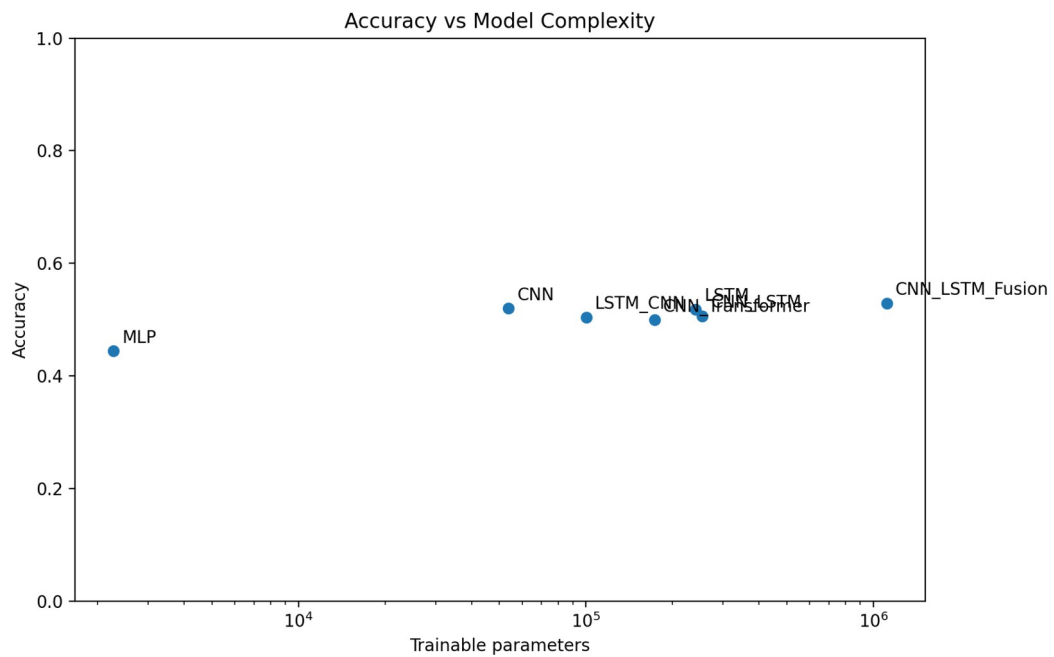


Figure 17.2. Accuracy compared with model complexity – subject-independent.

The confidence results also give some useful information. In the subject-dependent experiment, LSTM and CNN-LSTM had relatively high average confidence values of approximately **0.80** and **0.76**. CNN-LSTM-Fusion had a much lower average confidence of approximately **0.40**, even though its accuracy was approximately 76.5%. Its expected calibration error was also considerably higher than that of the other models.

This means that the confidence value produced by a model should not automatically be treated as a reliable estimate of how likely its prediction is to be correct.

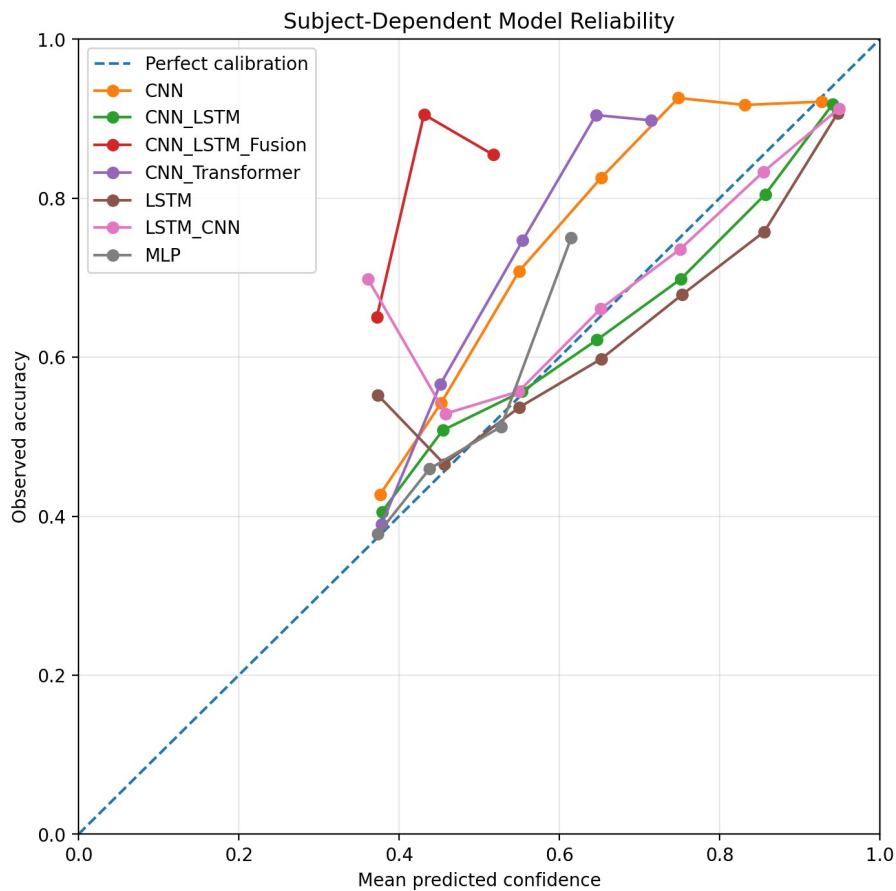


Figure 18.1. Reliability of the subject-dependent models.

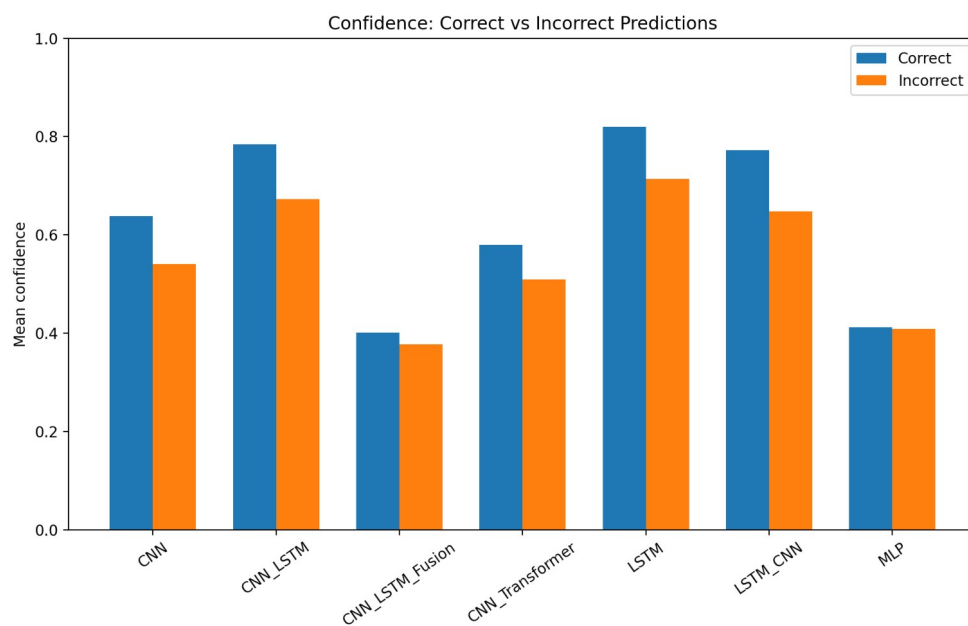


Figure 18.2. Reliability of the subject-dependent models.

In the subject-independent experiment, the confidence values for most models were closer to 0.5. This is reasonable because the models are much less certain when they are given data from a new subject. CNN-Transformer was different, producing an average confidence of approximately **0.59** despite having an accuracy of only around **0.50**. This suggests that it can be relatively confident even when its predictions are often wrong.

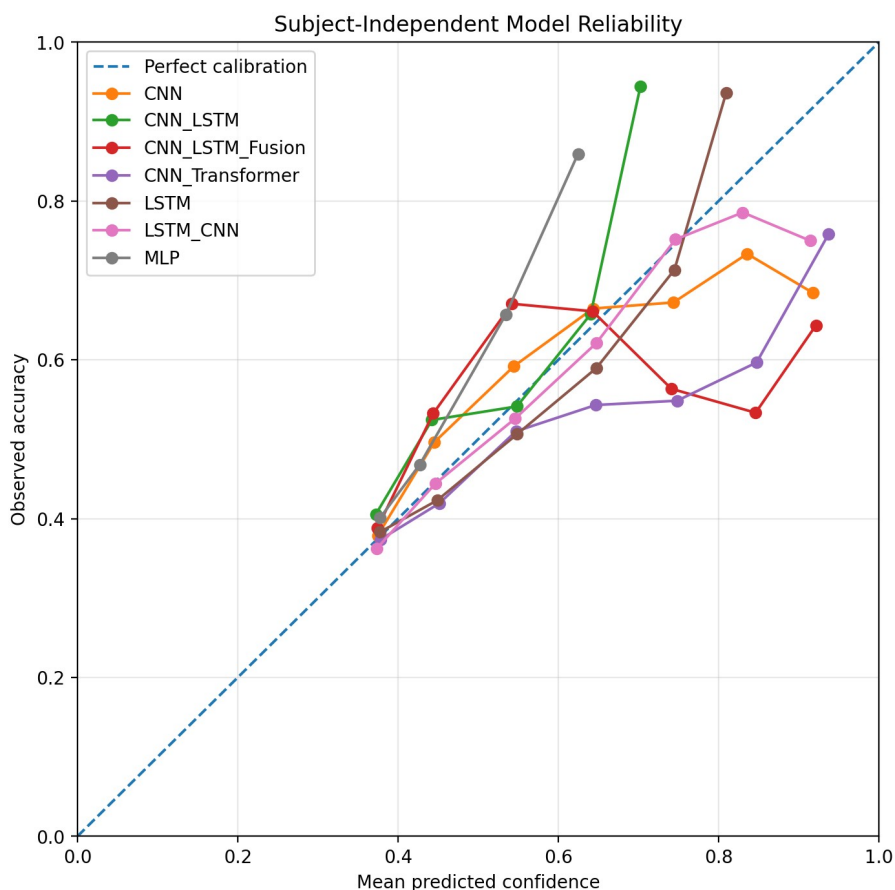


Figure 19.1. Reliability of the subject-independent models.

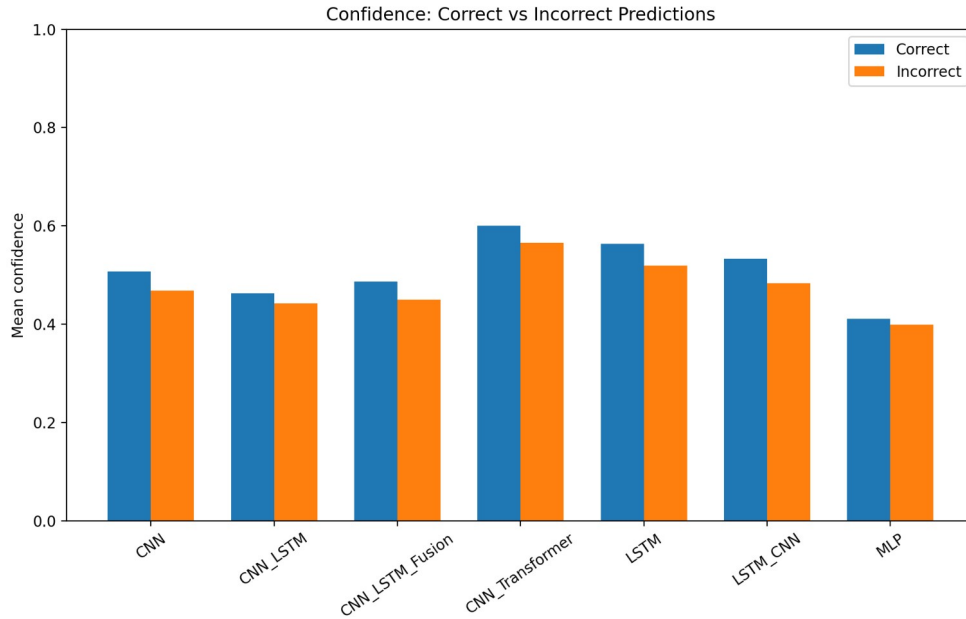


Figure 19.2. Reliability of the subject-independent models.

The class-level results also show that the main difficulty is not the same for all three activities. **Assembling** is consistently the easiest class to recognize, while **Searching for pieces** is the most difficult class in the subject-independent experiment.

The fact that Searching for pieces can reach an F1-score of approximately 0.76–0.77 in the subject-dependent experiment but only around 0.33–0.43 in the subject-independent experiment is especially important. This shows that the patterns used to identify this activity are strongly dependent on the individual subject.

Different participants can perform the searching activity in different ways, which can result in different gaze and head-motion patterns. This makes it more difficult for a model trained on one group of subjects to recognize the same activity in a new subject.

The results also show that there is no simple relationship between model size and performance. The CNN-Transformer model has many more parameters, but this does not give it better subject-independent performance. Therefore, increasing the model size alone is not enough to solve the generalization problem.

Overall, the experiments show that the multimodal gaze and head-motion data contains enough information to distinguish the three assembly activities when data from the same subjects is available during training. However, the large decrease in performance for unseen subjects shows that the models still learn a lot of subject-specific information.

The most promising architecture for generalization to new subjects is **CNN-LSTM-Fusion**, since it achieved the highest accuracy, macro F1-score, and balanced accuracy in the subject-independent experiment. For the subject-dependent experiment, **LSTM-CNN** achieved the highest numerical accuracy, although its advantage over the other CNN/LSTM-based models was not statistically significant.

Therefore, the main problem identified by the experiments is not that the models cannot classify the activities. Instead, the main problem is their **limited ability to generalize the learned patterns to completely new subjects**. This is especially visible for the Searching for pieces class.

Future work should therefore focus on reducing the influence of individual subject characteristics and improving the ability of the models to learn patterns that are common between different people.

12. *Conclusions, Limitations and Recommendations*

12.1. *Conclusions*

The main goal of this thesis was to investigate whether gaze and head movement information can be used to recognize different phases of a manual assembly task. Several deep learning architectures were tested using the collected multimodal data, and their performance was evaluated in both subject-dependent and subject-independent experiments.

The results show that the data contains useful information for distinguishing the three investigated activities. In the subject-dependent experiment, the main CNN- and LSTM-based models achieved approximately **76–77% accuracy**, which shows that the models were able to learn patterns related to the different activities. The best numerical result was achieved by LSTM-CNN with an accuracy of **76.87%**.

However, the subject-independent results were much lower. The best model, CNN-LSTM-Fusion, achieved an accuracy of **52.84%**. This large difference between the two experiments is one of the most important findings of the thesis. It shows that a significant part of the information learned by the models is related to individual subjects. In other words, the models can recognize the activities much better when they have already seen data from the same person during training.

The results also showed that the choice of architecture is not the only important factor. The CNN/LSTM-based models had very similar results in the subject-dependent experiment, despite having different structures and numbers of parameters. This suggests that having good input data and a suitable representation of the temporal information can be more important than simply making the model more complex.

Another important finding was the difference between the three classes. **Assembling** was generally the easiest activity to recognize, while **Searching for pieces** was the most difficult one, especially in the subject-independent experiment. This suggests that the behaviour during searching is more different between participants than the behaviour during active assembly.

The experiments therefore show that multimodal gaze and head movement data can be used for activity classification, but achieving good generalization to completely new subjects is much more difficult. The results also show that improving the quality and consistency of the collected data is likely to be at least as important as trying more complex machine learning models.

12.2. *Limitations*

One of the biggest limitations of this work was that the importance of the data quality and data collection process was underestimated at the beginning of the project. A large amount of time was spent developing the preprocessing pipeline and testing different models, while some problems in the original data had a much larger effect on the final results than initially expected.

The labeling of the data could also have been more precise. Some events did not have perfectly defined starting and ending points. Since the data was divided into fixed time windows, even a small error in the label boundaries could result in a window containing information from two different activities. This is especially important for activities that naturally transition into each other.

Another problem was the synchronization between the video and gaze data. The two sources were not perfectly synchronized, but this was discovered only after several participants had already been processed. Because the windowing process depended on the timing information, the synchronization problem could also affect where the windows started and ended. This made the problem more difficult to identify and resulted in additional time being spent checking and correcting the preprocessing process.

This experience shows that the data should be checked much earlier in a project. Before training different models, the synchronization, labels, timestamps, missing values, and window boundaries should be manually verified on several participants. This would make it possible to find problems before they affect a large part of the dataset.

The number of participants is another important limitation. The experiments were performed using **25 subjects**, and each subject had only **one recording**. Although this is enough to perform an initial investigation, it is a relatively small dataset for drawing strong conclusions about how well the models generalize to new people. In particular, having only one recording per subject makes it difficult to determine whether differences between recordings are caused by the subject or by other factors related to the particular recording session.

The use of a LEGO-based assembly task is also a limitation. The task was useful because it provided a controlled environment in which the three activities could be observed and labeled. However, assembling LEGO pieces is not the same as working on a real industrial assembly line. A real assembly task can contain more complicated movements, different tools, more objects, different working positions, and more variation in the order and duration of actions. Therefore, the results obtained in this controlled environment cannot automatically be applied to real industrial assembly.

Another limitation is that some planned experiments could not be completed because of the limited time available for the project. One of these was the comparison between hyperparameter optimization using **accuracy** and using **macro F1-score** as the optimization objective. This would have been useful because the three classes are not always equally difficult to recognize. Optimizing only accuracy can favour the more common or easier classes, while macro F1-score gives equal importance to all classes.

I also planned to train the same model architectures using **fixed model parameters** instead of the parameters selected through hyperparameter optimization. This would have allowed a better comparison between the effect of the architecture itself and the effect of the selected hyperparameters. For example, it would have been possible to determine whether the differences between the architectures remain when they are trained under the same fixed settings. Since this experiment was not completed, the current comparison mainly shows the performance of the models together with their selected hyperparameters, rather than a completely controlled comparison of the architectures.

These experiments would have provided additional information about the models, but they were not completed within the available time.

Finally, the amount of data available for the experiments limited the conclusions that could be made about the models. More data would not only provide more training examples, but would also make the statistical comparison between subjects and models more reliable.

12.3. Recommendations

The first recommendation for future work is to improve the **data collection and labeling process before experimenting with more advanced models**. The synchronization between all recording devices should be checked at the beginning of the experiment. A simple synchronization event that is clearly visible in all data sources could be used to verify that the timestamps match correctly.

The labeling process should also be made more precise. The start and end of each activity should be defined using clear rules, and transitions between activities should be handled consistently. It would also be useful to have multiple people independently label at least part of the dataset. Their labels could then be compared to estimate how much uncertainty exists in the ground-truth labels.

A much larger dataset should be collected in future work. More participants would make the subject-independent evaluation more reliable, while multiple recordings from each participant would make it possible to study both **between-subject** and **within-subject** variation. Different sessions could also be performed on different days to determine whether the models can generalize to changes in the same person's behaviour.

The future dataset should also contain a more realistic assembly task. LEGO is useful for an initial controlled experiment, but testing the approach on a real or simulated industrial assembly task would provide a better indication of whether the method can be used in a practical environment.

Another useful experiment would be to compare different optimization objectives. Hyperparameter optimization could be performed separately using accuracy and macro F1-score as the objective. The resulting models could then be compared on all evaluation metrics rather than only on the metric used during optimization. This would show whether optimizing for overall accuracy causes the model to sacrifice performance on the more difficult classes.

Repeating the experiments using fixed hyperparameters for all architectures would be another useful extension. The same training settings could be used for each model, while only the architecture is changed. This would provide a more controlled comparison and make it easier to determine how much of the difference in performance comes from the architecture itself and how much comes from the hyperparameter optimization. The fixed-parameter experiments could then be compared with the optimized experiments to determine whether hyperparameter tuning changes the ranking of the models.

The subject-independent results also suggest that future work should focus on reducing the influence of individual subjects. Possible approaches include collecting more diverse training data, using subject normalization, domain adaptation, or other methods specifically designed to learn features that are common between people. The fact that CNN-LSTM-Fusion performed best in the subject-independent experiment suggests that combining different types of temporal information may be useful, but more data is needed to determine whether this result is consistent.

It would also be useful to investigate the **Searching for pieces** class in more detail. Its large decrease in performance between the subject-dependent and subject-independent experiments suggests that this activity is strongly influenced by individual

behaviour. Examining the gaze trajectories, head movements, and temporal patterns for this class could help explain why it is more difficult to generalize.

Overall, the main lesson from this work is that **the quality and amount of data should be considered before the complexity of the model**. More advanced architectures can improve the results, but they cannot compensate for inaccurate labels, synchronization problems, limited recordings, or large differences between subjects. For this type of problem, future work would likely benefit more from a larger and better controlled dataset combined with a carefully validated preprocessing pipeline than from simply testing increasingly complex neural network architectures.

13. References

- [1] [Dr. K. Rayner, Psychology Department, University of California, San Diego; Dr. M. Castelhana, Department of Psychology, Queen's University, Canada; Eye movements; October, 2007](#)
- [2] [Haodong Chen, Department of Mechanical Engineering, University of Maryland, United States of America, Niloofar Zendeheidi and Ming C. Leu, Department of Mechanical and Aerospace Engineering, Missouri University of Science and Technology, United States of America, Zhaozheng Yin, Department of Biomedical Informatics and Department of Computer Science, Stony Brook University, United States of America; A gaze-driven manufacturing assembly assistant system with integrated step recognition, repetition analysis, and real-time feedback; April, 2025](#)
- [3] <https://docs.docker.com/compose/>
- [4] <https://docs.docker.com/reference/cli/docker/container/run/#interactive>
- [5] <https://docs.docker.com/reference/dockerfile/>
- [6] <https://docs.docker.com/get-started/docker-overview/>
- [7] <https://docs.docker.com/reference/compose-file/deploy/>
- [8] https://en.wikipedia.org/wiki/Comma-separated_values
- [9] <https://jupyter-server.readthedocs.io/en/latest/other/full-config.html#options>
- [10] <https://laptopmedia.com/bg/series/acer-aspire-5-a515-51-a515-51g/>
- [11] <https://lego.bravostore.bg/products/lego-creator-31134>
- [12] https://packaging.python.org/en/latest/key_projects/#pip
- [13] https://pip.pypa.io/en/stable/cli/pip_install/#pip-install
- [14] <https://pip.pypa.io/en/stable/topics/caching/>
- [15] <https://people.debian.org/~jak/apt-doc/man/apt-get.8.html>
- [16] <https://pupil-labs.com/about>
- [17] <https://pupil-labs.com/products/neon>
- [18] <https://pupil-labs.com/products/neon/software>
- [19] <https://pupil-labs.com/products/neon/specs>
- [20] <https://pupil-labs.com/products/neon/neonnet>
- [21] <https://www.cloudflare.com/learning/dns/glossary/dynamic-dns/>
- [22] https://www.gnu.org/software/bash/manual/html_node/Lists.html
- [23] <https://www.ibm.com/docs/en/aix/7.1.0?topic=r-rm-command>
- [24] https://wires.onlinelibrary.wiley.com/doi/abs/10.1002/wcs.1323?casa_token=A_yvrpreDXQAAAAA%3AUGndcySO1wi56WqXFITgPqBkR9FILVP8RxMSj6QIPMhuDNNPp5NlennPYsQR3wbflLM8oX2ciWalFbs
- [25] <https://yaml.org/spec/1.2.2/#chapter-1-introduction-to-yaml>
- [26] [Jiazhen Pang, Pai Zheng, Junming Fan, Tianyuan Liu; Department of Industrial and Systems Engineering, The Hong Kong Polytechnic University, Hong Kong](#)

Special Administrative Region, China; Towards cognition-augmented human-centric assembly: A visual computation perspective; December, 2023

[27] Máté Köles; A Review of Pupillometry for Human-computer Interaction Studies; September, 2017

[28] Maria K. Eckstein, Belén Guerra-Carrillo, Alison T. Miller Singley , Silvia A. Bunge; Department of Psychology, University of California at Berkeley, United States and Helen Wills Neuroscience Institute, University of California at Berkeley, United States; Beyond eye gaze: What else can eyetracking reveal about cognition and cognitive development?; May, 2017

[29] Pooneh. R. Tabrizi, Control and Intelligent Processing Center of Excellence, School of Electrical, Reza. A. Zoroofi, Computer Engineering, University of Tehran, Tehran, Iran; Open/Closed Eye Analysis for Drowsiness Detection; 23-26 November 2008

[30] Valerio Belcamino, Mariya Kilina, Alessandro Carfi and Akira Shimada, Università degli studi di Genova, Italy, Miwa Takase, Sota Shimizu and Fulvio Mastrogiovanni, Shibaura Institute of Technology, Japan; Gaze-Based Intention Recognition for Human-Robot Collaboration; June, 2024

[31] What is eye tracking? - Pupil Labs; December 10, 2020

14. Applications

APPLICATION A

Declaration of informed consent



UNIVERSITÀ DI PAVIA
Department of Electrical,
Computer and Biomedical
Engineering

INFORMED CONSENT FOR RESEARCH

Research title: *Gaze analysis during the assembly of LEGO models*

Dear Volunteer,

You are invited to participate in a research study

It is your right to be informed about the purpose and characteristics of the study so that you can make an informed and free decision to participate. Please read what follows carefully. The researchers involved in this project are available to answer your questions:

- Georgi Chaparov (Erasmus student visiting the University of Pavia from the University of Ruse, Bulgaria, email georgechaparov@gmail.com)
- Marco Porta (local thesis supervisor, email marco.porta@unipv.it, phone 0382 985214)
- Piercarlo Dondi (local thesis co-supervisor, piercarlo.dondi@unipv.it, 0382 985372)

What is the purpose of this study?

The present study aims to investigate the potential of eye tracking for analyzing gaze behavior during LEGO model assembly. At least in its initial phase, the study is carried out as part of the student Georgi Chaparov's bachelor's thesis.

How will the study be implemented?

The study involves collecting experimental data from different subjects, which will be analyzed using various techniques.

Why do we ask you to participate?

We ask you to participate because the more subjects there are, the better! There are no particular selection criteria.

Do you have to participate in the study?

Your participation is entirely free. Furthermore, should you change your mind during the tests and wish to withdraw, you may do so at any time without providing an explanation.

What are the steps to participate in the study?

You can participate in the study only after being informed about its characteristics. At the end of this information phase, you can agree to participate in the study by signing the informed consent form. You can actively participate in the proposed study only after you have given your consent.

What will you be asked to do?

You will be asked to assemble a simple LEGO model.

What are the possible risks of the study?

There are no known risks associated with this study. The eye-tracking data will be collected using a wearable *eye tracker*, which resembles a pair of glasses and does not cause discomfort.



How is the confidentiality of information guaranteed?

The only explicit personal data collected is gender and age. The analysis will be carried out on de-identified data by assigning a code to each participating subject. The eye tracker does not record the person's face; it only records eye data.

How will your personal information be used?

The data collected by the eye tracker may be used, **only in anonymous and aggregate form**, for theses and scientific publications, in accordance with Article 13 of the EU Regulation 2016/679-RGDP on the processing of personal data for scientific research purposes. The data may also be made available in publicly accessible datasets, while remaining fully anonymized.

We thank you for your collaboration!

DECLARATION OF THE RESEARCHER

I declare that I have provided the participant with complete information and detailed explanations about this research project's nature, purpose, and procedures.

GEORGE CHAPAROV	Signature		Date	
Name and surname of the researcher (in block letters)				
SIGNATURE OF THE PARTICIPANT				

I declare that I have received information sufficient to understand the research project, as well as the further clarifications I requested. I confirm that I received a copy of this information document.

	Signature	Date	
Name and surname of the participant (in block letters)			



UNIVERSITÀ DI PAVIA
**Department of Electrical,
Computer and Biomedical
Engineering**

INFORMED CONSENT DECLARATION

I, the undersigned _____

- Declare that I have received exhaustive explanations regarding the request to participate in the experimental study.
- Declare that I have been able to discuss these explanations, ask all the necessary questions, and have received satisfactory answers on the matter.
- Declare that I have been informed of my right to withdraw from the research at any time.

Therefore, in light of the information that has been provided to me:

☐ I AGREE	☐ I DO NOT AGREE	To participate in the study
----------------------------------	---	-----------------------------

Pavia,

Signature of the participant

Signature of the researcher

APPLICATION B

Docker Compose file for self-hosted Pi-hole service

```
services:
  pihole:
    container_name: pihole

    image: pihole/pihole:latest

    restart: unless-stopped

    volumes:
      - './etc-pihole:/etc/pihole'

    environment:
      TZ: 'Europe/Sofia'

      FTLCONF_webserver_api_password: ${PIHOLE_PASSWORD}
      FTLCONF_dns_listeningMode: 'ALL'

    ports:
      # DNS Ports
      - "53:53/tcp"
      - "53:53/udp"
      # DHCP server
      - "67:67/udp"

    expose:
      # Default HTTP Port
      - "80/tcp"

    networks:
      - caddy_proxy

    cap_add:
      # See https://github.com/pi-hole/docker-pi-hole#note-on-capabilities
      - NET_ADMIN
      - SYS_NICE

networks:
  caddy_proxy:
    external: true
```

APPLICATION C

Docker Compose file for self-hosted Caddy service

```
services:
  caddy:
    container_name: caddy

    image: caddy:latest

    restart: unless-stopped

  volumes:
    - ./config:/etc/caddy
    - ./srv:/srv
    - caddy_data:/data
    - caddy_config:/config

  environment:
    - SERVER_IP=${SERVER_IP}
    - JUPYTER_IP=${JUPYTER_IP}

  ports:
    - "80:80"
    - "443:443"
    - "443:443/udp"

  networks:
    - caddy_proxy
volumes:
  caddy_data:
  caddy_config:
networks:
  caddy_proxy:
    external: true
```

APPLICATION D

Docker Compose file for self-hosted WireGuard service

```
services:
  wireguard:
    container_name: wireguard

    image: lscr.io/linuxserver/wireguard:latest

    restart: unless-stopped

    volumes:
      - ./config:/config
      - ./modules:/lib/modules

    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Europe/Sofia
      - SERVERURL=${SERVER_URL}
      - SERVERPORT=${SERVER_Port}
      - PEERS=4
      - PEERDNS=auto

    network_mode: "host"

    cap_add:
      - NET_ADMIN
      - SYS_MODULE

    privileged: true
```

APPLICATION E

Preprocessing code

```
from pathlib import Path
import polars as pl

data_path = "../data/"
is_training_data = True

if is_training_data:
    saving_path = data_path + "Final Training Data/"
    working_data_path = data_path + "Training Data/"
else:
    saving_path = data_path + "Final Data For Labeling/"
    working_data_path = data_path + "Data For Labeling/"

Path(saving_path).mkdir(parents=True, exist_ok=True)

def read_csv(path: Path, schema_overrides=None):
    return pl.scan_csv(path, schema_overrides=schema_overrides)

# -----
# Process labels per recording.
# -----
def process_labels_for_recording(lf: pl.LazyFrame) -> pl.LazyFrame:
    """Process labels for a single recording."""
    return (
        lf
        .sort("start timestamp [ms]")
        .with_columns([
            pl.col("label").shift(1).alias("_prev_label"),
            pl.col("end timestamp [ms]").shift(1).alias("_prev_end"),
        ])
        .with_columns(
            (
                (pl.col("label") != pl.col("_prev_label"))
                | (pl.col("start timestamp [ms]") != pl.col("_prev_end"))
            )
        )
        .cast(pl.Int64)
        .cum_sum()
        .alias("group")
    )
    .group_by("group")
    .agg([
        pl.col("start timestamp [ms]").first(),
        pl.col("end timestamp [ms]").last(),
    ])
```

```

        pl.col("label").first(),
    ])
    .drop("group")
)

# -----
# Label gaze for a recording
# -----
def label_gaze_for_recording(gaze: pl.LazyFrame, labels: pl.LazyFrame) ->
pl.LazyFrame:
    gaze = gaze.with_columns(
        ((pl.col("timestamp [ns]") - pl.col("timestamp [ns]").min()) / 1_000_000)
        .cast(pl.Int64)
        .alias("time [ms]")
    )
    gaze = gaze.sort("time [ms]")
    labels = labels.sort("start timestamp [ms]")

    joined = gaze.join_asof(
        labels,
        left_on="time [ms]",
        right_on="start timestamp [ms]",
        strategy="backward",
    )
    return joined.filter(pl.col("time [ms]") <= pl.col("end timestamp [ms]"))

def process_recording(rec_dir: Path, is_training: bool, output_dir: Path):
    rec_id = rec_dir.name

    # 1. Read all relevant CSVs for this recording
    gaze_if = read_csv(rec_dir / "gaze.csv", schema_overrides={"fixation id": pl.Utf8,
"blink id": pl.Utf8, "saccades id": pl.Utf8})
    saccades_if = read_csv(rec_dir / "saccades.csv", schema_overrides={"saccades id":
pl.Utf8})
    fixations_if = read_csv(rec_dir / "fixations.csv", schema_overrides={"fixation id":
pl.Utf8})
    blinks_if = read_csv(rec_dir / "blinks.csv", schema_overrides={"blink id": pl.Utf8})
    imu_if = read_csv(rec_dir / "imu.csv")
    eye3d_if = read_csv(rec_dir / "3d_eye_states.csv")
    if is_training:
        labels_if = read_csv(rec_dir / "labels.csv")
    else:
        labels_if = None

    # 2. Filter and drop early
    gaze_if = gaze_if.filter(pl.col("worn") != 0)
    gaze_if = gaze_if.drop([
        "worn",

```

```

    "gaze mono left x [px]",
    "gaze mono left y [px]",
    "gaze mono right x [px]",
    "gaze mono right y [px]",
  ])

# 3. Process labels if training
if is_training and labels_lf is not None:
    labels_lf = process_labels_for_recording(labels_lf)
    gaze_lf = label_gaze_for_recording(gaze_lf, labels_lf)
    gaze_lf = gaze_lf.filter(pl.col("label") != "Undefined")

# 4. Join with saccades
saccades_lf = saccades_lf.drop("section id")
gaze_lf = gaze_lf.sort("timestamp [ns]")
saccades_lf = saccades_lf.sort("start timestamp [ns]")
gaze_lf = gaze_lf.join_asof(
    saccades_lf,
    left_on="timestamp [ns]",
    right_on="start timestamp [ns]",
    strategy="backward",
).filter(pl.col("timestamp [ns]") <= pl.col("end timestamp [ns]"))

# 5. Join fixations and blinks
fix_cols = ["fixation id", "duration [ms]", "fixation x [px]", "fixation y [px]"]
sacc_cols = ["saccade id", "duration [ms]", "amplitude [deg]", "mean velocity [px/s]",
"peak velocity [px/s]"]
blink_cols = ["blink id", "duration [ms]"]

gaze_lf = gaze_lf.join(fixations_lf.select(fix_cols), on="fixation id", how="left", suffix="
fix")
gaze_lf = gaze_lf.join(saccades_lf.select(sacc_cols), on="saccade id", how="left",
suffix=" sacc")
gaze_lf = gaze_lf.join(blinks_lf.select(blink_cols), on="blink id", how="left", suffix="
blink")

# 6. Merge IMU and 3D
imu_lf = imu_lf.sort("timestamp [ns]")
eye3d_lf = eye3d_lf.sort("timestamp [ns]")
gaze_lf = gaze_lf.sort("timestamp [ns]")
gaze_lf = gaze_lf.join_asof(imu_lf, on="timestamp [ns]", strategy="nearest",
tolerance=50_000_000, suffix=" imu")
gaze_lf = gaze_lf.join_asof(eye3d_lf, on="timestamp [ns]", strategy="nearest",
tolerance=50_000_000, suffix=" eye3d")

# 7. Drop extra columns
gaze_lf = gaze_lf.drop([
    "section id imu", "recording id imu",
    "section id eye3d", "recording id eye3d",
], strict=False)

```

```

# 8. Cast to Float32
float_cols = [
    "gaze x [px]", "gaze y [px]",
    "fixation x [px]", "fixation y [px]",
    "gyro x [deg/s]", "gyro y [deg/s]", "gyro z [deg/s]",
    "acceleration x [g]", "acceleration y [g]", "acceleration z [g]",
    "duration [ms]", "duration [ms] fix", "duration [ms] sacc", "duration [ms] blink",
]
gaze_lf = gaze_lf.with_columns([pl.col(c).cast(pl.Float32) for c in float_cols])

# 9. Fill nulls
gaze_lf = gaze_lf.with_columns([
    pl.col("fixation id").fill_null("unknown"),
    pl.col("blink id").fill_null("unknown"),
])
fill_cols = {
    "duration [ms] blink": -1.0,
    "fixation y [px]": -1.0,
    "fixation x [px]": -1.0,
    "duration [ms] fix": -1.0,
}
gaze_lf = gaze_lf.with_columns([pl.col(c).fill_null(v) for c, v in fill_cols.items()])

nan_cols = [
    col
    for col, dtype in gaze_lf.collect_schema().items()
    if dtype.is_float()
    and gaze_lf.select(pl.col(col).is_nan().any()).collect().item()
]

print(f"Cols with NaNs: {nan_cols}")
# 10. Write to per-recording Parquet
rec_output_path = output_dir / f"{rec_id}.parquet"
gaze_lf.sink_parquet(rec_output_path)

print(f"Processed recording: {rec_id} -> {rec_output_path.name}")

output_dir = Path(saving_path)
output_dir.mkdir(parents=True, exist_ok=True)

# Get all subdirectories that contain a "gaze.csv"
dirs = sorted([
    d for d in Path(working_data_path).iterdir()
    if d.is_dir() and not d.name.startswith(".") and (d / "gaze.csv").exists()
])

if not dirs:
    raise ValueError("No recording directories found (no gaze.csv found).")

```

```
for rec_dir in dirs:
    process_recording(rec_dir, is_training_data, output_dir)

combined = pl.scan_parquet(str(output_dir / "*.parquet"))
combined.sink_parquet(output_dir / "merged_output.parquet")
print("Combined Parquet saved as:", output_dir / "merged_output.parquet")
```

APPLICATION F

Windowing code

```
import numpy as np
import polars as pl
from pathlib import Path
import hashlib
import json
from sklearn.preprocessing import LabelEncoder
```

```
schema = {
    "section id": pl.Utf8,
    "recording id": pl.Utf8,

    "timestamp [ns]": pl.Int64,
    "gaze x [px]": pl.Float64,
    "gaze y [px]": pl.Float64,

    "fixation id": pl.Utf8,
    "blink id": pl.Utf8,
    "saccade id": pl.Utf8,

    "azimuth [deg]": pl.Float64,
    "elevation [deg]": pl.Float64,

    "start timestamp [ns]": pl.Int64,
    "end timestamp [ns]": pl.Int64,

    "duration [ms]": pl.Int64,
    "amplitude [px]": pl.Float64,
    "amplitude [deg]": pl.Float64,
    "mean velocity [px/s]": pl.Float64,
    "peak velocity [px/s]": pl.Float64,

    "timestamp [ms]": pl.Int64,

    "label": pl.Utf8,
    "norm_time": pl.Float64,
    "label_right": pl.Utf8,

    "duration [ms] fix": pl.Int64,
    "fixation x [px]": pl.Float64,
    "fixation y [px]": pl.Float64,

    "duration [ms] sacc": pl.Int64,
    "amplitude [deg] sacc": pl.Float64,
    "mean velocity [px/s] sacc": pl.Float64,
    "peak velocity [px/s] sacc": pl.Float64,
```

```

"duration [ms] blink": pl.Int64,

"gyro x [deg/s]": pl.Float64,
"gyro y [deg/s]": pl.Float64,
"gyro z [deg/s]": pl.Float64,

"acceleration x [g]": pl.Float64,
"acceleration y [g]": pl.Float64,
"acceleration z [g]": pl.Float64,

"roll [deg]": pl.Float64,
"pitch [deg]": pl.Float64,
"yaw [deg]": pl.Float64,

"quaternion w": pl.Float64,
"quaternion x": pl.Float64,
"quaternion y": pl.Float64,
"quaternion z": pl.Float64,

"pupil diameter left [mm]": pl.Float64,
"pupil diameter right [mm]": pl.Float64,

"eyeball center left x [mm]": pl.Float64,
"eyeball center left y [mm]": pl.Float64,
"eyeball center left z [mm]": pl.Float64,

"eyeball center right x [mm]": pl.Float64,
"eyeball center right y [mm]": pl.Float64,
"eyeball center right z [mm]": pl.Float64,

"optical axis left x": pl.Float64,
"optical axis left y": pl.Float64,
"optical axis left z": pl.Float64,

"optical axis right x": pl.Float64,
"optical axis right y": pl.Float64,
"optical axis right z": pl.Float64,

"eyelid angle top left [rad]": pl.Float64,
"eyelid angle bottom left [rad]": pl.Float64,
"eyelid aperture left [mm]": pl.Float64,

"eyelid angle top right [rad]": pl.Float64,
"eyelid angle bottom right [rad]": pl.Float64,
"eyelid aperture right [mm]": pl.Float64,
}

```

```

feature_cols = [
    "gaze x [px]",
    "gaze y [px]",
    "azimuth [deg]",
    "elevation [deg]",
    "pupil diameter left [mm]",
    "pupil diameter right [mm]",

    "optical axis left x",
    "optical axis left y",
    "optical axis left z",
    "optical axis right x",
    "optical axis right y",
    "optical axis right z",

    "gyro x [deg/s]",
    "gyro y [deg/s]",
    "gyro z [deg/s]",
    "acceleration x [g]",
    "acceleration y [g]",
    "acceleration z [g]",

    "roll [deg]",
    "pitch [deg]",
    "yaw [deg]",

    "fixation id",
    "blink id",
    "saccade id",
    "duration [ms]",
    "fixation x [px]",
    "fixation y [px]",
    "duration [ms] sacc",
    "amplitude [deg]",
    "mean velocity [px/s]",
    "peak velocity [px/s]",
    "duration [ms] blink",
    "quaternion w",
    "quaternion x",
    "quaternion y",
    "quaternion z",
    "eyeball center left x [mm]",
    "eyeball center left y [mm]",
    "eyeball center left z [mm]",
    "eyeball center right x [mm]",
    "eyeball center right y [mm]",
    "eyeball center right z [mm]",
    "eyelid angle top left [rad]",
    "eyelid angle bottom left [rad]",
    "eyelid aperture left [mm]",

```

```

    "eyelid angle top right [rad]",
    "eyelid angle bottom right [rad]",
    "eyelid aperture right [mm]",
]

def load_parquet(parquet_path):
    parquet_path = Path(parquet_path)

    if parquet_path.exists():
        print("Loading parquet...")
        return pl.scan_parquet(parquet_path)

    print("Parquet does not exist!!!!!!")
    return None

data_path = '../data/'
training_data_path = data_path + "Final Training Data/"
merged_df = load_parquet(training_data_path + "merged_output.parquet")
to_label_df = pl.read_csv(data_path + "Final Data For Labeling/" +
    "merged_output.csv")

labels = merged_df.select("label").unique().collect().to_series().to_numpy()

print(merged_df.select(pl.col("recording id").value_counts()).collect())

for label in merged_df.select(pl.col("label").value_counts()).collect():
    print(label)

le = LabelEncoder()
le.fit(labels)

label_map = dict(zip(le.classes_, range(len(le.classes_))))

merged_df = merged_df.with_columns(
    pl.col("label").replace_strict(label_map).alias("label")
)

def get_cache_dir(training_data_path, settings, feature_cols):
    config = {
        "window_size": settings["window size"],
        "overlap": settings["overlap"],
        "target_length": settings["target length"],
        "min_samples": settings["min samples"],
        "features": feature_cols,
        "use video min length": settings["use video min length"],
    }

```

```

        "use engineered features": settings["use engineered features"],
        "use raw features": settings["use raw features"],
        "video percentage": settings["video percentage"],
        "video offset": settings["video offset"]
    }

    config_str = json.dumps(config, sort_keys=True).encode()
    config_hash = hashlib.md5(config_str).hexdigest()

    cache_dir = Path(training_data_path) / "Windowed Data" / config_hash
    cache_dir.mkdir(parents=True, exist_ok=True)

    return cache_dir

def crop_video_percentage(df: pl.DataFrame, percentage: float, offset: float):
    if percentage >= 1.0 and offset == 0.0:
        return df

    if not (0 <= offset <= 1):
        raise ValueError("offset must be between 0 and 1")

    if not (0 < percentage <= 1):
        raise ValueError("percentage must be in (0, 1]")

    if offset + percentage > 1:
        raise ValueError("offset + percentage cannot exceed 1")

    t0 = df["timestamp [ns]"][0]
    t1 = df["timestamp [ns]"][-1]

    duration = t1 - t0

    start = t0 + offset * duration
    end = start + percentage * duration

    return df.filter(
        (pl.col("timestamp [ns]") >= start) &
        (pl.col("timestamp [ns]") < end)
    )

def build_cache(subjects, merged_df, feature_cols, settings, cache_dir):

    if settings["use video min length"]:
        subject_lengths = compute_subject_lengths(subjects, merged_df)
        min_len = min(subject_lengths.values())

        print("Shortest subject length:", min_len)

```

```

cache_dir = Path(cache_dir)
with open(cache_dir / "settings.json", "w", encoding="utf-8") as f:
    json.dump(settings, f, indent=4)

for s in subjects:
    path = cache_dir / f"{s}"
    path.mkdir(parents=True, exist_ok=True)

    extracted_features_path = path / f"{s}_extracted.npz"
    raw_features_path = path / f"{s}_raw.npz"
    window_times_path = path / f"{s}_window_times.npz"

    extracted_exists = extracted_features_path.exists()
    if extracted_exists:
        print(f"[CACHE] {s}_extracted exists")

    raw_exists = raw_features_path.exists()
    if raw_exists:
        print(f"[CACHE] {s}_raw exists")

    if extracted_exists and raw_exists:
        continue

    df_s = merged_df.filter(pl.col("recording id") == s).collect(engine="streaming")

    if settings["use video min length"]:
        df_s = df_s.sort("timestamp [ns]").head(min_len)
    else:
        df_s = crop_video_percentage(
            df_s,
            settings["video percentage"],
            settings["video offset"],
        )

    X, y, window_times = window_subject_raw(
        df_s,
        feature_cols,
        "label",
        settings
    )

    np.savez_compressed(window_times_path, window_times=window_times)

    if settings["use engineered features"] and raw_exists is False:
        extracted_windows = pl.DataFrame(
            [extract_features(window) for window in X]
        ).to_numpy().astype(np.float32)

        if np.isnan(extracted_windows).any():
            print("NaNs found before saving")

```

```

print(np.where(np.isnan(extracted_windows)))

nan_cols = [
    col
    for col, dtype in df_s.schema.items()
    if dtype.is_float()
    and df_s.select(pl.col(col).is_nan().any()).item()
]
print(f"Cols with NaNs: {nan_cols}")
raise ValueError("Feature extraction produced NaNs")

print(f"[CACHE] {s}_extracted BUILD")
np.savez_compressed(extracted_features_path, X=extracted_windows, y=y)

if settings["use raw features"] and raw_exists is False:
    raw_windows = np.stack([
        window.drop(["fixation id", "blink id", "saccade id", "timestamp
[ns]"]).to_numpy()
        for window in X
    ]).astype(np.float32)

    if np.isnan(raw_windows).any():
        print("NaNs found before saving")
        print(np.where(np.isnan(raw_windows)))

    nan_cols = [
        col
        for col, dtype in df_s.schema.items()
        if dtype.is_float()
        and df_s.select(pl.col(col).is_nan().any()).item()
    ]
    print(f"Cols with NaNs: {nan_cols}")
    raise ValueError("raw data have NaNs")

    print(f"[CACHE] {s}_raw BUILD")
    np.savez_compressed(raw_features_path, X=raw_windows, y=y)

def compute_features_per_subject(df: pl.DataFrame) -> pl.DataFrame:
    df = df.sort(["recording id", "timestamp [ns]"])

    df = df.with_columns([
        # --- diffs per recording ---
        pl.col("gaze x [px]").diff().over("recording id").alias("dx"),
        pl.col("gaze y [px]").diff().over("recording id").alias("dy"),
        pl.col("timestamp [ns]").diff().over("recording id").alias("dt"),
    ])

    df = df.with_columns([
        # --- velocity ---

```

```

(
    (pl.col("dx")**2 + pl.col("dy")**2).sqrt()
    / pl.when(pl.col("dt") == 0).then(None).otherwise(pl.col("dt"))
).alias("velocity").fill_null(0),

# --- pupil mean ---
(
    (pl.col("pupil diameter left [mm]") + pl.col("pupil diameter right [mm]")) / 2
).alias("pupil mean"),
])

# --- pupil normalization per recording ---
df = df.with_columns([
    ((pl.col("pupil mean") - pl.col("pupil mean").mean().over("recording id"))
    / (pl.col("pupil mean").std().over("recording id") + 1e-6)
    ).alias("pupil norm")
])

return df.drop(["dx", "dy", "dt"])

```

```
def extract_features(window: pl.DataFrame, width=None, height=None):
```

```

    def s(col):
        return window[col]

    features = {}

    vel = s("velocity").drop_nulls()

    # -----
    # Velocity
    # -----
    if len(vel) > 0:
        features.update({
            "vel mean": vel.mean(),
            "vel std": vel.std(),
            "vel max": vel.max(),
            "vel p25": vel.quantile(0.25),
            "vel p75": vel.quantile(0.75),
        })
    else:
        features.update({k: 0 for k in
            ["vel mean", "vel std", "vel max", "vel p25", "vel p75"]
        })

    # -----
    # Event ratios
    # -----
    features["saccade ratio"] = s("saccade id").is_not_null().mean()

```

```

features["fixation ratio"] = s("fixation id").is_not_null().mean()
features["blink ratio"] = s("blink id").is_not_null().mean()

features["num fixations"] = s("fixation id").n_unique()
features["num saccades"] = s("saccade id").n_unique()

# -----
# Fixation duration
# -----
fix = window.filter(pl.col("fixation id").is_not_null())

if fix.height > 0:
    fix_dur = (
        fix.group_by("fixation id")
        .agg((pl.col("timestamp [ns]").max() - pl.col("timestamp [ns]").min()).alias("dur"))
        ["dur"]
    )

    features.update({
        "fix dur mean": fix_dur.mean(),
        "fix dur std": fix_dur.std(),
        "fix dur max": fix_dur.max(),
        "fix dur p25": fix_dur.quantile(0.25),
        "fix dur p75": fix_dur.quantile(0.75),
    })
else:
    features.update({k: 0 for k in ["fix dur mean", "fix dur std", "fix dur max", "fix dur
p25", "fix dur p75"]})

# -----
# Saccade amplitude
# -----
if window.height > 1:
    dx = s("gaze x [px]").diff().drop_nulls()
    dy = s("gaze y [px]").diff().drop_nulls()

    sac_amp = (dx**2 + dy**2).sqrt()

    features.update({
        "sac amp mean": sac_amp.mean(),
        "sac amp std": sac_amp.std(),
        "sac amp max": sac_amp.max(),
    })
else:
    features.update({"sac amp mean": 0, "sac amp std": 0, "sac amp max": 0})

# -----
# Event-specific velocity
# -----
fix_vel = window.filter(pl.col("fixation id").is_not_null())["velocity"].drop_nulls()

```

```

sac_vel = window.filter(pl.col("saccade id").is_not_null())["velocity"].drop_nulls()

features["fix vel mean"] = fix_vel.mean() if len(fix_vel) else 0
features["sac vel mean"] = sac_vel.mean() if len(sac_vel) else 0
features["fix vel std"] = fix_vel.std() if len(fix_vel) else 0
features["sac vel std"] = sac_vel.std() if len(sac_vel) else 0

# -----
# Entropy
# -----
hist, _, _ = np.histogram2d(s("gaze x [px]"), s("gaze y [px]"), bins=10)
prob = hist / hist.sum() if hist.sum() > 0 else hist
prob = prob[prob > 0]

features["entropy"] = -np.sum(prob * np.log(prob)) if len(prob) else 0

# -----
# Dispersion
# -----
features["dispersion"] = (
    s("gaze x [px]").max() - s("gaze x [px]").min()
) + (
    s("gaze y [px]").max() - s("gaze y [px]").min()
)

# -----
# Gyro magnitude
# -----
gyro_mag = (
    (s("gyro x [deg/s]")**2 +
     s("gyro y [deg/s]")**2 +
     s("gyro z [deg/s]")**2)
    .sqrt()
)

if len(gyro_mag):
    features.update({
        "gyro mag mean": gyro_mag.mean(),
        "gyro mag std": gyro_mag.std(),
        "gyro mag max": gyro_mag.max(),
        "head stability": 1 / (gyro_mag.std() + 1e-5),
    })
else:
    features.update({"gyro mag mean": 0, "gyro mag std": 0, "gyro mag max": 0, "head
stability": 0})

# -----
# Pupil
# -----
features["pupil mean"] = s("pupil norm").mean()

```

```

features["pupil std"] = s("pupil norm").std()

# -----
# Inside frame
# -----
if width is not None and height is not None:
    inside = (
        (s("gaze x [px]") >= 0) &
        (s("gaze x [px]") <= width) &
        (s("gaze y [px]") >= 0) &
        (s("gaze y [px]") <= height)
    )
    features["inside ratio"] = inside.mean()

features = {
    k: 0.0 if v is None or np.isnan(v) else float(v)
    for k, v in features.items()
}

return features

```

```

def window_subject_raw(
    df: pl.DataFrame,
    feature_cols,
    label_col,
    settings
):
    window_size = int(settings["window size"] * 1e9)
    stride = int(window_size * (1 - settings["overlap"]))
    target_length = settings["target length"]

    df = df.sort("timestamp [ns]")

    times = df["timestamp [ns]"]
    max_time = times[-1]
    min_time = times[0]

    X = []
    y = []
    window_times = []

    start = min_time

    while start + window_size <= max_time:
        end = start + window_size

        window = df.filter(
            (pl.col("timestamp [ns]") >= start) &
            (pl.col("timestamp [ns]") < end)

```

```

)

if window.height < settings["min samples"]:
    start += stride
    continue
label = window[label_col][0]

window_features = window.select(feature_cols)

if window_features.height < settings["min samples"]:
    start += stride
    continue

window_center = start + window_size / 2

relative_time = (
    window_center - min_time
) / (
    max_time - min_time
)

window_times.append(relative_time)

window_features = window_features.with_columns(
    pl.lit(relative_time).alias("relative_time")
)

if window_features.height >= target_length:
    window_features = window_features.head(target_length)
else:
    pad_size = target_length - window_features.height

    last_row = window_features.tail(1)

    padding = pl.concat([last_row] * pad_size, how="vertical")

    window_features = pl.concat([window_features, padding], how="vertical")

X.append(window_features)
y.append(label)

start += stride

return X, np.asarray(y), np.asarray(window_times)

def compute_subject_lengths(subjects, merged_df):
    lengths = {}

    for s in subjects:

```

```

df_s = merged_df.filter(pl.col("recording id") == s).collect(engine="streaming")
lengths[s] = df_s.height

return lengths

settings = {
    "window size": 0.22,
    "overlap": 0.0,
    "target length": 45, # Sample rate is 200 Hz and window size is 0.22 so 45 is around
    22% of 200
    "min samples": 5,
    "use video min length": False,
    "video percentage": 0.65, # Fraction of the video to use (1.0 = 100%)
    "video offset": 0.0, # Fraction to skip from the beginning
    "use engineered features": True,
    "use raw features": True,
}

subjects = (
    merged_df.select("recording id")
    .unique()
    .sort("recording id")
    .collect(engine="streaming")
    .to_series()
    .to_list()
)

cache_dir = get_cache_dir(training_data_path, settings, feature_cols)
print(f"Using cache dir: {cache_dir}")

merged_df = compute_features_per_subject(merged_df)
feature_cols.append("timestamp [ns]")
feature_cols.append('velocity')
feature_cols.append('pupil mean')
feature_cols.append('pupil norm')

build_cache(
    subjects=subjects,
    merged_df=merged_df,
    feature_cols=feature_cols,
    settings=settings,
    cache_dir=cache_dir
)

```

APPLICATION G

Code for training subject-independent models

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE
import numpy as np
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
import copy
import json
from pathlib import Path
import torch.profiler
from enum import Enum
from sklearn.model_selection import GroupShuffleSplit
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    balanced_accuracy_score,
    confusion_matrix,
    classification_report,
)
import matplotlib.pyplot as plt
import os
import random

torch.set_float32_matmul_precision('high')
torch.backends.cuda.matmul.allow_tf32 = False
torch.backends.cudnn.allow_tf32 = False
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
torch.use_deterministic_algorithms(True)

def seed_all(seed=42):
    os.environ["PYTHONHASHSEED"] = str(seed)

    random.seed(seed)
    np.random.seed(seed)

    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

class ModelType(Enum):
    CNN = 1
    LSTM = 2
```

```

CNN_LSTM = 3
LSTM_CNN = 4
CNN_LSTM_Fusion = 5
CNN_Transformer = 6
MLP = 7

```

```

class EvalMetric(Enum):
    accuracy = 1
    F1 = 2
    custom = 3

```

```

class ExperimentType(Enum):
    fixed_model_params = 1
    best_model = 2

```

```

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

```

```

torch.backends.cudnn.benchmark = True

```

```

print(device)

```

```

root = "../.."
data_path = f'{root}data/'
training_data_path = data_path + "Final Training Data/"

```

```

class CNN(nn.Module):
    def __init__(self, input_channels, output_channels, num_classes, activation_fn,
dropout, output_c_multip):
        super().__init__()

        C1 = output_channels
        C2 = C1*output_c_multip
        C3 = C2*output_c_multip

        self.features = nn.Sequential(
            nn.Conv1d(input_channels, C1, 5, padding=2),
            nn.BatchNorm1d(C1),
            activation_fn(),

            nn.Conv1d(C1, C2, 5, padding=2),
            nn.BatchNorm1d(C2),
            activation_fn(),

            nn.Conv1d(C2, C3, 3, padding=1),
            nn.BatchNorm1d(C3),

```

```

        activation_fn(),
    )

    self.pool = nn.AdaptiveAvgPool1d(1)

    self.classifier = nn.Sequential(
        nn.Linear(C3, C3),
        activation_fn(),
        nn.Dropout(dropout),
        nn.Linear(C3, num_classes)
    )

    def forward(self, x):
        x = x.permute(0, 2, 1)
        x = self.features(x)
        x = self.pool(x).squeeze(-1)
        return self.classifier(x)

```

```

class LSTM(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, num_classes, dropout):
        super().__init__()

        H = hidden_size

        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=H,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout
        )

        self.head = nn.Sequential(
            nn.Linear(H, H),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(H, num_classes)
        )

    def forward(self, x):
        x, _ = self.lstm(x)
        x = x.mean(dim=1)
        return self.head(x)

class AttentionPooling(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()

```

```

        self.attn = nn.Linear(hidden_dim, 1)

    def forward(self, x):

        weights = self.attn(x)
        weights = torch.softmax(weights, dim=1)

        pooled = torch.sum(x * weights, dim=1)
        return pooled


class CNN_LSTM(nn.Module):
    def __init__(self, input_channels, output_channels, hidden_size, num_layers,
num_classes, activation_fn, dropout, output_c_multip):
        super().__init__()

        C1 = output_channels
        C2 = C1 * output_c_multip
        C3 = C2 * output_c_multip
        H = hidden_size

        self.cnn = nn.Sequential(
            nn.Conv1d(input_channels, C1, kernel_size=5, padding=2),
            nn.BatchNorm1d(C1),
            activation_fn(),
            nn.Dropout(dropout),

            nn.Conv1d(C1, C2, kernel_size=5, padding=2),
            nn.BatchNorm1d(C2),
            activation_fn(),
            nn.Dropout(dropout),

            nn.Conv1d(C2, C3, kernel_size=3, padding=1),
            nn.BatchNorm1d(C3),
            activation_fn(),

            nn.MaxPool1d(2)
        )

        self.lstm = nn.LSTM(
            input_size=C3,
            hidden_size=H,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout
        )

```

```

self.norm = nn.LayerNorm(H)

self.pool = AttentionPooling(H)

self.head = nn.Sequential(
    nn.Linear(H, H),
    nn.ReLU(),
    nn.Dropout(dropout),

    nn.Linear(H, H // 2),
    nn.ReLU(),
    nn.Dropout(dropout),

    nn.Linear(H // 2, num_classes)
)

def forward(self, x):
    # x: (B, T, F)
    x = x.permute(0, 2, 1)
    x = self.cnn(x)
    x = x.permute(0, 2, 1)

    x, _ = self.lstm(x)
    x = self.norm(x)
    x = self.pool(x)

    return self.head(x)

```

```

class CNN_Transformer(nn.Module):
    def __init__(self, input_channels, output_channels, num_layers, num_classes,
feature_dim, activation_fn, dropout, output_c_multip):
        super().__init__()

        C1 = output_channels
        C2 = C1*output_c_multip
        C3 = C2*output_c_multip

        self.cnn = nn.Sequential(
            nn.Conv1d(input_channels, C1, 5, padding=2),
            nn.BatchNorm1d(C1),
            activation_fn(),

            nn.Conv1d(C1, C2, 5, padding=2),
            nn.BatchNorm1d(C2),
            activation_fn(),

            nn.Conv1d(C2, C3, 5, padding=2),

```

```

        nn.BatchNorm1d(C3),
        activation_fn(),

        nn.MaxPool1d(2)
    )

    self.transformer = nn.TransformerEncoder(
        nn.TransformerEncoderLayer(
            d_model=C3,
            nhead=4,
            batch_first=True
        ),
        num_layers=num_layers
    )

    self.fusion = nn.Sequential(
        nn.Linear(C3 + feature_dim, 256),
        nn.GELU(),
        nn.Dropout(dropout),
        nn.Linear(256, num_classes)
    )

    def forward(self, x, x_engineered):
        x = x.permute(0, 2, 1)
        x = self.cnn(x)
        x = x.permute(0, 2, 1)

        x = self.transformer(x)
        x = x.mean(dim=1)

        x = torch.cat([x, x_engineered], dim=1)
        return self.fusion(x)

```

```

class LSTM_CNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, num_classes, activation_fn,
        dropout):
        super().__init__()

        H = hidden_size

        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=H,
            num_layers=num_layers,
            batch_first=True
        )

```

```

self.cnn = nn.Sequential(
    nn.Conv1d(H, H, 5, padding=2),
    nn.BatchNorm1d(H),
    activation_fn(),

    nn.MaxPool1d(2),

    nn.Conv1d(H, H, 3, padding=1),
    activation_fn(),
)

self.pool = nn.AdaptiveAvgPool1d(1)

self.head = nn.Sequential(
    nn.Linear(H, H),
    activation_fn(),
    nn.Dropout(dropout),
    nn.Linear(H, num_classes)
)

def forward(self, x):
    x, _ = self.lstm(x)
    x = x.permute(0, 2, 1)
    x = self.cnn(x)
    x = self.pool(x).squeeze(-1)
    return self.head(x)

class CNN_LSTM_Fusion(nn.Module):
    def __init__(self, input_channels, output_channels, hidden_size, num_layers,
num_classes, activation_fn, dropout, output_c_multip):
        super().__init__()

        C1 = output_channels
        C2 = C1 * output_c_multip
        H = hidden_size

        # CNN branch
        self.cnn = nn.Sequential(
            nn.Conv1d(input_channels, C1, 5, padding=2),
            nn.BatchNorm1d(C1),
            activation_fn(),

            nn.Conv1d(C1, C2, kernel_size=5, padding=2),
            nn.BatchNorm1d(C2),
            activation_fn(),

```

```

        nn.MaxPool1d(2),
    )

    # LSTM branch
    self.lstm = nn.LSTM(
        input_size=input_channels,
        hidden_size=H,
        num_layers=num_layers,
        batch_first=True
    )

    # fusion head
    self.classifier = nn.Sequential(
        nn.Linear(C2 + H, 96),
        activation_fn(),
        nn.Dropout(dropout),
        nn.Linear(96, num_classes)
    )

    def forward(self, x):
        # CNN
        x_cnn = x.permute(0, 2, 1)
        x_cnn = self.cnn(x_cnn)
        x_cnn = x_cnn.mean(dim=-1)

        # LSTM
        x_lstm, _ = self.lstm(x)
        x_lstm = x_lstm.mean(dim=1)

        x = torch.cat([x_cnn, x_lstm], dim=1)
        return self.classifier(x)

```

```

class MLP(nn.Module):
    def __init__(self, input_features, output_channels, num_classes, activation_fn,
        dropout, output_c_multip):
        super().__init__()

        C1 = output_channels
        C2 = C1*output_c_multip

        self.net = nn.Sequential(
            nn.Linear(input_features, C1),
            nn.LayerNorm(C1),
            activation_fn(),
            nn.Dropout(dropout),

            nn.Linear(C1, C2),

```

```

        nn.LayerNorm(C2),
        activation_fn(),
        nn.Dropout(dropout),

        nn.Linear(C2, num_classes)
    )

    def forward(self, x):
        return self.net(x)

def load_npz(path):
    path = Path(path)
    data = np.load(f"{path}.npz", allow_pickle=True)
    return data["X"], data["y"]

def load_subject(cache_dir, subject, use_raw, use_engineered):
    X_raw = None
    X_engineered = None
    y = None

    cache_dir = Path(cache_dir)
    if use_raw:
        X_raw, y = load_npz(cache_dir / subject / f"{subject}_raw")

    if use_engineered:
        X_engineered, y_engineered = load_npz(cache_dir / subject / f"{subject}_extracted")

    if y is None:
        y = y_engineered
    elif not np.array_equal(y, y_engineered):
        raise ValueError("Labels do not match")

    return X_raw, X_engineered, y

def get_subjects(cache_dir, use_raw, use_engineered):
    cache_dir = Path(cache_dir)
    subjects = set()

    if use_raw:
        subjects.update(
            p.stem.removesuffix("_raw")
            for p in Path(cache_dir).rglob("*_raw.npz")
        )

    if use_engineered:
        subjects.update(
            p.stem.removesuffix("_extracted")
            for p in Path(cache_dir).rglob("*_extracted.npz")
        )

```

```

    )

    return sorted(subjects)

def compute_accuracy(y_true, y_pred):
    return accuracy_score(y_true, y_pred)

def compute_f1_score(y_true, y_pred):
    return f1_score(
        y_true, y_pred,
        average="macro",
        zero_division=0
    )

def compute_metrics(y_true, y_pred):
    return {
        "accuracy": compute_accuracy(y_true, y_pred),
        "precision_macro": precision_score(
            y_true, y_pred,
            average="macro",
            zero_division=0
        ),
        "recall_macro": recall_score(
            y_true, y_pred,
            average="macro",
            zero_division=0
        ),
        "f1_macro": compute_f1_score(y_true, y_pred),
        "balanced_accuracy": balanced_accuracy_score(
            y_true, y_pred
        ),
        "confusion_matrix": confusion_matrix(y_true, y_pred),
        "classification_report": classification_report(
            y_true,
            y_pred,
            digits=4,
            zero_division=0,
            output_dict=True
        )
    }

def get_loader(settings, X_train_raw, X_train_extracted, y_train):
    sampler = None
    shuffle = True
    batch_size = settings["batch size"]
    if settings["weight"] == "weighted":

        class_counts = np.bincount(y_train)

```

```

weights = class_counts.sum() / class_counts

weights = weights / weights.mean()
#weights = 1.0 / class_counts

weights = torch.tensor(weights, dtype=torch.float32).to(device)

loss_fn = torch.nn.CrossEntropyLoss(weight=weights)
else:
    loss_fn = torch.nn.CrossEntropyLoss(label_smoothing = settings["label
smoothing"])

if X_train_raw is None:
    X_train_raw = torch.empty((len(y_train), 0, 0), dtype=torch.float32)

if X_train_extracted is None:
    X_train_extracted = torch.empty((len(y_train), 0), dtype=torch.float32)

return DataLoader(
    TensorDataset(X_train_raw, X_train_extracted, y_train),
    batch_size=batch_size,
    shuffle=shuffle,
    sampler=sampler,
    #num_workers=2,
    pin_memory=True,
    #prefetch_factor=2
), loss_fn

# Gradient Accumulation
def train_epoch(model, loader, optimizer, loss_fn, accumulation_steps=64):
    model.train()
    optimizer.zero_grad()
    for i, (X_batch_raw, X_batch_extracted, y_batch) in enumerate(loader):
        X_batch_raw = X_batch_raw.to(device, non_blocking=True)
        X_batch_extracted = X_batch_extracted.to(device, non_blocking=True)
        y_batch = y_batch.to(device, non_blocking=True)

        torch.compiler.cudagraph_mark_step_begin()

        with torch.amp.autocast(device_type="cuda"):
            logits = model_forward(model, X_batch_raw, X_batch_extracted)
            loss = loss_fn(logits, y_batch)
            loss = loss / accumulation_steps

        loss.backward()

        if (i + 1) % accumulation_steps == 0:
            optimizer.step()
            optimizer.zero_grad()

```

```

def validate_epoch(model, X_val_raw, X_val_extracted, y_val, metadata):

    model.eval()

    with torch.no_grad():

        X_val_raw = X_val_raw.to(device)
        X_val_extracted = X_val_extracted.to(device)
        y_val = y_val.to(device)

        logits = model_forward(
            model,
            X_val_raw,
            X_val_extracted
        )

        probs = torch.softmax(logits, dim=1)

        preds = torch.argmax(probs, dim=1)
        preds = preds.cpu().numpy()

        if metadata["eval_metric"] == EvalMetric.accuracy:
            val_metric = compute_accuracy(
                y_val.cpu(),
                preds
            )

        elif metadata["eval_metric"] == EvalMetric.F1:
            val_metric = compute_f1_score(
                y_val.cpu(),
                preds
            )
        else:
            confidences, preds = torch.max(probs, dim=1)

            mask = confidences >= 0.8

            if mask.sum() == 0:
                val_metric = 0
            else:
                coverage = mask.float().mean().item()
                precision = (
                    preds[mask] == y_val[mask]
                ).float().mean().item()

                val_metric = precision * coverage

    return val_metric

```

```

def model_forward(model, x_raw, x_extracted):

    has_raw = x_raw.numel() > 0
    has_extracted = x_extracted.numel() > 0

    if has_raw and has_extracted:
        return model(x_raw, x_extracted)

    if has_raw:
        return model(x_raw)

    if has_extracted:
        return model(x_extracted)

    raise ValueError("No input features provided")


def get_model(model_settings, X_train_main_raw, X_train_main_extracted,
y_train_main):
    match model_settings["activation_fn"]:
        case "relu":
            activation_fn = nn.ReLU
        case "gelu":
            activation_fn = nn.GELU

    dropout = model_settings["dropout"]
    hidden_size = model_settings["hidden_size"]
    num_layers = model_settings["num_layers"]
    output_channels = model_settings["output_channels"]
    output_c_multip = model_settings["output_c_multip"]

    input_channels_raw = X_train_main_raw.shape[-1]
    input_channels_extracted = X_train_main_extracted.shape[1]
    num_classes = len(torch.unique(y_train_main))

    match model_settings["model"]:
        case ModelType.CNN:
            model = CNN(
                input_channels_raw,
                output_channels,
                num_classes,
                activation_fn,
                dropout,
                output_c_multip
            ).to(device)
        case ModelType.LSTM:
            model = LSTM(
                input_channels_raw,
                hidden_size,
                num_layers,

```

```

        num_classes,
        dropout
    ).to(device)
case ModelType.CNN_LSTM:
    model = CNN_LSTM(
        input_channels_raw,
        output_channles,
        hidden_size,
        num_layers,
        num_classes,
        activation_fn,
        dropout,
        output_c_multip
    ).to(device)
case ModelType.LSTM_CNN:
    model = LSTM_CNN(
        input_channels_raw,
        hidden_size,
        num_layers,
        num_classes,
        activation_fn,
        dropout
    ).to(device)
case ModelType.CNN_Transformer:
    model = CNN_Transformer(
        input_channels_raw,
        output_channles,
        num_layers,
        num_classes,
        input_channels_extracted,
        activation_fn,
        dropout,
        output_c_multip
    ).to(device)
case ModelType.CNN_LSTM_Fusion:
    model = CNN_LSTM_Fusion(
        input_channels_raw,
        output_channles,
        hidden_size,
        num_layers,
        num_classes,
        activation_fn,
        dropout,
        output_c_multip
    ).to(device)
case ModelType.MLP:
    model = MLP(
        input_channels_extracted,
        output_channles,
        num_classes,

```

```

        activation_fn,
        dropout,
        output_c_multip
    ).to(device)

```

```

return model

```

```

def model_forward(model, x_raw, x_extracted):

```

```

    has_raw = x_raw.numel() > 0
    has_extracted = x_extracted.numel() > 0

```

```

    if has_raw and has_extracted:
        return model(x_raw, x_extracted)

```

```

    if has_raw:
        return model(x_raw)

```

```

    if has_extracted:
        return model(x_extracted)

```

```

    raise ValueError("No input features provided")

```

```

def train_loso(

```

```

    X_train_raw, X_test_raw,
    X_train_extracted, X_test_extracted,
    y_train, y_test,
    groups,
    subject_idx, test_subject,
    settings, metadata,

```

```

):

```

```

    # -----
    # Convert to CPU tensors
    # -----

```

```

    if X_train_raw is not None:
        X_train_raw = torch.tensor(X_train_raw, dtype=torch.float32)
        X_test_raw = torch.tensor(X_test_raw, dtype=torch.float32)

```

```

    if X_train_extracted is not None:
        X_train_extracted = torch.tensor(X_train_extracted, dtype=torch.float32)
        X_test_extracted = torch.tensor(X_test_extracted, dtype=torch.float32)

```

```

    y_train = torch.tensor(y_train, dtype=torch.long)

```

```

    # -----
    # Split train / validation
    # -----

```

```

    split_source = X_train_raw if X_train_raw is not None else X_train_extracted

```

```

split_np = split_source.numpy()
y_train_np = y_train.numpy()

gss = GroupShuffleSplit(
    n_splits=1,
    test_size=0.2,
    random_state=seed + subject_idx,
)

train_idx, val_idx = next(
    gss.split(split_np, y_train_np, groups)
)

assert len(groups) == len(split_np)

# -----
# Slice data
# -----
X_train_main_raw = None
X_val_raw = None

X_train_main_extracted = None
X_val_extracted = None

if X_train_raw is not None:
    X_train_main_raw = X_train_raw[train_idx]
    X_val_raw = X_train_raw[val_idx]

if X_train_extracted is not None:
    X_train_main_extracted = X_train_extracted[train_idx]
    X_val_extracted = X_train_extracted[val_idx]

y_train_main = y_train[train_idx]
y_val = y_train[val_idx]

if metadata["eval_metric"] == EvalMetric.custom:
    y_val = y_val.float()

if X_train_main_raw is None:
    X_train_main_raw = torch.empty(
        (len(y_train_main), 0, 0),
        dtype=torch.float32
    )

if X_val_raw is None:
    X_val_raw = torch.empty(
        (len(y_val), 0, 0),
        dtype=torch.float32
    )

```

```

    )

if X_train_main_extracted is None:
    X_train_main_extracted = torch.empty(
        (len(y_train_main), 0),
        dtype=torch.float32
    )

if X_val_extracted is None:
    X_val_extracted = torch.empty(
        (len(y_val), 0),
        dtype=torch.float32
    )

# -----
# Model
# -----
model = get_model(
    settings["model settings"],
    X_train_main_raw,
    X_train_main_extracted,
    y_train_main,
)

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=settings["training settings"]["learning rate"],
    weight_decay=settings["training settings"]["weight decay"],
)

loader, loss_fn = get_loader(
    settings["training settings"],
    X_train_main_raw,
    X_train_main_extracted,
    y_train_main,
)

# -----
# Training
# -----
best_val = -1
best_state = copy.deepcopy(model.state_dict())
patience_counter = 0

for epoch in range(settings["training settings"]["epoch"]):

    train_epoch(
        model,
        loader,
        optimizer,
        loss_fn,

```

```

        settings["training settings"]["accumulation_steps"],
    )

    val_score = validate_epoch(
        model,
        X_val_raw,
        X_val_extracted,
        y_val,
        metadata
    )

    if val_score > best_val:
        best_val = val_score
        best_state = copy.deepcopy(model.state_dict())
        patience_counter = 0
    else:
        patience_counter += 1

    if patience_counter >= settings["training settings"]["patience"]:
        break

print(epoch - patience_counter)

model.load_state_dict(best_state)

# -----
# Test
# -----
model.eval()

with torch.no_grad():

    if X_test_raw is None:
        X_test_raw = torch.empty((len(y_test), 0, 0))

    if X_test_extracted is None:
        X_test_extracted = torch.empty((len(y_test), 0))

    logits = model_forward(
        model,
        X_test_raw.to(device),
        X_test_extracted.to(device),
    )

    prob = torch.softmax(logits, dim=1).cpu()

    confidences, preds = torch.max(prob, dim=1)

return (
    preds.numpy(),

```

```

        confidences,
        prob,
        model,
    )

def normalize_train_test_raw(X_train, X_test, eps=1e-8):
    mean = X_train.mean(axis=(0, 1), keepdims=True)
    std = X_train.std(axis=(0, 1), keepdims=True)

    X_train = (X_train - mean) / (std + eps)
    X_test = (X_test - mean) / (std + eps)

    return X_train, X_test

def normalize_train_test_engineered(X_train, X_test, eps=1e-8):
    mean = X_train.mean(axis=0, keepdims=True)
    std = X_train.std(axis=0, keepdims=True)

    X_train = (X_train - mean) / (std + eps)
    X_test = (X_test - mean) / (std + eps)

    return X_train, X_test

def plot_confusion_matrix(cm, class_names, title="Confusion Matrix"):
    plt.figure(figsize=(7, 6))
    plt.imshow(cm, interpolation='nearest', cmap='Blues')
    plt.title(title)
    plt.colorbar()

    # set axis labels using class names
    tick_marks = np.arange(len(class_names))
    plt.xticks(tick_marks, class_names, rotation=45, ha="right")
    plt.yticks(tick_marks, class_names, rotation=45)

    plt.xlabel("Predicted label")
    plt.ylabel("True label")

    # annotate values inside cells
    thresh = cm.max() / 2.0
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(
                j, i,
                format(cm[i, j], "d"),
                ha="center",
                va="center",
                color="white" if cm[i, j] > thresh else "black"
            )

```

```

plt.tight_layout()
plt.show()

def evaluate_per_fold(per_fold_preds, per_fold_y_test):
    """
    Computes metrics per fold and plots them.

    Returns:
        fold_metrics: list of dicts (one per fold)
    """

    fold_metrics = []

    for i, item in enumerate(per_fold_preds):
        y_pred = np.array(item["pred"])
        y_true = np.array(per_fold_y_test[i])

        metrics = compute_metrics(y_true, y_pred)
        fold_metrics.append(metrics)

    # -----
    # PLOTTING (per fold)
    # -----
    folds = np.arange(len(fold_metrics))

    accs = [m["accuracy"] for m in fold_metrics]
    f1s = [m["f1_macro"] for m in fold_metrics]
    bals = [m["balanced_accuracy"] for m in fold_metrics]

    plt.figure()
    plt.plot(folds, accs, label="Accuracy")
    plt.plot(folds, f1s, label="Macro F1")
    plt.plot(folds, bals, label="Balanced Acc")

    plt.xlabel("Fold (Subject)")
    plt.ylabel("Score")
    plt.title("Per-Fold Performance")
    plt.legend()
    plt.grid()
    plt.show()

    return fold_metrics

def evaluate_final(fold_metrics, per_fold_preds, per_fold_y_test):
    """
    Aggregates metrics across all folds and plots final results.
    """

    # -----

```

```

# Aggregate scalar metrics
# -----
accs = np.array([m["accuracy"] for m in fold_metrics])
precs = np.array([m["precision_macro"] for m in fold_metrics])
recs = np.array([m["recall_macro"] for m in fold_metrics])
f1s = np.array([m["f1_macro"] for m in fold_metrics])
bals = np.array([m["balanced_accuracy"] for m in fold_metrics])

print("\n===== FINAL RESULTS =====")
print(f"Model:      {settings['model settings']['model']}")
print(f"Seed:      {seed}")
print(f"Accuracy:    {accs.mean():.4f} ± {accs.std():.4f}")
print(f"Precision (macro): {precs.mean():.4f} ± {precs.std():.4f}")
print(f"Recall (macro):  {recs.mean():.4f} ± {recs.std():.4f}")
print(f"F1 macro:     {f1s.mean():.4f} ± {f1s.std():.4f}")
print(f"Balanced Acc:   {bals.mean():.4f} ± {bals.std():.4f}")

# -----
# Global confusion matrix
# -----
all_y_true = []
all_y_pred = []

for i, item in enumerate(per_fold_preds):
    all_y_pred.extend(item["pred"])
    all_y_true.extend(per_fold_y_test[i])

class_names = ["Looking at guide book", "Assembling", "Searching for pieces"]
cm = confusion_matrix(all_y_true, all_y_pred)
print(f"Confusion matrix: {cm}")
plot_confusion_matrix(cm, class_names)

# -----
# Metric summary plot
# -----
labels = ["Acc", "Prec", "Rec", "F1", "BalAcc"]
means = [accs.mean(), precs.mean(), recs.mean(), f1s.mean(), bals.mean()]

plt.figure()
plt.bar(labels, means)
plt.ylim(0, 1)
plt.title("Average Metrics Across Folds")
plt.grid(axis="y")
plt.show()

return {
    "accuracy": float(accs.mean()),
    "precision_macro": float(precs.mean()),
    "recall_macro": float(recs.mean()),
    "f1_macro": float(f1s.mean()),

```

```

        "balanced_accuracy": float(bals.mean()),

        "accuracy_std": float(accs.std()),
        "precision_macro_std": float(precs.std()),
        "recall_macro_std": float(recs.std()),
        "f1_macro_std": float(f1s.std()),
        "balanced_accuracy_std": float(bals.std()),

        "confusion_matrix": cm.tolist()
    }

def save_model_settings(metadata, settings, cache_hash, final_results):
    path = root + "experiments"

    settingsDict = getSettingsAsDict(metadata, settings)
    experiment_type = settingsDict["experiment_type"]
    eval_metric = settingsDict["eval_metric"]
    model_type = settingsDict["model_type"]

    results_path = f"{path}/{experiment_type}/{eval_metric}/subject
independent/results/{model_type}"
    relative_model_path = f"../models/{model_type}"

    os.makedirs(results_path, exist_ok=True)

    config = {
        "settings": copy.deepcopy(settings),
        "windowing_setting_hash": cache_hash,
        "relative_model_path": relative_model_path,
    }
    config["settings"]["model settings"]["model"] = model_type

    with open(f"{results_path}config.json", "w") as f:
        json.dump(config, f, indent=2)

    with open(f"{results_path}result.json", "w") as f:
        json.dump(final_results, f, indent=2)

def getSettingsAsDict(metadata, settings):
    match metadata["experiment_type"]:
        case ExperimentType.fixed_model_params:
            experiment_type = "fixed_model_params"
        case ExperimentType.best_model:
            experiment_type = "best_model"

    match settings["model settings"]["model"]:
        case ModelType.CNN:
            model_type = "CNN"

```

```

case ModelType.LSTM:
    model_type = "LSTM"
case ModelType.CNN_LSTM:
    model_type = "CNN_LSTM"
case ModelType.LSTM_CNN:
    model_type = "LSTM_CNN"
case ModelType.CNN_LSTM_Fusion:
    model_type = "CNN_LSTM_Fusion"
case ModelType.CNN_Transformer:
    model_type = "CNN_Transformer"
case ModelType.MLP:
    model_type = "MLP"

match metadata["eval_metric"]:
    case EvalMetric.accuracy:
        eval_metric = "accuracy"
    case EvalMetric.F1:
        eval_metric = "macro_f1"
return {
    "experiment_type": experiment_type,
    "model_type": model_type,
    "eval_metric": eval_metric
}

def save_model(metadata, result, subject_id, model):
    path = root + "experiments"

    settings = getSettingsAsDict(metadata, result["settings"])
    experiment_type = settings["experiment_type"]
    eval_metric = settings["eval_metric"]
    model_type = settings["model_type"]

    models_path = f"{path}/{experiment_type}/{eval_metric}/subject
independent/models/{model_type}"
    results_path = f"{path}/{experiment_type}/{eval_metric}/subject
independent/results/{model_type}/{subject_id}"

    os.makedirs(models_path, exist_ok=True)
    os.makedirs(results_path, exist_ok=True)

    torch.save(model.state_dict(), f"{models_path}/{subject_id}.pt")

    np.save(f"{results_path}y_true.npy", result["y_true"])
    np.save(f"{results_path}y_pred.npy", result["y_pred"])
    np.save(f"{results_path}prob.npy", result["probabilities"])
    np.save(f"{results_path}conf.npy", result["confidences"])

```

```

def export_class_stats(X_embedded, y):
    classes = np.unique(y)

    output = []

    for c in classes:
        mask = y == c
        pts = X_embedded[mask]

        output.append(f"{c}:")
        output.append(f" mean = ({pts[:,0].mean():.4f}, {pts[:,1].mean():.4f})")
        output.append(f" std = ({pts[:,0].std():.4f}, {pts[:,1].std():.4f})")
        output.append("")

    return "\n".join(output)

def export_embedding_json(X_embedded, y, path=None, max_points=None):
    data = []

    n = len(X_embedded)
    idx = np.arange(n)

    # Optional subsample (important if dataset is large)
    if max_points is not None and n > max_points:
        idx = np.random.choice(idx, max_points, replace=False)

    for i in idx:
        data.append({
            "label": str(y[i]),
            "x": float(X_embedded[i, 0]),
            "y": float(X_embedded[i, 1])
        })

    json_str = json.dumps(data, indent=2)

    if path:
        with open(path, "w") as f:
            f.write(json_str)

    return json_str

def plot_pca_tsne(X, y):
    X_flat = X.mean(axis=1)
    X_scaled = StandardScaler().fit_transform(X_flat)
    pca = PCA(n_components=2)
    X_pca = pca.fit_transform(X_scaled)
    plt.figure(figsize=(8,6))

    for label in np.unique(y):
        idx = y == label

```

```

plt.scatter(X_pca[idx, 0], X_pca[idx, 1], label=str(label), s=5)

plt.legend()
plt.title("PCA projection of feature space")
plt.show()

tsne = TSNE(
    n_components=2,
    perplexity=30,
    learning_rate=200,
    init="pca",
    random_state=42
)

X_tsne = tsne.fit_transform(X_scaled)

plt.figure(figsize=(8,6))

for label in np.unique(y):
    idx = y == label
    plt.scatter(X_tsne[idx, 0], X_tsne[idx, 1], label=str(label), s=5)

plt.legend()
plt.title("t-SNE projection of feature space")
plt.show()
# print(export_embedding_json(X_tsne, y, max_points=500))
# print(export_class_stats(X_tsne, y))

settings = {
    "training settings": {
        "learning rate": 0.00006531043608504866,
        "weight decay": 0.0000900114150574730,

        "patience": 15,
        "epoch": 150,

        "weight": None,
        "label smoothing": 0.3320948079353587,
        "batch size": 65,
        "accumulation_steps": 1,
    },
    "model settings": {
        "model": ModelType.LSTM,
        "activation_fn": "gelu",
        "dropout": 0.09484922529599855,
        "output_channels": 64,
        "output_c_multip": 2,
        "hidden_size": 128,
    }
}

```

```

        "num_layers": 2,
    }
}

```

```

model_type = settings["model settings"]["model"]

```

```

use_raw = model_type in [
    ModelType.CNN,
    ModelType.LSTM,
    ModelType.CNN_LSTM,
    ModelType.LSTM_CNN,
    ModelType.CNN_Transformer,
    ModelType.CNN_LSTM_Fusion,
]

```

```

use_engineered = model_type in [
    ModelType.MLP,
    ModelType.CNN_Transformer,
]

```

```

files_suffix = "_raw" if use_raw else "_extracted"

```

```

cache_hash = "7f614e721ed6b83d1d2a0c91f5a7c0ef"
cache_dir = f"{data_path}Final Training Data/Windowed Data/{cache_hash}"

```

```

subjects = get_subjects(cache_dir, use_raw, use_engineered)

```

```

save_experiment = True
metadata = {
    "eval_metric": EvalMetric.accuracy,
    "experiment_type": ExperimentType.best_model,
}

```

```

per_fold_preds = []
per_fold_y_test = []

```

```

best_seed = 0
best_val_metric = 0
num_seed_runs = 1

```

```

if Path("best_seed.json").exists():
    with open("best_seed.json") as f:
        best_file = json.load(f)
        best_seed = best_file["seed"]
        best_val_metric = best_file["val metric"]

```

```

for i in range(num_seed_runs):

```

```

seed = 42 if num_seed_runs == 1 else random.randint(1, 10000000)
seed_all(seed)

for subject_idx, test_subject in enumerate(subjects):

    train_subjects = [s for s in subjects if s != test_subject]
    test_subject = test_subject.removesuffix(files_suffix)
    print(f"\nSubject {subject_idx}: {test_subject}")
    print("Loading data...")

    X_train_raw_list = []
    X_train_engineered_list = []
    y_train_list = []
    groups = []

    for s in train_subjects:
        s = s.removesuffix(files_suffix)
        X_s_raw, X_s_engineered, y_s = load_subject(cache_dir, s, use_raw,
use_engineered)
        if use_raw:
            X_train_raw_list.append(X_s_raw)

        if use_engineered:
            X_train_engineered_list.append(X_s_engineered)

        y_train_list.append(y_s)
        groups.extend([s] * len(X_s_raw if X_s_raw is not None else X_s_engineered))

    X_train_raw = np.concatenate(X_train_raw_list) if use_raw else None
    X_train_engineered = np.concatenate(X_train_engineered_list) if use_engineered
else None
    y_train = np.concatenate(y_train_list)
    groups = np.array(groups)

    X_test_raw, X_test_engineered, y_test = load_subject(cache_dir, test_subject,
use_raw, use_engineered)
    if use_raw:
        X_train_raw, X_test_raw = normalize_train_test_raw(
            X_train_raw,
            X_test_raw
        )

    if use_engineered:
        X_train_engineered, X_test_engineered = normalize_train_test_engineered(
            X_train_engineered,
            X_test_engineered
        )

    #plot_pca_tsne(X_train, y_train)
    per_fold_y_test.append(y_test)

```

```

print("Training...")
pred, confidences, prob, model = train_loso(
    X_train_raw, X_test_raw,
    X_train_engineered, X_test_engineered,
    y_train, y_test,
    groups,
    subject_idx, test_subject,
    settings, metadata
)

if save_experement:
    result = {
        "y_true": y_test.tolist(),
        "y_pred": pred.tolist(),

        "probabilities": prob.tolist(),
        "confidences": confidences.tolist(),

        "settings": settings,
        "windowing_setting_hash": cache_hash
    }

    save_model(metadata, result, test_subject, model)

per_fold_preds.append({
    "subject": test_subject,
    "pred": pred
})

if save_experement:
    fold_metrics = evaluate_per_fold(per_fold_preds, per_fold_y_test)

    final_results = evaluate_final(
        fold_metrics,
        per_fold_preds,
        per_fold_y_test
    )

    save_model_settings(metadata, settings, cache_hash, final_results)

fold_metrics = []

for i, item in enumerate(per_fold_preds):
    y_pred = np.array(item["pred"])
    y_true = np.array(per_fold_y_test[i])

    metrics = compute_accuracy(y_true, y_pred)
    fold_metrics.append(metrics)

```

```

if num_seed_runs != 1:
    metrics_mean = np.array(fold_metrics).mean()
    print(f"Current best seed: {best_seed}")
    print(f"Current best val metric: {best_val_metric}")
    print("")
    print(f"Current seed: {seed}")
    print(f"Current metric: {metrics_mean}")

    if best_val_metric < metrics_mean:
        best_val_metric = metrics_mean
        best_seed = seed
        with open(f"best_seed.json", "w") as f:
            json.dump({"seed": best_seed, "val metric": best_val_metric}, f, indent=2)

    print("")
    print("")
    print("")

fold_metrics = evaluate_per_fold(per_fold_preds, per_fold_y_test)

final_results = evaluate_final(
    fold_metrics,
    per_fold_preds,
    per_fold_y_test
)

```

APPLICATION H

Code for training subject-dependent models

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE
import numpy as np
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
import copy
import json
from pathlib import Path
import torch.profiler
from enum import Enum
from sklearn.model_selection import GroupShuffleSplit
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    balanced_accuracy_score,
    confusion_matrix,
    classification_report,
)
import matplotlib.pyplot as plt
import os
import random
from sklearn.model_selection import StratifiedGroupKFold

torch.set_float32_matmul_precision('high')
torch.backends.cuda.matmul.allow_tf32 = False
torch.backends.cudnn.allow_tf32 = False
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
torch.use_deterministic_algorithms(True)

def seed_all(seed=42):
    os.environ["PYTHONHASHSEED"] = str(seed)

    random.seed(seed)
    np.random.seed(seed)

    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

class ModelType(Enum):
    CNN = 1
```

```
LSTM = 2
CNN_LSTM = 3
LSTM_CNN = 4
CNN_LSTM_Fusion = 5
CNN_Transformer = 6
MLP = 7
```

```
class EvalMetric(Enum):
    accuracy = 1
    F1 = 2
    custom = 3
```

```
class ExperimentType(Enum):
    fixed_model_params = 1
    best_model = 2
```

```
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
```

```
torch.backends.cudnn.benchmark = True
```

```
print(device)
```

```
root = "../.."
data_path = f'{root}data/'
training_data_path = data_path + "Final Training Data/"
```

```
class CNN(nn.Module):
    def __init__(self, input_channels, output_channels, num_classes, activation_fn,
dropout, output_c_multip):
        super().__init__()

        C1 = output_channels
        C2 = C1*output_c_multip
        C3 = C2*output_c_multip

        self.features = nn.Sequential(
            nn.Conv1d(input_channels, C1, 5, padding=2),
            nn.BatchNorm1d(C1),
            activation_fn(),

            nn.Conv1d(C1, C2, 5, padding=2),
            nn.BatchNorm1d(C2),
            activation_fn(),

            nn.Conv1d(C2, C3, 3, padding=1),
```

```

        nn.BatchNorm1d(C3),
        activation_fn(),
    )

    self.pool = nn.AdaptiveAvgPool1d(1)

    self.classifier = nn.Sequential(
        nn.Linear(C3, C3),
        activation_fn(),
        nn.Dropout(dropout),
        nn.Linear(C3, num_classes)
    )

    def forward(self, x):
        x = x.permute(0, 2, 1)
        x = self.features(x)
        x = self.pool(x).squeeze(-1)
        return self.classifier(x)

```

```

class LSTM(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, num_classes, dropout):
        super().__init__()

        H = hidden_size

        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=H,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout
        )

        self.head = nn.Sequential(
            nn.Linear(H, H),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(H, num_classes)
        )

    def forward(self, x):
        x, _ = self.lstm(x)
        x = x.mean(dim=1)
        return self.head(x)

```

```

class AttentionPooling(nn.Module):
    def __init__(self, hidden_dim):

```

```

super().__init__()
self.attn = nn.Linear(hidden_dim, 1)

```

```

def forward(self, x):

```

```

    weights = self.attn(x)
    weights = torch.softmax(weights, dim=1)

```

```

    pooled = torch.sum(x * weights, dim=1)
    return pooled

```

```

class CNN_LSTM(nn.Module):
    def __init__(self, input_channels, output_channels, hidden_size, num_layers,
num_classes, activation_fn, dropout, output_c_multip):
        super().__init__()

```

```

        C1 = output_channels
        C2 = C1 * output_c_multip
        C3 = C2 * output_c_multip
        H = hidden_size

```

```

        self.cnn = nn.Sequential(
            nn.Conv1d(input_channels, C1, kernel_size=5, padding=2),
            nn.BatchNorm1d(C1),
            activation_fn(),
            nn.Dropout(dropout),

```

```

            nn.Conv1d(C1, C2, kernel_size=5, padding=2),
            nn.BatchNorm1d(C2),
            activation_fn(),
            nn.Dropout(dropout),

```

```

            nn.Conv1d(C2, C3, kernel_size=3, padding=1),
            nn.BatchNorm1d(C3),
            activation_fn(),

```

```

            nn.MaxPool1d(2)
        )

```

```

        self.lstm = nn.LSTM(
            input_size=C3,
            hidden_size=H,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout
        )

```

```

self.norm = nn.LayerNorm(H)

self.pool = AttentionPooling(H)

self.head = nn.Sequential(
    nn.Linear(H, H),
    nn.ReLU(),
    nn.Dropout(dropout),

    nn.Linear(H, H // 2),
    nn.ReLU(),
    nn.Dropout(dropout),

    nn.Linear(H // 2, num_classes)
)

def forward(self, x):
    # x: (B, T, F)
    x = x.permute(0, 2, 1)
    x = self.cnn(x)
    x = x.permute(0, 2, 1)

    x, _ = self.lstm(x)
    x = self.norm(x)
    x = self.pool(x)

    return self.head(x)

class CNN_Transformer(nn.Module):
    def __init__(self, input_channels, output_channels, num_layers, num_classes,
feature_dim, activation_fn, dropout, output_c_multip):
        super().__init__()

        C1 = output_channels
        C2 = C1*output_c_multip
        C3 = C2*output_c_multip

        self.cnn = nn.Sequential(
            nn.Conv1d(input_channels, C1, 5, padding=2),
            nn.BatchNorm1d(C1),
            activation_fn(),

            nn.Conv1d(C1, C2, 5, padding=2),
            nn.BatchNorm1d(C2),
            activation_fn(),

```

```

        nn.Conv1d(C2, C3, 5, padding=2),
        nn.BatchNorm1d(C3),
        activation_fn(),

        nn.MaxPool1d(2)
    )

    self.transformer = nn.TransformerEncoder(
        nn.TransformerEncoderLayer(
            d_model=C3,
            nhead=4,
            batch_first=True
        ),
        num_layers=num_layers
    )

    self.fusion = nn.Sequential(
        nn.Linear(C3 + feature_dim, 256),
        nn.GELU(),
        nn.Dropout(dropout),
        nn.Linear(256, num_classes)
    )

    def forward(self, x, x_engineered):
        x = x.permute(0, 2, 1)
        x = self.cnn(x)
        x = x.permute(0, 2, 1)

        x = self.transformer(x)
        x = x.mean(dim=1)

        x = torch.cat([x, x_engineered], dim=1)
        return self.fusion(x)

class LSTM_CNN(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, num_classes, activation_fn,
        dropout):
        super().__init__()

        H = hidden_size

        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=H,
            num_layers=num_layers,
            batch_first=True
        )

```

```

self.cnn = nn.Sequential(
    nn.Conv1d(H, H, 5, padding=2),
    nn.BatchNorm1d(H),
    activation_fn(),

    nn.MaxPool1d(2),

    nn.Conv1d(H, H, 3, padding=1),
    activation_fn(),
)

self.pool = nn.AdaptiveAvgPool1d(1)

self.head = nn.Sequential(
    nn.Linear(H, H),
    activation_fn(),
    nn.Dropout(dropout),
    nn.Linear(H, num_classes)
)

def forward(self, x):
    x, _ = self.lstm(x)
    x = x.permute(0, 2, 1)
    x = self.cnn(x)
    x = self.pool(x).squeeze(-1)
    return self.head(x)

class CNN_LSTM_Fusion(nn.Module):
    def __init__(self, input_channels, output_channels, hidden_size, num_layers,
num_classes, activation_fn, dropout, output_c_multip):
        super().__init__()

        C1 = output_channels
        C2 = C1 * output_c_multip
        H = hidden_size

        # CNN branch
        self.cnn = nn.Sequential(
            nn.Conv1d(input_channels, C1, 5, padding=2),
            nn.BatchNorm1d(C1),
            activation_fn(),

            nn.Conv1d(C1, C2, kernel_size=5, padding=2),
            nn.BatchNorm1d(C2),
            activation_fn(),

```

```

        nn.MaxPool1d(2),
    )

    # LSTM branch
    self.lstm = nn.LSTM(
        input_size=input_channels,
        hidden_size=H,
        num_layers=num_layers,
        batch_first=True
    )

    # fusion head
    self.classifier = nn.Sequential(
        nn.Linear(C2 + H, 96),
        activation_fn(),
        nn.Dropout(dropout),
        nn.Linear(96, num_classes)
    )

    def forward(self, x):
        # CNN
        x_cnn = x.permute(0, 2, 1)
        x_cnn = self.cnn(x_cnn)
        x_cnn = x_cnn.mean(dim=-1)

        # LSTM
        x_lstm, _ = self.lstm(x)
        x_lstm = x_lstm.mean(dim=1)

        x = torch.cat([x_cnn, x_lstm], dim=1)
        return self.classifier(x)

```

```

class MLP(nn.Module):
    def __init__(self, input_features, output_channels, num_classes, activation_fn,
dropout, output_c_multip):
        super().__init__()

        C1 = output_channels
        C2 = C1*output_c_multip

        self.net = nn.Sequential(
            nn.Linear(input_features, C1),
            nn.LayerNorm(C1),
            activation_fn(),
            nn.Dropout(dropout),

```

```

        nn.Linear(C1, C2),
        nn.LayerNorm(C2),
        activation_fn(),
        nn.Dropout(dropout),

        nn.Linear(C2, num_classes)
    )

    def forward(self, x):
        return self.net(x)

def load_npz(path, subject_root_path, subject):
    path = Path(path)
    data = np.load(f"{path}.npz", allow_pickle=True)
    relative_windows_times = np.load(subject_root_path / f"{subject}_window_times.npz", allow_pickle=True)
    return data["X"], data["y"], relative_windows_times["window_times"]

def load_subject(cache_dir, subject, use_raw, use_engineered):
    X_raw = None
    X_engineered = None
    y = None

    cache_dir = Path(cache_dir)
    subject_root_path = cache_dir / subject
    if use_raw:
        X_raw, y, relative_windows_times = load_npz(subject_root_path / f"{subject}_raw",
            subject_root_path, subject)

    if use_engineered:
        X_engineered, y_engineered, relative_windows_times =
load_npz(subject_root_path / f"{subject}_extracted", subject_root_path, subject)

        if y is None:
            y = y_engineered
        elif not np.array_equal(y, y_engineered):
            raise ValueError("Labels do not match")

    return X_raw, X_engineered, y, relative_windows_times

def get_subjects(cache_dir, use_raw, use_engineered):
    cache_dir = Path(cache_dir)
    subjects = set()

    if use_raw:
        subjects.update(
            p.stem.removesuffix("_raw")
            for p in Path(cache_dir).rglob("*_raw.npz")
        )

```

```

if use_engineered:
    subjects.update(
        p.stem.removesuffix("_extracted")
        for p in Path(cache_dir).rglob("**_extracted.npz")
    )

return sorted(subjects)

def compute_accuracy(y_true, y_pred):
    return accuracy_score(y_true, y_pred)

def compute_f1_score(y_true, y_pred):
    return f1_score(
        y_true, y_pred,
        average="macro",
        zero_division=0
    )

def compute_metrics(y_true, y_pred):
    return {
        "accuracy": compute_accuracy(y_true, y_pred),
        "precision_macro": precision_score(
            y_true, y_pred,
            average="macro",
            zero_division=0
        ),
        "recall_macro": recall_score(
            y_true, y_pred,
            average="macro",
            zero_division=0
        ),
        "f1_macro": compute_f1_score(y_true, y_pred),
        "balanced_accuracy": balanced_accuracy_score(
            y_true, y_pred
        ),
        "confusion_matrix": confusion_matrix(y_true, y_pred),
        "classification_report": classification_report(
            y_true,
            y_pred,
            digits=4,
            zero_division=0,
            output_dict=True
        )
    }

def get_loader(settings, X_train_raw, X_train_extracted, y_train):
    sampler = None

```

```

shuffle = True
batch_size = settings["batch size"]
if settings["weight"] == "weighted":

    class_counts = np.bincount(y_train)
    weights = class_counts.sum() / class_counts

    weights = weights / weights.mean()
    #weights = 1.0 / class_counts

    weights = torch.tensor(weights, dtype=torch.float32).to(device)

    loss_fn = torch.nn.CrossEntropyLoss(weight=weights)
else:
    loss_fn = torch.nn.CrossEntropyLoss(label_smoothing = settings["label
smoothing"])

if X_train_raw is None:
    X_train_raw = torch.empty((len(y_train), 0, 0), dtype=torch.float32)

if X_train_extracted is None:
    X_train_extracted = torch.empty((len(y_train), 0), dtype=torch.float32)

return DataLoader(
    TensorDataset(X_train_raw, X_train_extracted, y_train),
    batch_size=batch_size,
    shuffle=shuffle,
    sampler=sampler,
    #num_workers=2,
    pin_memory=True,
    #prefetch_factor=2
), loss_fn

# Gradient Accumulation
def train_epoch(model, loader, optimizer, loss_fn, accumulation_steps=64):
    model.train()
    optimizer.zero_grad()
    for i, (X_batch_raw, X_batch_extracted, y_batch) in enumerate(loader):
        X_batch_raw = X_batch_raw.to(device, non_blocking=True)
        X_batch_extracted = X_batch_extracted.to(device, non_blocking=True)
        y_batch = y_batch.to(device, non_blocking=True)

        torch.compiler.cudagraph_mark_step_begin()

        with torch.amp.autocast(device_type="cuda"):
            logits = model_forward(model, X_batch_raw, X_batch_extracted)
            loss = loss_fn(logits, y_batch)
            loss = loss / accumulation_steps

```

```

loss.backward()

if (i + 1) % accumulation_steps == 0:
    optimizer.step()
    optimizer.zero_grad()

def validate_epoch(model, X_val_raw, X_val_extracted, y_val, metadata):

    model.eval()

    with torch.no_grad():

        X_val_raw = X_val_raw.to(device)
        X_val_extracted = X_val_extracted.to(device)
        y_val = y_val.to(device)

        logits = model_forward(
            model,
            X_val_raw,
            X_val_extracted
        )

        probs = torch.softmax(logits, dim=1)

        preds = torch.argmax(probs, dim=1)
        preds = preds.cpu().numpy()

        if metadata["eval_metric"] == EvalMetric.accuracy:
            val_metric = compute_accuracy(
                y_val.cpu(),
                preds
            )

        elif metadata["eval_metric"] == EvalMetric.F1:
            val_metric = compute_f1_score(
                y_val.cpu(),
                preds
            )
        else:
            confidences, preds = torch.max(probs, dim=1)

            mask = confidences >= 0.8

            if mask.sum() == 0:
                val_metric = 0
            else:
                coverage = mask.float().mean().item()
                precision = (
                    preds[mask] == y_val[mask]
                ).float().mean().item()

```

```

        val_metric = precision * coverage

    return val_metric

def model_forward(model, x_raw, x_extracted):

    has_raw = x_raw.numel() > 0
    has_extracted = x_extracted.numel() > 0

    if has_raw and has_extracted:
        return model(x_raw, x_extracted)

    if has_raw:
        return model(x_raw)

    if has_extracted:
        return model(x_extracted)

    raise ValueError("No input features provided")

def get_model(model_settings, X_train_main_raw, X_train_main_extracted,
y_train_main):
    match model_settings["activation_fn"]:
        case "relu":
            activation_fn = nn.ReLU
        case "gelu":
            activation_fn = nn.GELU

    dropout = model_settings["dropout"]
    hidden_size = model_settings["hidden_size"]
    num_layers = model_settings["num_layers"]
    output_channels = model_settings["output_channels"]
    output_c_multip = model_settings["output_c_multip"]

    input_channels_raw = X_train_main_raw.shape[-1]
    input_channels_extracted = X_train_main_extracted.shape[1]
    num_classes = len(torch.unique(y_train_main))

    match model_settings["model"]:
        case ModelType.CNN:
            model = CNN(
                input_channels_raw,
                output_channels,
                num_classes,
                activation_fn,
                dropout,
                output_c_multip
            ).to(device)

```

```

case ModelType.LSTM:
    model = LSTM(
        input_channels_raw,
        hidden_size,
        num_layers,
        num_classes,
        dropout
    ).to(device)
case ModelType.CNN_LSTM:
    model = CNN_LSTM(
        input_channels_raw,
        output_channles,
        hidden_size,
        num_layers,
        num_classes,
        activation_fn,
        dropout,
        output_c_multip
    ).to(device)
case ModelType.LSTM_CNN:
    model = LSTM_CNN(
        input_channels_raw,
        hidden_size,
        num_layers,
        num_classes,
        activation_fn,
        dropout
    ).to(device)
case ModelType.CNN_Transformer:
    model = CNN_Transformer(
        input_channels_raw,
        output_channles,
        num_layers,
        num_classes,
        input_channels_extracted,
        activation_fn,
        dropout,
        output_c_multip
    ).to(device)
case ModelType.CNN_LSTM_Fusion:
    model = CNN_LSTM_Fusion(
        input_channels_raw,
        output_channles,
        hidden_size,
        num_layers,
        num_classes,
        activation_fn,
        dropout,
        output_c_multip
    ).to(device)

```

```

    case ModelType.MLP:
        model = MLP(
            input_channels_extracted,
            output_channles,
            num_classes,
            activation_fn,
            dropout,
            output_c_multip
        ).to(device)

    return model

def model_forward(model, x_raw, x_extracted):

    has_raw = x_raw.numel() > 0
    has_extracted = x_extracted.numel() > 0

    if has_raw and has_extracted:
        return model(x_raw, x_extracted)

    if has_raw:
        return model(x_raw)

    if has_extracted:
        return model(x_extracted)

    raise ValueError("No input features provided")

def create_subject_split(relative_times, n_blocks = 20):

    blocks = np.floor(relative_times * n_blocks).astype(int)

    return blocks

def train(
    X_raw, X_extracted, y,
    relative_windows_times,
    subject_idx, test_subject,
    settings, metadata,
):
    # -----
    # Convert to CPU tensors
    # -----
    if X_raw is not None:
        X_raw = torch.tensor(X_raw, dtype=torch.float32)

    if X_extracted is not None:

```

```

X_extracted = torch.tensor(X_extracted, dtype=torch.float32)

y = torch.tensor(y, dtype=torch.long)

# -----
# Split train / validation
# -----
split_source = X_raw if X_raw is not None else X_extracted

split_np = split_source.numpy()
y_np = y.numpy()

groups = create_subject_split(
    relative_windows_times
)

splitter = StratifiedGroupKFold(
    n_splits=5,
    shuffle=True,
    random_state=seed
)

splits = list(
    splitter.split(
        split_np,
        y_np,
        groups
    )
)

test_idx = splits[0][1]
val_idx = splits[1][1]

train_idx = np.setdiff1d(
    np.arange(len(y)),
    np.concatenate([
        test_idx,
        val_idx
    ])
)

print(
    "Train blocks:",
    np.unique(groups[train_idx])
)

print(
    "Val blocks:",
    np.unique(groups[val_idx])
)

```

```

print(
    "Test blocks:",
    np.unique(groups[test_idx])
)

assert len(set(groups[train_idx]) & set(groups[val_idx])) == 0
assert len(set(groups[train_idx]) & set(groups[test_idx])) == 0
assert len(set(groups[val_idx]) & set(groups[test_idx])) == 0
# -----
# Slice data
# -----
X_train_raw = None
X_val_raw = None
X_test_raw = None

X_train_extracted = None
X_val_extracted = None
X_test_extracted = None

if X_raw is not None:
    X_train_raw = X_raw[train_idx]
    X_val_raw = X_raw[val_idx]
    X_test_raw = X_raw[test_idx]

    mean = X_train_raw.mean(dim=(0,1), keepdim=True)
    std = X_train_raw.std(dim=(0,1), keepdim=True)

    X_train_raw = (X_train_raw - mean) / (std + 1e-8)
    X_val_raw = (X_val_raw - mean) / (std + 1e-8)
    X_test_raw = (X_test_raw - mean) / (std + 1e-8)

if X_extracted is not None:
    X_train_extracted = X_extracted[train_idx]
    X_val_extracted = X_extracted[val_idx]
    X_test_extracted = X_extracted[test_idx]

    mean = X_train_extracted.mean(dim=(0,1), keepdim=True)
    std = X_train_extracted.std(dim=(0,1), keepdim=True)

    X_train_extracted = (X_train_extracted - mean) / (std + 1e-8)
    X_val_extracted = (X_val_extracted - mean) / (std + 1e-8)
    X_test_extracted = (X_test_extracted - mean) / (std + 1e-8)

y_train = y[train_idx]
y_val = y[val_idx]
y_test = y[test_idx]

```

```

if metadata["eval_metric"] == EvalMetric.custom:
    y_val = y_val.float()

if X_train_raw is None:
    X_train_raw = torch.empty(
        (len(y_train), 0, 0),
        dtype=torch.float32
    )

if X_val_raw is None:
    X_val_raw = torch.empty(
        (len(y_val), 0, 0),
        dtype=torch.float32
    )

if X_test_raw is None:
    X_test_raw = torch.empty(
        (len(y_test), 0, 0),
        dtype=torch.float32
    )

if X_train_extracted is None:
    X_train_extracted = torch.empty(
        (len(y_train), 0),
        dtype=torch.float32
    )

if X_val_extracted is None:
    X_val_extracted = torch.empty(
        (len(y_val), 0),
        dtype=torch.float32
    )

if X_test_extracted is None:
    X_test_extracted = torch.empty(
        (len(y_test), 0),
        dtype=torch.float32
    )

print("Train", np.bincount(y_train))
print("Val", np.bincount(y_val))
print("Test", np.bincount(y_test))
# -----
# Model
# -----
model = get_model(
    settings["model settings"],
    X_train_raw,

```

```

    X_train_extracted,
    y_train,
)

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=settings["training settings"]["learning rate"],
    weight_decay=settings["training settings"]["weight decay"],
)

loader, loss_fn = get_loader(
    settings["training settings"],
    X_train_raw,
    X_train_extracted,
    y_train,
)

# -----
# Training
# -----
best_val = -1
best_state = copy.deepcopy(model.state_dict())
patience_counter = 0

for epoch in range(settings["training settings"]["epoch"]):

    train_epoch(
        model,
        loader,
        optimizer,
        loss_fn,
        settings["training settings"]["accumulation_steps"],
    )

    val_score = validate_epoch(
        model,
        X_val_raw,
        X_val_extracted,
        y_val,
        metadata
    )

    if val_score > best_val:
        best_val = val_score
        best_state = copy.deepcopy(model.state_dict())
        patience_counter = 0
    else:
        patience_counter += 1

    if patience_counter >= settings["training settings"]["patience"]:
```

```

        break

    print(epoch - patience_counter)

    model.load_state_dict(best_state)

    # -----
    # Test
    # -----
    model.eval()

    with torch.no_grad():

        logits = model_forward(
            model,
            X_test_raw.to(device),
            X_test_extracted.to(device),
        )

        prob = torch.softmax(logits, dim=1).cpu()

        confidences, preds = torch.max(prob, dim=1)

    return (
        preds.numpy(),
        confidences,
        prob,
        model,
        y_test
    )

def normalize_train_test_raw(X_train, X_test, eps=1e-8):
    mean = X_train.mean(axis=0, keepdims=True)
    std = X_train.std(axis=0, keepdims=True)

    X_train = (X_train - mean) / (std + eps)
    X_test = (X_test - mean) / (std + eps)

    return X_train, X_test

def normalize_train_test_engineered(X_train, X_test, eps=1e-8):
    mean = X_train.mean(axis=0, keepdims=True)
    std = X_train.std(axis=0, keepdims=True)

    X_train = (X_train - mean) / (std + eps)
    X_test = (X_test - mean) / (std + eps)

    return X_train, X_test

```

```

def plot_confusion_matrix(cm, class_names, title="Confusion Matrix"):
    plt.figure(figsize=(7, 6))
    plt.imshow(cm, interpolation='nearest', cmap='Blues')
    plt.title(title)
    plt.colorbar()

    # set axis labels using class names
    tick_marks = np.arange(len(class_names))
    plt.xticks(tick_marks, class_names, rotation=45, ha="right")
    plt.yticks(tick_marks, class_names, rotation=45)

    plt.xlabel("Predicted label")
    plt.ylabel("True label")

    # annotate values inside cells
    thresh = cm.max() / 2.0
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(
                j, i,
                format(cm[i, j], "d"),
                ha="center",
                va="center",
                color="white" if cm[i, j] > thresh else "black"
            )

    plt.tight_layout()
    plt.show()

def evaluate_per_fold(per_fold_preds, per_fold_y_test):
    """
    Computes metrics per fold and plots them.

    Returns:
        fold_metrics: list of dicts (one per fold)
    """

    fold_metrics = []

    for i, item in enumerate(per_fold_preds):
        y_pred = np.array(item["pred"])
        y_true = np.array(per_fold_y_test[i])

        metrics = compute_metrics(y_true, y_pred)
        fold_metrics.append(metrics)

    # -----
    # PLOTTING (per fold)
    # -----
    folds = np.arange(len(fold_metrics))

```

```

accs = [m["accuracy"] for m in fold_metrics]
f1s = [m["f1_macro"] for m in fold_metrics]
bals = [m["balanced_accuracy"] for m in fold_metrics]

plt.figure()
plt.plot(folds, accs, label="Accuracy")
plt.plot(folds, f1s, label="Macro F1")
plt.plot(folds, bals, label="Balanced Acc")

plt.xlabel("Fold (Subject)")
plt.ylabel("Score")
plt.title("Per-Fold Performance")
plt.legend()
plt.grid()
plt.show()

return fold_metrics

def evaluate_final(fold_metrics, per_fold_preds, per_fold_y_test):
    """
    Aggregates metrics across all folds and plots final results.
    """

    # -----
    # Aggregate scalar metrics
    # -----
    accs = np.array([m["accuracy"] for m in fold_metrics])
    precs = np.array([m["precision_macro"] for m in fold_metrics])
    recs = np.array([m["recall_macro"] for m in fold_metrics])
    f1s = np.array([m["f1_macro"] for m in fold_metrics])
    bals = np.array([m["balanced_accuracy"] for m in fold_metrics])

    print("\n===== FINAL RESULTS =====")
    print(f"Model:      {settings['model settings']['model']}")
    print(f"Seed:      {seed}")
    print(f"Accuracy:    {accs.mean():.4f} ± {accs.std():.4f}")
    print(f"Precision (macro): {precs.mean():.4f} ± {precs.std():.4f}")
    print(f"Recall (macro):   {recs.mean():.4f} ± {recs.std():.4f}")
    print(f"F1 macro:      {f1s.mean():.4f} ± {f1s.std():.4f}")
    print(f"Balanced Acc:   {bals.mean():.4f} ± {bals.std():.4f}")

    # -----
    # Global confusion matrix
    # -----
    all_y_true = []
    all_y_pred = []

    for i, item in enumerate(per_fold_preds):
        all_y_pred.extend(item["pred"])

```

```

all_y_true.extend(per_fold_y_test[i])

class_names = ["Looking at guide book", "Assembling", "Searching for pieces"]
cm = confusion_matrix(all_y_true, all_y_pred)
print(f"Confusion matrix: {cm}")
plot_confusion_matrix(cm, class_names)

# -----
# Metric summary plot
# -----
labels = ["Acc", "Prec", "Rec", "F1", "BalAcc"]
means = [accs.mean(), precs.mean(), recs.mean(), f1s.mean(), bals.mean()]

plt.figure()
plt.bar(labels, means)
plt.ylim(0, 1)
plt.title("Average Metrics Across Folds")
plt.grid(axis="y")
plt.show()

return {
    "accuracy": (accs.mean(), accs.std()),
    "precision_macro": (precs.mean(), precs.std()),
    "recall_macro": (recs.mean(), recs.std()),
    "f1_macro": (f1s.mean(), f1s.std()),
    "balanced_accuracy": (bals.mean(), bals.std()),
    "confusion_matrix": cm
}

def save_model_settings(metadata, settings):
    path = root + "experiments"

    settings = getSettingsAsDict(metadata, settings)
    experiment_type = settings["experiment_type"]
    eval_metric = settings["eval_metric"]
    model_type = settings["model_type"]

    results_path = f"{path}/{experiment_type}/{eval_metric}/subject
dependent/results/{model_type}"
    relative_model_path = f"../../models/{model_type}"

    os.makedirs(results_path, exist_ok=True)

    config = {
        "settings": copy.deepcopy(result["settings"]),
        "windowing_setting_hash": result["windowing_setting_hash"],
        "relative_model_path": relative_model_path
    }
    config["settings"]["model settings"]["model"] = model_type

```

```

with open(f"{results_path}config.json", "w") as f:
    json.dump(config, f, indent=2)

def getSettingsAsDict(metadata, settings):
    match metadata["experiment_type"]:
        case ExperimentType.fixed_model_params:
            experiment_type = "fixed_model_params"
        case ExperimentType.best_model:
            experiment_type = "best_model"

    match settings["model settings"]["model"]:
        case ModelType.CNN:
            model_type = "CNN"
        case ModelType.LSTM:
            model_type = "LSTM"
        case ModelType.CNN_LSTM:
            model_type = "CNN_LSTM"
        case ModelType.LSTM_CNN:
            model_type = "LSTM_CNN"
        case ModelType.CNN_LSTM_Fusion:
            model_type = "CNN_LSTM_Fusion"
        case ModelType.CNN_Transformer:
            model_type = "CNN_Transformer"
        case ModelType.MLP:
            model_type = "MLP"

    match metadata["eval_metric"]:
        case EvalMetric.accuracy:
            eval_metric = "accuracy"
        case EvalMetric.F1:
            eval_metric = "macro_f1"
    return {
        "experiment_type": experiment_type,
        "model_type": model_type,
        "eval_metric": eval_metric
    }

def save_model(metadata, result, subject_id, model):
    path = root + "experiments"

    settings = getSettingsAsDict(metadata, result["settings"])
    experiment_type = settings["experiment_type"]
    eval_metric = settings["eval_metric"]
    model_type = settings["model_type"]

    models_path = f"{path}/{experiment_type}/{eval_metric}/subject
dependent/models/{model_type}/"
    results_path = f"{path}/{experiment_type}/{eval_metric}/subject
dependent/results/{model_type}/{subject_id}/"

```

```

os.makedirs(models_path, exist_ok=True)
os.makedirs(results_path, exist_ok=True)

torch.save(model.state_dict(), f"{models_path}/{subject_id}.pt")

np.save(f"{results_path}y_true.npy", result["y_true"])
np.save(f"{results_path}y_pred.npy", result["y_pred"])
np.save(f"{results_path}prob.npy", result["probabilities"])
np.save(f"{results_path}conf.npy", result["confidences"])

# settings = {
#     "training settings": {
#         "learning rate": 0.0020698221413079114,
#         "weight decay": 0.002949983609168271,

#         "patience": 15,
#         "epoch": 150,

#         "weight": None,
#         "label smoothing": 0.33704179167039766,
#         "batch size": 64,
#         "accumulation_steps": 1,
#     },
#     "model settings": {
#         "model": ModelType.CNN,
#         "activation_fn": "gelu",
#         "dropout": 0.38368033893216336,
#         "output_channels": 32,
#         "output_c_multip": 1,
#         "hidden_size": 0,
#         "num_layers": 0,
#     }
# }

# settings = {
#     "training settings": {
#         "learning rate": 0.00011914011225325311,
#         "weight decay": 0.00006678834286562204,

#         "patience": 15,
#         "epoch": 150,

#         "weight": None,
#         "label smoothing": 0.057104690296186644,
#         "batch size": 32,
#         "accumulation_steps": 1,
#     },

```

```

# "model settings": {
#     "model": ModelType.LSTM,
#     "activation_fn": "gelu",
#     "dropout": 0.046728189117519796,
#     "output_channels": 32,
#     "output_c_multip": 1,
#     "hidden_size": 512,
#     "num_layers": 2,
# }
# }

# settings = {
#     "training settings": {
#         "learning rate": 0.00007586601541491673,
#         "weight decay": 0.00025047318545112345,

#         "patience": 15,
#         "epoch": 150,

#         "weight": None,
#         "label smoothing": 0.08960878598190943,
#         "batch size": 32,
#         "accumulation_steps": 2,
#     },
#     "model settings": {
#         "model": ModelType.LSTM_CNN,
#         "activation_fn": "gelu",
#         "dropout": 0.08987452615346886,
#         "output_channels": 32,
#         "output_c_multip": 1,
#         "hidden_size": 256,
#         "num_layers": 3,
#     }
# }

# settings = {
#     "training settings": {
#         "learning rate": 0.0000615293917160367,
#         "weight decay": 0.0003188683463381101,

#         "patience": 15,
#         "epoch": 150,

#         "weight": None,
#         "label smoothing": 0.039481001704900126,
#         "batch size": 32,
#         "accumulation_steps": 1,
#     },
#     "model settings": {
#         "model": ModelType.CNN_LSTM,

```

```

#     "activation_fn": "gelu",
#     "dropout": 0.1665073766749321,
#     "output_channels": 32,
#     "output_c_multip": 1,
#     "hidden_size": 128,
#     "num_layers": 2,
# }
# }

# settings = {
#     "training settings": {
#         "learning rate": 0.00009814222679919155,
#         "weight decay": 0.0003188683463381101,

#         "patience": 15,
#         "epoch": 150,

#         "weight": None,
#         "label smoothing": 0.8364107422752113,
#         "batch size": 256,
#         "accumulation_steps": 1,
#     },
#     "model settings": {
#         "model": ModelType.CNN_LSTM_Fusion,
#         "activation_fn": "gelu",
#         "dropout": 0.16330618785075873,
#         "output_channels": 256,
#         "output_c_multip": 1,
#         "hidden_size": 32,
#         "num_layers": 2,
#     }
# }

```

```

settings = {
    "training settings": {
        "learning rate": 0.0001761850838167857,
        "weight decay": 0.003471038530269548,

        "patience": 15,
        "epoch": 150,

        "weight": None,
        "label smoothing": 0.5166474399302594,
        "batch size": 64,
        "accumulation_steps": 1,
    },
    "model settings": {
        "model": ModelType.CNN_Transformer,
        "activation_fn": "gelu",
        "dropout": 0.7427142360089525,
    }
}

```

```

        "output_channels": 512,
        "output_c_multip": 1,
        "hidden_size": 512,
        "num_layers": 2,
    }
}

# settings = {
#     "training settings": {
#         "learning rate": 0.0007297747509706949,
#         "weight decay": 0.00016357322538998245,

#         "patience": 15,
#         "epoch": 150,

#         "weight": None,
#         "label smoothing": 0.42736297306678356,
#         "batch size": 512,
#         "accumulation_steps": 1,
#     },
#     "model settings": {
#         "model": ModelType.MLP,
#         "activation_fn": "gelu",
#         "dropout": 0.29346761726617043,
#         "output_channels": 256,
#         "output_c_multip": 1,
#         "hidden_size": 0,
#         "num_layers": 0,
#     }
# }

```

```

model_type = settings["model settings"]["model"]

```

```

use_raw = model_type in [
    ModelType.CNN,
    ModelType.LSTM,
    ModelType.CNN_LSTM,
    ModelType.LSTM_CNN,
    ModelType.CNN_Transformer,
    ModelType.CNN_LSTM_Fusion,
]

```

```

use_engineered = model_type in [
    ModelType.MLP,
    ModelType.CNN_Transformer,
]

```

```

files_suffix = "_raw" if use_raw else "_extracted"

```

```

cache_hash = "7f614e721ed6b83d1d2a0c91f5a7c0ef"
cache_dir = f"{data_path}Final Training Data/Windowed Data/{cache_hash}"

subjects = get_subjects(cache_dir, use_raw, use_engineered)

save_experiment = True
metadata = {
    "eval_metric": EvalMetric.accuracy,
    "experiment_type": ExperimentType.best_model,
}

per_fold_preds = []
per_fold_y = []

best_seed = 0
best_val_metric = 0
num_seed_runs = 1

if Path("best_seed.json").exists():
    with open("best_seed.json") as f:
        best_file = json.load(f)
        best_seed = best_file["seed"]
        best_val_metric = best_file["val metric"]

for i in range(num_seed_runs):
    seed = 42 if num_seed_runs == 1 else random.randint(1, 10000000)
    seed_all(seed)

    for subject_idx, test_subject in enumerate(subjects):

        test_subject = test_subject.removesuffix(files_suffix)
        print(f"\nSubject {subject_idx}: {test_subject}")
        print("Loading data...")

        X_train_raw_list = []
        X_train_engineered_list = []
        y_train_list = []

        X_raw, X_engineered, y, relative_windows_times = load_subject(cache_dir,
test_subject, use_raw, use_engineered)

        #plot_pca_tsne(X_train, y_train)

        print("Training...")
        pred, confidences, prob, model, y_test = train(
            X_raw, X_engineered, y,

```

```

        relative_windows_times,
        subject_idx, test_subject,
        settings, metadata
    )

    per_fold_y.append(y_test)

    if save_experement:
        result = {
            "y_true": y_test.tolist(),
            "y_pred": pred.tolist(),

            "probabilities": prob.tolist(),
            "confidences": confidences.tolist(),

            "settings": settings,
            "windowing_setting_hash": cache_hash
        }

        save_model(metadata, result, test_subject, model)

    per_fold_preds.append({
        "subject": test_subject,
        "pred": pred
    })

    if save_experement:
        save_model_settings(metadata, settings)

    fold_metrics = []

    for i, item in enumerate(per_fold_preds):
        y_pred = np.array(item["pred"])
        y_true = np.array(per_fold_y[i])

        metrics = compute_accuracy(y_true, y_pred)
        fold_metrics.append(metrics)

    if num_seed_runs != 1:
        metrics_mean = np.array(fold_metrics).mean()
        print(f"Current best seed: {best_seed}")
        print(f"Current best val metric: {best_val_metric}")
        print("")
        print(f"Current seed: {seed}")
        print(f"Current metric: {metrics_mean}")

    if best_val_metric < metrics_mean:
        best_val_metric = metrics_mean
        best_seed = seed

```

```

        with open(f"best_seed.json", "w") as f:
            json.dump({"seed": best_seed, "val metric": best_val_metric}, f, indent=2)

    print("")
    print("")
    print("")

def export_class_stats(X_embedded, y):
    classes = np.unique(y)

    output = []

    for c in classes:
        mask = y == c
        pts = X_embedded[mask]

        output.append(f"{c}:")
        output.append(f"  mean = ({pts[:,0].mean():.4f}, {pts[:,1].mean():.4f})")
        output.append(f"  std  = ({pts[:,0].std():.4f}, {pts[:,1].std():.4f})")
        output.append("")

    return "\n".join(output)

def export_embedding_json(X_embedded, y, path=None, max_points=None):
    data = []

    n = len(X_embedded)
    idx = np.arange(n)

    # Optional subsample (important if dataset is large)
    if max_points is not None and n > max_points:
        idx = np.random.choice(idx, max_points, replace=False)

    for i in idx:
        data.append({
            "label": str(y[i]),
            "x": float(X_embedded[i, 0]),
            "y": float(X_embedded[i, 1])
        })

    json_str = json.dumps(data, indent=2)

    if path:
        with open(path, "w") as f:
            f.write(json_str)

    return json_str

def plot_pca_tsne(X, y):

```

```

X_flat = X.mean(axis=1)
X_scaled = StandardScaler().fit_transform(X_flat)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
plt.figure(figsize=(8,6))

for label in np.unique(y):
    idx = y == label
    plt.scatter(X_pca[idx, 0], X_pca[idx, 1], label=str(label), s=5)

plt.legend()
plt.title("PCA projection of feature space")
plt.show()

```

```

tsne = TSNE(
    n_components=2,
    perplexity=30,
    learning_rate=200,
    init="pca",
    random_state=42
)

X_tsne = tsne.fit_transform(X_scaled)

plt.figure(figsize=(8,6))

for label in np.unique(y):
    idx = y == label
    plt.scatter(X_tsne[idx, 0], X_tsne[idx, 1], label=str(label), s=5)

plt.legend()
plt.title("t-SNE projection of feature space")
plt.show()
# print(export_embedding_json(X_tsne, y, max_points=500))
# print(export_class_stats(X_tsne, y))

```

```

fold_metrics = evaluate_per_fold(per_fold_preds, per_fold_y)

```

```

final_results = evaluate_final(
    fold_metrics,
    per_fold_preds,
    per_fold_y
)

```

APPLICATION I

DataHandler.py

```
import os
```

```
import pandas as pd
```

```
import json
```

```
from PyQt6.QtWidgets import QFileDialog
```

```
import globals
```

```
from models import Event
```

```
def load_settings():
```

```
    try:
```

```
        with open(globals.vidDetails.root + "settings.json", "r", encoding="utf-8") as file:
```

```
            data = json.load(file)
```

```
            offset_x = data["offset_x"]
```

```
            offset_y = data["offset_y"]
```

```
            overlay_delay = data["overlay_delay"]
```

```
            return offset_x, offset_y, overlay_delay
```

```
    except FileNotFoundError:
```

```
        print("Settings file does not exist")
```

```
    return 0,0,0
```

```
def load_data():
```

```
    """
```

```
        Lets the user select a folder.
```

```
        Goes through each sub folder in the selected folder and finds the gaze data as well as
```

```
        it searches for labels that may have been created before that, and loads them.
```

```
        It does the same for the workspace video feed.
```

```
    """
```

```
    events: list[Event]= []
```

```

dir = QFileDialog.getExistingDirectory(None, "Choose folder")

if dir == "":
    raise RuntimeError

found_video = False
found_gaze = False

for entry in os.scandir(dir):
    if not entry.is_dir():
        continue

    if entry.name.find("_Data") != -1:
        found_video = True

        path = dir + "/" + entry.name + "/"
        globals.vidDetails.root = path

        gaze_df = pd.read_csv(path + "gaze.csv")
        print(f"Loading {path}gaze.csv")

        globals.vidDetails.recording_id = gaze_df["recording id"][0]
        gaze_df = gaze_df.dropna(subset=["gaze x [px]", "gaze y [px]"])

        gaze_df["timestamp [ms]"] = gaze_df["timestamp [ns]"] / 1_000_000
        gaze_df["timestamp [ms]"] -= gaze_df["timestamp [ms]"].iloc[0]

        globals.vidDetails.gaze_df = gaze_df.sort_values("timestamp
[ms]").reset_index(drop=True)
        globals.vidDetails.gaze_timestamp = gaze_df["timestamp [ms]"].values

    if os.path.exists(path + "labels.csv"):
        globals.vidDetails.output_path = path + "labels.csv"

```

```

labels_file = pd.read_csv(globals.vidDetails.output_path)
labels_arr = labels_file.to_numpy()

start_time = 0
end_time = 0

for row in labels_arr:

    start_time = row[1]
    end_time = row[2]
    label = row[3]
    event = Event(start_time, end_time, label)

    events.append(event)

elif entry.name.find("_Raw") != -1:
    found_gaze = True
    globals.vidDetails.path = dir + "/" + entry.name + "/Neon Scene Camera v1
ps1.mp4"
else:
    print(f"No relevant information found in {entry.path}")

if not found_gaze:
    raise FileNotFoundError("Gaze data have not been found.")

if not found_video:
    raise FileNotFoundError("Video data have not been found.")

return events

def save_df_to_csv(df: pd.DataFrame, override: bool = False):
    path = globals.vidDetails.output_path

    if os.path.exists(path):
        df.to_csv(path, mode = "w" if override else "a", index=False)

```

else:

```
df.to_csv(path, index=False)
```

def save_settings(settings):

```
with open(globals.vidDetails.root + "settings.json", "w", encoding="utf-8") as file:
```

```
    json.dump(settings, file, indent=4)
```

APPLICATION J

EventHandler.py

```
from models import Event, CurrentEvent
import utils
from DataHandler import save_df_to_csv

import pandas as pd
from PyQt6.QtCore import pyqtSignal, QObject

class EventHandler(QObject):
    """
    Handles all CRUD actions for events

    Attributes:
        current_event: The event that is currently selected.
        current_event_changed: Event, fired when the current event have changed.
        events_change_requested: Event, fired when the array "events" have changed.
    """

    current_event_changed = pyqtSignal(str)
    events_change_requested = pyqtSignal(int)

    current_event = CurrentEvent(0)

    def __init__(self, events: list[Event]):
        super().__init__()
        self.__events = events

    @property
    def events(self):
        return self.__events
```

```

@events.setter
def events(self, value):
    self.__export_events()
    self.__events = value

def update_current_event_label(self, new_label: str):

    old_event = self.events[self.current_event.index]
    new_event = Event(old_event.start_time, old_event.end_time, new_label)
    self.events[self.current_event.index] = new_event
    self.set_current_event(self.current_event.index)

    self.__export_events()
    self.events_change_requested.emit(self.current_event.start_time)

def delete_current_event(self):

    self.__delete_event(self.current_event.index)

def add_event(self, label: str, end_time: int, delay: int):
    """
    Adds new event

    Args:
        label: The label of the new event.
        end_time: The end time of the event.
        delay: The delay used to display the gaze.
    """

    # First event ever
    if len(self.events) == 0:
        start = 0
    else:
        start = self.current_event.end_time

```

```

end = end_time

if start == end_time:
    raise ValueError("The start time cannot be the same as the end time.")

# Save event
event = Event(start, end, label)
self.events.append(event)

end = max(0, end - delay)

if start <= end:
    end += 1

self.__export_events()

# Update state
self.set_current_event(self.current_event.index + 1)
self.events_change_requested.emit(self.current_event.end_time)

def split_event(self, time: int):
    """
    Splits an event to two events.

    The first event have the start time of the current event and the end time of the
    "time" argument.

    The second event have the start time of the "time" argument and the end time of
    the current event.

    Args:
        time: The time at which the event will be split.
    """

    if self.current_event.start_time > time or self.current_event.end_time < time:
        raise ValueError("The time have to be between the start and the end time of the
        current event.")

```

```

new_event = Event(self.current_event.start_time, time, self.current_event.label)

self.events[self.current_event.index].start_time = time
self.events.insert(self.current_event.index, new_event)

self.set_current_event(self.current_event.index + 1)
self.events_change_requested.emit(self.current_event.start_time)

def change_start_time(self, time: int):
    if self.current_event.end_time < time:
        raise ValueError("The time have to be smaller then the end time of the current
event.")

    # Change the start time of the current event
    self.events[self.current_event.index].start_time = time

    # Check if there is another event before that one.
    # Check if the new start time is smaller then the start time of the last event.
    # And if so, delete the last event. Do this for all events before the current one.
    index_to_check = self.current_event.index
    while True:
        if index_to_check > 0:
            index_to_check = self.current_event.index - 1

            prev_event = self.events[index_to_check]
            if prev_event.start_time < time:
                prev_event.end_time = time

                break
        else:
            self.__delete_event(index_to_check, self.current_event.index)
            index_to_check -= 1
    else:
        break

```

```

self.events_change_requested.emit(self.current_event.start_time)

def change_end_time(self, time: int):
    if self.current_event.start_time > time:
        raise ValueError("The time have to be smaller then the start time of the current
event.")

    # Change the end time of the current event
    self.events[self.current_event.index].end_time = time

    # Check if there is another event after that one.
    # Check if the new end time is bigger then the end time of the next event.
    # And if so, delete the next event. Do this for all events after the current one.
    index_to_check = self.current_event.index
    while True:
        if index_to_check < len(self.events) - 1:
            index_to_check = self.current_event.index + 1

            next_event = self.events[index_to_check]
            if next_event.end_time > time:
                next_event.start_time = time

                break
            else:
                self.__delete_event(index_to_check, self.current_event.index)
                index_to_check += 1
        else:
            break

    self.events_change_requested.emit(self.current_event.start_time)

def set_current_event(self, index: int):
    """

```

Sets the current event to the event with the given index.

Args:

index: The index of the event that is to be set as the current one.

"""

try:

idx = utils.validate_list_index(index, self.events)

except IndexError as e:

print(e)

return

event = self.events[idx]

self.current_event = CurrentEvent(idx, event = event)

self.current_event_changed.emit(self.current_event.label)

def __delete_event(self, index: int, new_current_event_idx = -1):

"""

Deletes the event with the given index.

Args:

index: The index of the event that is to be deleted.

new_current_event_idx: The index of the event that is to become the current event. If not set, the current event becomes the last one.

"""

try:

idx = utils.validate_list_index(index, self.events)

except IndexError as e:

print(e)

```

        return

    event_to_del = self.events[idx]

    self.events.remove(event_to_del)

    self.__export_events()
    if new_current_event_idx == -1:
        self.set_current_event(self.current_event.index - 1)
    else:
        try:
            idx = utils.validate_list_index(new_current_event_idx, self.events)
        except IndexError as e:
            print(e)
            return

        self.set_current_event(new_current_event_idx)

    self.events_change_requested.emit(self.current_event.end_time)

def __export_events(self):
    df = pd.DataFrame([
        {
            "recording id": event.recording_id,
            "start timestamp [ms]": event.start_time,
            "end timestamp [ms]": event.end_time,
            "label": event.label,
        }
        for event in self.events
    ])

    save_df_to_csv(df, True)

```

APPLICATION K

LabelingApp.py

```
from PyQt6.QtWidgets import QWidget, QGraphicsView, QGraphicsScene, QLabel,
QComboBox, QPushButton, QMessageBox
```

```
from PyQt6.QtMultimediaWidgets import QGraphicsVideoItem
```

```
from PyQt6.QtGui import QShortcut, QKeySequence
```

```
from PyQt6.QtCore import QUrl, Qt
```

```
import numpy as np
```

```
from classes.TimelineWidget import TimelineWidget
```

```
from classes.MainWindowUI import MainWindowUI
```

```
from classes.EventHandler import EventHandler
```

```
from classes.VideoPlayer import VideoPlayer
```

```
from classes.TimeMachine import TimeMachine
```

```
from globals import vidDetails as globVidDetails
```

```
import DataHandler
```

```
import const
```

```
class LabelingApp(QWidget):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.setWindowTitle("Eye Tracking Labeling Tool")
```

```
        # ----- MEDIA -----
```

```
        self.player = VideoPlayer()
```

```
        self.scene = QGraphicsScene()
```

```
        self.view = QGraphicsView(self.scene)
```

```
        self.video_item = QGraphicsVideoItem()
```

```
        self.scene.addItem(self.video_item)
```

```

self.player.setVideoOutput(self.video_item)

self.scene.addItem(self.player.dot)

# ----- LABELS -----
self.event_buttons:dict[str, QPushButton] = {}

# ----- OFFSETS -----
self.offset_x = 0
self.offset_y = 0
self.video_render_delay_ms = 0

# ----- DATA LOADING -----
data_loaded = False

while not data_loaded:
    try:
        events = DataHandler.load_data()
        self.offset_x, self.offset_y, self.video_render_delay_ms =
DataHandler.load_settings()
        data_loaded = True
    except FileNotFoundError as e:
        QMessageBox.critical(self, "File Not Found", str(e))
    except RuntimeError:
        raise RuntimeError

# ----- TIMELINE -----
self.timeline = TimelineWidget()
self.timeline.seek_requested.connect(self.player.setPosition)
self.timeline.set_events(events)

# ----- UI -----
self.time_label: QLabel = None
self.offset_label_X: QLabel = None
self.offset_label_Y: QLabel = None

```

```

self.render_delay_label: QLabel = None
self.combo_box: QComboBox = None

self.ui = MainWindowUI()
self.ui.setup_ui(self)

# ----- EVENTS -----
self.event_handler = EventHandler(events)

self.event_handler.current_event_changed.connect(self.on_current_event_changed)

self.event_handler.events_change_requested.connect(self.on_events_change_request
ed)

self.player.setSource(QUrl.fromLocalFile(globVidDetails.path))

events_len = len(events)
if events_len > 1:
    self.event_handler.set_current_event(events_len - 1)

self.time_machine = TimeMachine(events, self.event_handler.current_event)

# ----- SIGNALS -----
self.player.positionChanged.connect(self.update_timeline_text)
self.player.positionChanged.connect(self.update_overlay)
self.player.durationChanged.connect(self.on_timeline_duration_changed)
self.video_item.nativeSizeChanged.connect(self.fit_video)

# ----- SHORTCUTS -----
self.init_shortcuts()

# Shortcuts
def init_shortcuts(self):
    QShortcut(QKeySequence("K"),
self).activated.connect(self.event_handler.delete_current_event)

```

```

        QShortcut(QKeySequence("E"), self).activated.connect(lambda:
self.player.jump_to_time(self.event_handler.current_event.end_time))

        QShortcut(QKeySequence("Q"), self).activated.connect(lambda:
self.player.jump_to_time(self.event_handler.current_event.start_time))

        QShortcut(QKeySequence("S"), self).activated.connect(self.prev_event)

        QShortcut(QKeySequence("W"), self).activated.connect(self.next_event)

        QShortcut(QKeySequence("A"), self).activated.connect(lambda: self.player.seek(-
const.skip_amount))

        QShortcut(QKeySequence("D"), self).activated.connect(lambda:
self.player.seek(const.skip_amount))

        QShortcut(QKeySequence("Space"),
self).activated.connect(self.player.toggle_play)

        QShortcut(QKeySequence("Shift+A"), self).activated.connect(lambda:
self.player.seek(-const.skip_amount * 3))

        QShortcut(QKeySequence("Ctrl+Z"), self).activated.connect(self.undo_event)

        QShortcut(QKeySequence("Ctrl+Y"), self).activated.connect(self.redo_event)

        QShortcut(QKeySequence("Ctrl+S"), self).activated.connect(self.split_event)

        QShortcut(QKeySequence("Ctrl+A"),
self).activated.connect(self.change_event_start_time)

        QShortcut(QKeySequence("Ctrl+D"),
self).activated.connect(self.change_event_end_time)


    for idx, label in enumerate(const.events):

        QShortcut(QKeySequence(f"{idx+1}"), self).activated.connect(lambda
checked=False, label=label: self.handle_add_event(label))


# Event actions
def first_event(self):
    self.event_handler.set_current_event(0)
    self.player.jump_to_time(self.event_handler.current_event.start_time)


def last_event(self):
    self.event_handler.set_current_event(len(self.event_handler.events) - 1)
    self.player.jump_to_time(self.event_handler.current_event.start_time)


def next_event(self):
    self.event_handler.set_current_event(self.event_handler.current_event.index + 1)
    self.player.jump_to_time(self.event_handler.current_event.start_time)

```

```

def prev_event(self):
    self.event_handler.set_current_event(self.event_handler.current_event.index - 1)
    self.player.jump_to_time(self.event_handler.current_event.start_time)

# Event handlers
def fit_video(self):
    rect = self.video_item.boundingRect()
    self.scene.setSceneRect(rect)
    self.view.fitInView(rect, Qt.AspectRatioMode.KeepAspectRatio)

def update_timeline_text(self, position):
    total_seconds = position // 1000
    minutes = total_seconds // 60
    seconds = total_seconds % 60

    self.time_label.setText(f"{position} ms | {minutes}:{seconds:02d}")
    self.timeline.set_position(position)

def on_timeline_duration_changed(self, duration):
    self.timeline.set_duration(duration)

def on_current_event_changed(self, label: str):
    self.combo_box.setCurrentText(label)

def on_events_change_requested(self, jump_time: int):
    self.time_machine.take_snapshot(self.event_handler.events,
    self.event_handler.current_event)

    self.timeline.set_events(self.event_handler.events)
    self.player.jump_to_time(jump_time)

## settings
def on_offset_X_changed(self, value: int):

```

```

self.offset_x = value
self.offset_label_X.setText(f"Offset X: {value}px")
self.update_overlay(self.player.position())

self.save_settings()

def on_offset_Y_changed(self, value: int):
    self.offset_y = value
    self.offset_label_Y.setText(f"Offset Y: {value}px")
    self.update_overlay(self.player.position())

    self.save_settings()

def on_render_delay_changed(self, value: int):
    self.video_render_delay_ms = value
    self.render_delay_label.setText(f"Overlay render delay: {value}ms")
    self.update_overlay(self.player.position())

    self.save_settings()

def save_settings(self):
    settings = {
        "offset_x": self.offset_x,
        "offset_y": self.offset_y,
        "overlay_delay": self.video_render_delay_ms
    }
    DataHandler.save_settings(settings)

# Overlay
def update_overlay(self, t: int):

    x, y = self.interpolate_gaze(t)

    x += self.offset_x
    y += self.offset_y

```

```

rect = self.video_item.boundingRect()

x = x / const.video_width * rect.width()
y = y / const.video_height * rect.height()

self.player.dot.setPos(x, y)

def interpolate_gaze(self, t: int):
    """
    Interpolates between the current gaze point and the next gaze point in order to
    lessen the sensor noise.
    """
    timestamps = globVidDetails.gaze_timestamp

    # Render delay may be added in order to better aline the gaze data over the video.
    t = max(0, t - self.video_render_delay_ms)
    idx = np.searchsorted(
        timestamps,
        t,
        side="right"
    ) - 1

    idx = np.clip(idx, 0, len(timestamps) - 2)

    t1 = timestamps[idx]
    t2 = timestamps[idx + 1]

    alpha = 0 if t2 == t1 else (t - t1) / (t2 - t1)

    df = globVidDetails.gaze_df

    x = np.interp(
        alpha,
        [0, 1],

```

```

        [df.iloc[idx]["gaze x [px]"], df.iloc[idx + 1]["gaze x [px]"]]
    )

    y = np.interp(
        alpha,
        [0, 1],
        [df.iloc[idx]["gaze y [px]"], df.iloc[idx + 1]["gaze y [px]"]]
    )

    return x, y

# EVENTS
def handle_add_event(self, label: str):
    t = self.player.position()

    # Update events
    try:
        self.event_handler.add_event(label, t, self.video_render_delay_ms)
    except ValueError as e:
        QMessageBox.warning(self, "End time not valid", str(e))
    return

# UI feedback
for _, btn in self.event_buttons.items():
    btn.setStyleSheet("")
self.event_buttons[label].setStyleSheet(
    "background-color:#e74c3c;color:white;font-weight:bold;"
)
self.timeline.set_events(self.event_handler.events)

def undo_event(self):
    self.event_handler.events, current_event = self.time_machine.undo()
    self.event_handler.set_current_event(current_event.index)

    self.timeline.set_events(self.event_handler.events)

```

```

self.player.jump_to_time(self.event_handler.current_event.start_time)

def redo_event(self):
    self.event_handler.events, current_event = self.time_machine.redo()
    self.event_handler.set_current_event(current_event.index)

    self.timeline.set_events(self.event_handler.events)
    self.player.jump_to_time(self.event_handler.current_event.start_time)

def split_event(self):
    time = self.player.position()

    try:
        self.event_handler.split_event(time)
    except ValueError as e:
        QMessageBox.warning(self, "Split time not valid", str(e))

def change_event_start_time(self):
    time = self.player.position()

    try:
        self.event_handler.change_start_time(time)
    except ValueError as e:
        QMessageBox.warning(self, "Start time not valid", str(e))

def change_event_end_time(self):
    time = self.player.position()

    try:
        self.event_handler.change_end_time(time)
    except ValueError as e:
        QMessageBox.warning(self, "End time not valid", str(e))

```

APPLICATION L

MainWindowUI.py

```
from PyQt6.QtWidgets import (
    QSlider, QPushButton, QVBoxLayout,
    QHBoxLayout, QLineEdit, QLabel, QComboBox
)
from PyQt6.QtCore import Qt

import const

class MainWindowUI():
    def setup_ui(self, parent):
        layout = QHBoxLayout()

        video_layout = QVBoxLayout()
        video_layout.addWidget(parent.view)

        # Time label
        parent.time_label = QLabel("0 ms | 0 s")
        parent.time_label.setAlignment(Qt.AlignmentFlag.AlignCenter)

        video_layout.addWidget(parent.time_label)

        # Timeline
        video_layout.addWidget(parent.timeline)

        layout.addLayout(video_layout, 3)

        UI_layout = QVBoxLayout()
        UI_layout.setAlignment(Qt.AlignmentFlag.AlignBottom)
        UI_layout.setSpacing(10)

        # Combo box
        combo_box_layout = QHBoxLayout()
```

```

combo_box_UI = QHBoxLayout()

parent.combo_box = QComboBox()
parent.combo_box.activated.connect(lambda _:
parent.eventHandler.update_current_event(parent.combo_box.currentText()))

for label in const.events:
    parent.combo_box.addItem(label)

combo_box_UI.addWidget(parent.combo_box)

combo_box_layout.addLayout(combo_box_UI)
UI_layout.addLayout(combo_box_layout)

# Jump

jump_layout = QHBoxLayout()
jump_UI = QHBoxLayout()

time_input = QLineEdit()
time_input.setPlaceholderText("Enter ms")
btn_jump = QPushButton("Jump")
btn_jump.clicked.connect(lambda: parent.player.jump_to_time(time_input.text()))

jump_UI.addWidget(QLabel("Go:"))
jump_UI.addWidget(time_input)
jump_UI.addWidget(btn_jump)

jump_layout.addLayout(jump_UI)
UI_layout.addLayout(jump_layout)

# Offsets

offset_layout = QHBoxLayout()
offset_UI = QVBoxLayout()

```

```

offset_X_UI = QVBoxLayout()
offset_Y_UI = QVBoxLayout()
render_delay_UI = QVBoxLayout()

parent.offset_label_X = QLabel(f"Offset X: {parent.offset_x}px")
parent.offset_label_X.setAlignment(Qt.AlignmentFlag.AlignCenter)
parent.offset_label_Y = QLabel(f"Offset Y: {parent.offset_y}px")
parent.offset_label_Y.setAlignment(Qt.AlignmentFlag.AlignCenter)
parent.render_delay_label = QLabel(f"Overlay render delay:
{parent.video_render_delay_ms}ms")
parent.render_delay_label.setAlignment(Qt.AlignmentFlag.AlignCenter)

x_slider = self.make_slider()
x_slider.setValue(parent.offset_x)

y_slider = self.make_slider()
y_slider.setValue(parent.offset_y)

delay_slider = self.make_slider()
delay_slider.setValue(parent.video_render_delay_ms)
delay_slider.setMaximum(1000)
delay_slider.setMinimum(-1000)

x_slider.valueChanged.connect(parent.on_offset_X_changed)
y_slider.valueChanged.connect(parent.on_offset_Y_changed)
delay_slider.valueChanged.connect(parent.on_render_delay_changed)

offset_X_UI.addWidget(parent.offset_label_X)
offset_X_UI.addWidget(x_slider)
offset_Y_UI.addWidget(parent.offset_label_Y)
offset_Y_UI.addWidget(y_slider)
render_delay_UI.addWidget(parent.render_delay_label)
render_delay_UI.addWidget(delay_slider)

offset_UI.addLayout(offset_X_UI)

```

```

offset_UI.addLayout(offset_Y_UI)
offset_UI.addLayout(render_delay_UI)

offset_layout.addLayout(offset_UI)
UI_layout.addLayout(offset_layout)

# controls
controls_layout = QHBoxLayout()
controls_UI = QVBoxLayout()

controls_first_last_event_UI = QHBoxLayout()

btn_first_event = QPushButton("Go to first event")
btn_first_event.clicked.connect(parent.first_event)

btn_last_event = QPushButton("Go to last event")
btn_last_event.clicked.connect(parent.last_event)

controls_first_last_event_UI.addWidget(btn_first_event)
controls_first_last_event_UI.addWidget(btn_last_event)

controls_UI.addLayout(controls_first_last_event_UI)

controls_layout.addLayout(controls_UI)
UI_layout.addLayout(controls_layout)

# Labels
label_layout = QHBoxLayout()
labels_UI = QHBoxLayout()

for label in const.events:
    btn = QPushButton(label)
    btn.setFixedHeight(100)

    btn.clicked.connect(lambda _, l=label: parent.handle_add_event(l))

```

```
labels_UI.addWidget(btn)
parent.event_buttons[label] = btn

label_layout.addLayout(labels_UI)
UI_layout.addLayout(label_layout)

layout.addLayout(UI_layout, 1)

parent.setLayout(layout)

def make_slider(self):
    s = QSlider(Qt.Orientation.Horizontal)
    s.setRange(-200, 200)
    s.setSingleStep(1)
    return s
```

APPLICATION M

TimelineWidget.py

```
from PyQt6.QtWidgets import QWidget
from PyQt6.QtCore import pyqtSignal
from PyQt6.QtGui import QColor, QPainter
```

```
import const
from models import Event
```

```
class TimelineWidget(QWidget):
```

```
    """
```

Creates a timeline Bar that shows the progress of the video, as well as,
the position and length of the different events in different colors.

Attributes:

 seek_requested: Event, fired when the user press on the timeline in order to move
the video to another point.

```
    """
```

```
    seek_requested = pyqtSignal(int)
```

```
    def __init__(self):
        super().__init__()
```

```
        self.duration = 1
        self.position = 0
        self.events: list[Event] = []
```

```
        self.setMinimumHeight(40)
```

```
    def set_duration(self, duration: int):
        self.duration = max(1, duration)
        self.update()
```

```

def set_position(self, pos: int):
    self.position = pos
    self.update()

def set_events(self, events: list[Event]):
    self.events = events
    self.update()

def mousePressEvent(self, e):
    ratio = e.position().x() / self.width()
    self.seek_requested.emit(int(ratio * self.duration))

def paintEvent(self, _):
    """Paints each event over the timeline based on there position and duration."""
    painter = QPainter(self)
    w, h = self.width(), self.height()

    painter.fillRect(0, 0, w, h, QColor("#2b2b2b"))

    # events
    for event in self.events:
        color = const.events.get(event.label, QColor("white"))

        x1 = int((event.start_time / self.duration) * w)
        x2 = int((event.end_time / self.duration) * w)

        painter.fillRect(x1, 0, max(1, x2 - x1), h, color)

    # playhead
    x = int((self.position / self.duration) * w)
    painter.setPen(QColor("white"))
    painter.fillRect(x, 0, 1, h, QColor("white"))

```

APPLICATION N

TimeMachine.py

```
from models import Event, CurrentEvent
```

```
import utils
```

```
import copy
```

```
class Snapshot:
```

```
    def __init__(self, events: list[Event], last_event: CurrentEvent):
```

```
        """
```

```
        The snapshot represents all events and the selected event, in a given moment.
```

```
        Args:
```

```
            events: List with all of the events in the given moment.
```

```
            last_event: The information of the event that is selected in the given moment.
```

```
        """
```

```
        self.events = copy.deepcopy(events)
```

```
        self.last_event = copy.deepcopy(last_event)
```

```
class TimeMachine:
```

```
    """
```

```
    Enables undo and redo commands on the events.
```

```
    Attributes:
```

```
        time_machine: Stores all of the taken snapshots.
```

```
        max_snapshots: The maximum number of snapshots that can be stored.
```

```
    """
```

```
    time_machine: list[Snapshot] = []
```

```
    max_snapshots = 10
```

```
    def __init__(self, events: list[Event], last_event: CurrentEvent, max_snapshots = 10):
```

```
        snapshot = Snapshot(events, last_event)
```

```
        self.time_machine.append(snapshot)
```

```

self.snapshot_index = utils.validate_list_index(0, self.time_machine)
self.max_snapshots = max_snapshots

def undo(self):
    self.snapshot_index = utils.validate_list_index(self.snapshot_index - 1,
self.time_machine)
    snapshot = self.time_machine[self.snapshot_index]
    snapshot_copy = copy.deepcopy(snapshot)
    return snapshot_copy.events, snapshot_copy.last_event

def redo(self):
    self.snapshot_index = utils.validate_list_index(self.snapshot_index + 1,
self.time_machine)
    snapshot = self.time_machine[self.snapshot_index]
    snapshot_copy = copy.deepcopy(snapshot)
    return snapshot_copy.events, snapshot_copy.last_event

def take_snapshot(self, events: list[Event], current_event: CurrentEvent):
    index = 0 if len(self.time_machine) == 0 else self.snapshot_index + 1
    self.time_machine = self.time_machine[:index]

    self.time_machine[len(self.time_machine) - 1].last_event = current_event

    snapshot = Snapshot(events, current_event)
    self.time_machine.append(snapshot)

    if len(self.time_machine) > self.max_snapshots:
        self.time_machine.pop(0)
    else:
        self.snapshot_index += 1

```

APPLICATION O

VideoPlayer.py

```
from PyQt6.QtMultimedia import QMediaPlayer
from PyQt6.QtWidgets import QGraphicsEllipseItem, QMessageBox
from PyQt6.QtGui import QBrush, QColor, QPen

class VideoPlayer(QMediaPlayer):

    def __init__(self):
        super().__init__()

        self.dot = QGraphicsEllipseItem(0, 0, 10, 10)
        self.dot.setBrush(QBrush(QColor(255, 0, 0)))
        self.dot.setPen(QPen(QColor(255, 0, 0)))
        self.dot.setZValue(1)

    def toggle_play(self):
        if self.playbackState() == QMediaPlayer.PlaybackState.PlayingState:
            self.pause()
        else:
            self.play()

    # ----- Jump -----
    def jump_to_time(self, time: int):
        try:
            value = int(time)
            self.setPosition(value)
        except ValueError as e:
            QMessageBox.warning(self, "The input must be an integer", str(e))
            print("Invalid timestamp")

    def seek(self, time: int):
        self.setPosition(self.position() + time)
```